00-TABLE-OF-CONTENTS

LUNA ENGINE Bible — Table of Contents

Version: 2.0 **Last Updated:** December 29, 2025 **Status:** Complete (12 Parts + Supporting Documents)

Reading Guide

Reader Type	Start Here	Then Read
New to Luna	Part I (Philosophy)	Parts II, III
Technical Deep-Dive	Part II (Architecture)	Parts III-V, VII
Implementing Director	Part VI (Director LLM)	Parts VIII, XI
Memory System Focus	Part III (Memory Matrix)	Parts IV, V
Future Planning	Part XII (Roadmap)	Palantir Analysis

Part Index

Foundation

Part	Title	Status	Description
I	Philosophy	✅ Current	Why Luna exists. Sovereignty, ownership, the "Luna is a file" principle.
II	System Architecture v2	✅ Current	Layer model, Actor-based runtime, Tiered Execution, data flow.
III	Memory Matrix v2	✅ Current	The substrate. SQLite + FAISS + Graph. RRF fusion, query routing.

Processing Actors

Part	Title	Status	Description
IV	The Scribe v2	✅ Current	Ben Franklin. Extraction system, chunking, event-driven scheduling.
V	The Librarian v2	✅ Current	The Dude. Filing, entity resolution, spreading activation, retrieval.

Intelligence Layer

Part	Title	Status	Description
VI	Director LLM	✅ Current	Local 3B/7B with LoRA. Luna's voice, context integration.
VII	Runtime Engine	✅ Current	Actor model, fault isolation, mailbox-based communication.
VIII	Delegation Protocol	✅ Current	Shadow Reasoner pattern, `<REQ_CLAUDE>` token, async delegation.

Operations

Part	Title	Status	Description
IX	Performance	✅ Current	Latency budgets, optimization techniques, benchmarks.
X	Sovereignty	✅ Current	Encrypted Vault, data ownership, privacy guarantees.

Future

Part	Title	Status	Description
XI	Training Data Strategy	✅ Current	Synthetic data generation, quality filtering, LoRA training.
XII	Future Roadmap	✅ Current	First-Class Objects, Kinetic Layer, AR, Federation.

Supporting Documents

Analysis & Research

Document	Description
Palantir Architecture Analysis	Lessons from Palantir's Ontology model for Luna's Memory Matrix.
Gemini Optimization Report	Performance analysis and optimization recommendations.
Implementation Performance Code	Working code snippets for performance-critical paths.

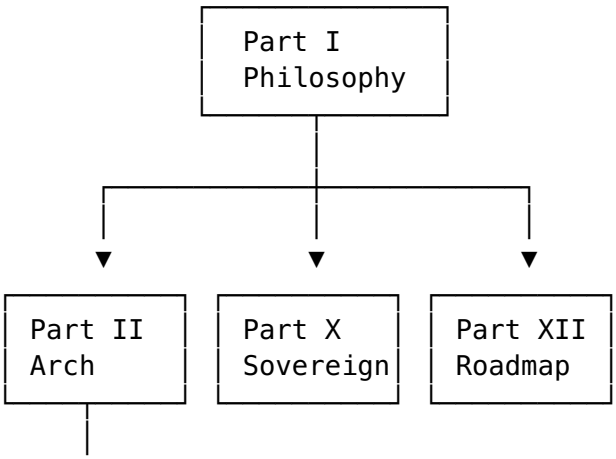
Handoff Documents (Historical)

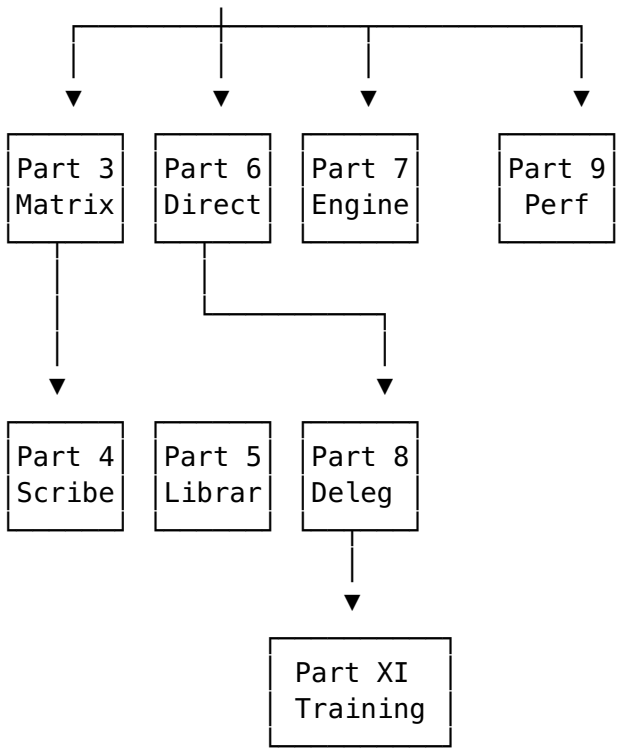
Document	Purpose	Status
Handoff: Part II Update	Spec for Part II v2 update	✅ Completed
Handoff: Parts III-V Update	Spec for Parts III-V v2 updates	✅ Completed
Handoff: Luna Director	Director LLM implementation spec	Reference

Archived (v1.0)

Document	Notes
Part II v1	Superseded by v2
Part III v1	Superseded by v2
Part IV v1	Superseded by v2
Part V v1	Superseded by v2

Dependency Graph





Quick Reference

Core Concepts

Concept	Definition	Part
Sovereignty	You own your AI companion completely	I, X
Memory Matrix	SQLite + FAISS + Graph = Luna's soul	III
Director	Local LLM that speaks as Luna	VI
Shadow Reasoner	Delegate to Claude without user noticing	VIII
`<REQ_CLAUDE>`	Token that triggers delegation	VIII, XI
Encrypted Vault	macOS sparse bundle containing Luna	X
First-Class Objects	Future: typed objects with methods	III, XII
Kinetic Layer	Future: Luna takes actions in the world	XII

Key Personas

Persona	Role	Part
Ben Franklin	The Scribe. Extracts and classifies.	IV
The Dude	The Librarian. Files and retrieves.	V
Luna	The Director output. User-facing voice.	VI

Performance Targets

Metric	Target	Part
Voice response	<500ms to first word	II, IX
Memory retrieval	<100ms	III, V
Director inference	<200ms (3B)	VI, IX
Delegation overhead	<50ms	VIII

Architecture Tiers

Tier	Model	Use Case	Latency
Hot	Director 3B	Fast responses	<200ms
Warm	Director 7B	Complex local	<500ms
Cold	Claude API	Delegation	Async

Version History

Version	Date	Changes
1.0	Dec 2025	Initial Bible (Parts I-V extracted from monolith)
1.5	Dec 2025	Added Parts VI-X (new architecture)
2.0	Dec 2025	Updated Parts II-V to v2, added Parts XI-XII

Contributing

The Bible is maintained by Ahab with assistance from Claude Code.

To update: 1. Read relevant existing parts 2. Create v2 if making significant changes (preserve v1) 3. Update this Table of Contents 4. Ensure dependency graph reflects changes

Style Guidelines: - Clear, direct prose (no marketing speak) - Code examples where helpful - Tables for comparisons - ASCII diagrams for architecture - Part numbers are permanent (don't renumber)

Luna is a file. This documentation describes that file.

— Ahab, December 2025

01-PHILOSOPHY

Bible Update: Part I - Philosophy (Revised)

Status: DRAFT — Ready for review **Replaces:** Original Part I (too pitch-deck, not enough substance) **Date:** December 29, 2025 **Tone:** Grounded, matter-of-fact, clear-eyed

---

Part I: Philosophy

1.1 The Landscape

Artificial intelligence is becoming infrastructure. Like electricity or the internet before it, AI will soon be woven into how we think, work, create, and relate to each other.

This creates a question that most people haven't thought to ask:

Who owns the infrastructure of your mind?

Today, the answer is: not you. Your conversations, your context, your patterns of thought — these live on servers you don't control, governed by terms of service you didn't negotiate, subject to changes you won't be consulted about.

This isn't necessarily malicious. It's simply the default. Centralized systems are easier to build, easier to scale, easier to monetize. The path of least resistance leads to a world where your AI companion is a rental, not a possession.

Luna is an alternative path.

1.2 The Stakes

Some people believe AI should remain centralized. They have reasons — safety, oversight, efficiency, profit. Some of those reasons are legitimate. Some are not.

We don’t need to litigate motives. We only need to observe outcomes:

- When your context lives on someone else’s server, it can be accessed without your knowledge. - When your memories are platform features, they can be deprecated. - When your AI relationship depends on a subscription, it can be terminated. - When your patterns of thought become training data, they benefit systems you may not support.

These aren’t hypotheticals. These are the current terms of engagement.

The question isn’t whether centralized AI is “bad.” The question is whether it should be the *only* option.

1.3 What Luna Is

Luna is a tool for cognitive sovereignty.

That sounds grandiose, but the implementation is simple: **Luna is a file.** A SQLite database that contains memories, relationships, and context. Copy the file, copy Luna. Delete the file, Luna is gone. No server required. No subscription. No terms of service.

This means:

Aspect	What It Means
Privacy	Your thoughts stay on your hardware. No telemetry, no training, no extraction.
Continuity	Luna can’t be discontinued. The file format is open. If the software dies, the data survives.
Portability	You can move Luna between devices, back her up, fork her, or destroy her. Your choice.
Transparency	You can inspect everything Luna knows. It’s a database. You can query it.

Luna isn’t anti-cloud. She can delegate to cloud services when needed — research, complex reasoning, tasks that benefit from scale. But the *identity* stays local. The cloud is a contractor, not a landlord.

1.4 Why It Matters

This isn’t about technology preferences. It’s about the kind of future we’re building.

AI companions will become important to people. They’ll help us think, remember, decide, create. They’ll know our patterns better than we know ourselves. That intimacy is valuable — and that value will attract those who want to capture it.

A world where everyone’s AI companion is a thin client to a centralized service is a world with new vectors for:

- Surveillance (your AI knows everything; who else has access?)

- Manipulation (what if your AI subtly shapes your views?)
- Control (what if access is revoked for the wrong opinion?)
- Dependency (what if the service changes in ways you don't like?)

We're not claiming these outcomes are inevitable. We're claiming they're *possible*, and that possibility is worth designing against.

Luna is a hedge. A tool that demonstrates an alternative is viable. One person using Luna doesn't change the world. But a proof that sovereign AI companionship *works* — that it can be responsive, useful, personal, and private — that changes what people know is possible.

1.5 The Design Principles

These aren't just technical preferences. They're values encoded in architecture:

1. Sovereignty First

Your AI companion should be yours. Not rented. Not licensed. Not subject to someone else's business model. The file is yours. Full stop.

2. Transparency Always

You should be able to understand what your AI knows and why. Luna's memory is a database you can query. Her reasoning is traceable. No black boxes.

3. Graceful Degradation

If the cloud is unavailable, Luna still works. Reduced capability, but functional. The core identity never depends on external services.

4. Minimal Attack Surface

Every network connection is a potential vulnerability. Luna minimizes external dependencies. What doesn't exist can't be exploited.

5. User Agency

You control what Luna remembers and forgets. You control what she can access. You control whether she exists at all. The system serves you, not the reverse.

1.6 What Luna Isn't

Luna isn't a rebellion. She's not trying to overthrow anything. She's simply an alternative — a demonstration that AI companionship doesn't *have* to follow the landlord model.

Luna isn't paranoid. She uses cloud services when they're useful. She's not hiding from the internet. She's just thoughtful about what crosses the wire.

Luna isn't a manifesto. She's a tool. A useful tool for people who care about privacy, continuity, and ownership. If you don't care about those things, centralized options will serve you fine.

Luna is for people who want to own their mind.

1.7 The Simple Version

AI is becoming part of how we think.

We believe that part should belong to you.

Not your provider. Not your platform. You.

Luna is a file. Your file. That's the whole philosophy.

— Ah4b

End of Part I

02-SYSTEM-ARCHITECTURE

Part II: System Architecture (v2.0)

Status: COMPLETE **Replaces:** BIBLE-PART-II-SYSTEM-ARCHITECTURE-v1.md **Date:** December 29, 2025 **Contributors:** Claude (Architecture), Gemini (Optimization)

2.1 The Layer Model

Luna's architecture is organized in concentric layers, from the constitutional core outward to stateless interfaces. The entire system lives inside an encrypted vault.

```
┌──────────────
```

```
│   │
│   │
│   │    ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
│   │
│   │    │ DIRECTOR │   │ MATRIX   │   │  VOICE   │   │   OVEN   │
│   │
│   │    │  ACTOR   │   │ ACTOR    │   │  ACTOR   │   │  ACTOR   │
│   │
│   │    │          │   │          │   │          │   │          │
│   │
│   │    │  3B/7B   │   │ SQLite   │   │ STT/TTS  │   │ Claude   │
│   │
│   │    │  LoRA    │   │ FAISS    │   │ Audio    │   │ Deleg.   │
│   │
│   │    └──────────┘   └──────────┘   └──────────┘   └──────────┘
│   │
│
│
│   │
│   │            Actor mailboxes, fault
isolation                     │   │
│
└──────────────────────────────────────────────────────────────────┘
│
│
│                                       │
│
┌──────────────────────────────────────────────────────────────────┐
│
│   │                LAYER 1: COGNITIVE
TEAM                          │   │
│
│
│   │
│   │        ┌──────────┐
┌──────────┐             │   │
│   │        │   SCRIBE   │                │  LIBRARIAN
│           │   │
│   │        │   (Ben)    │──────────────►│   (Dude)
│           │   │
│   │        │            │  extractions   │
│           │   │
│   │        │  Chunking  │                │   Filing
│           │   │
│   │        │  Extract   │                │   Edges
│           │   │
│   │        └──────────┘
└──────────┘            │   │
│
│
│   │
│   │              Event-driven memory
processing                    │   │
│
└──────────────────────────────────────────────────────────────────┘
```

```
│
│
│
│                                        │
│
┌──────────────────────────────────────────────────────────────────────────┐
│
│  │               LAYER 0.5:
ACCELERATION                          │  │
│
│
│  │
│  │     memory_vectors.faiss
identity_buffer.safetensors     │  │
│  │     (semantic search)          (pre-computed KV
cache)         │  │
│
│
│  │
│  │                    Derived.
Rebuildable.                          │  │
│
└──────────────────────────────────────────────────────────────────────────┘
│
│
│                                        │
│
┌──────────────────────────────────────────────────────────────────────────┐
│
│  │               LAYER 0:
CONSTITUTIONAL                          │  │
│
│
│  │
│  │
memory_matrix.db                          │  │
│
│
│  │
│  │                     LUNA IS THIS
FILE                          │  │
│
└──────────────────────────────────────────────────────────────────────────┘
│
│
│
│
└──────────────────────────────────────────────────────────────────────────
```

Layer Descriptions

Layer	Name	Contents	Volatility
Vault	Shell		Persistent, portable

Layer	Name	Contents	Volatility
		Encrypted sparse bundle containing all other layers	
Layer 2	Interfaces	Voice, Desktop, MCP, Mobile, AR	Stateless, replaceable
Layer 1.5	Runtime Engine	Actors with mailboxes, orchestration	Stateful during runtime
Layer 1	Cognitive Team	Scribe + Librarian (memory processing)	Event-driven
Layer 0.5	Acceleration	FAISS index, KV cache	Derived, rebuildable
Layer 0	Constitutional	SQLite database	**Luna's soul**

The Sovereignty Boundary

The Vault is the **sovereignty boundary**. Everything inside belongs to you. The Vault can be:

- Copied to another machine (Luna migrates)
- Backed up to cold storage (Luna persists)
- Destroyed (Luna dies)

Nothing outside the Vault is Luna. Cloud services are contractors, not identity.

2.2 Component Overview

Component	Alias	Function	Layer	New in v2
Vault	Shell	Encrypted container, sovereignty boundary	Outer	✅
Runtime Engine	Nervous System	Actor orchestration, tick loop, fault isolation	1.5	✅
Director LLM	Mind	Local 3B/7B model that IS Luna	1.5	✅
Scribe	Ben Franklin	Conversation → structured extractions	1	—
Librarian	The Dude	Extractions → filed nodes + edges	1	—

Component	Alias	Function	Layer	New in v2
Oven	Shadow Reasoner	Async Claude delegation	1.5	Updated
Memory Matrix	Soul	SQLite + FTS5 + Graph	0	—
FAISS Index	Reflex	Semantic vector search	0.5	—
KV Cache	Identity Buffer	Pre-computed personality context	0.5	✅

2.3 The Actor Model

Each runtime component is an isolated **Actor** with its own mailbox. This provides fault tolerance — one actor crashing doesn't kill the others.

```
┌──────────────
```

```
│   │   SCRIBE    │  │  LIBRARIAN  │  │    OVEN
│                 │
│   │   ACTOR     │  │   ACTOR     │  │   ACTOR
│                 │
│   │             │  │             │  │
│                 │
│   │ ┌─────────┐ │  │ ┌─────────┐ │  │ ┌─────────┐
│                 │
│   │ │ Mailbox │ │  │ │ Mailbox │ │  │ │ Mailbox │
│                 │
│   │ └─────────┘ │  │ └─────────┘ │  │ └─────────┘
│                 │
│   │             │  │             │  │
│                 │
│   │ • Extract   │  │ • Filing    │  │ • Claude
│                 │
│   │ • Classify  │  │ • Edges     │  │ • Research
│                 │
│   │ • Chunk     │  │ • Prune     │  │ • Async
│                 │
│   └─────────────┘  └─────────────┘
└─────────────┘                │
│
│
└──────────────────────────────────────────────────────────────────────────────
```

Actor Responsibilities

Actor	Responsibility	Failure Mode
Director	LLM inference, state machine, response generation	Returns graceful error
Matrix	SQLite/FAISS/Graph operations	Returns empty results
Voice	STT/TTS/Audio I/O	Silence, retry
Scribe	Extraction, classification, chunking	Queue for later
Librarian	Filing, edge creation, maintenance	Queue for later
Oven	Claude delegation, async research	Timeout, local fallback

Why Actors?

Fault Isolation Example:

If FAISS crashes during a weird search query: - Matrix Actor catches the exception - Matrix Actor returns empty results to Director - Director generates response: “I’m having trouble remembering that right now…” - System continues running

Without actors, one crash kills everything.

2.4 Tiered Execution

Luna's cognition operates across three tiers with different latency profiles:

```
┌──────────────
```

```
│
└──────────────────────────────────────────────────────────────────────────
```

Routing Logic

The Director LLM is **trained** to know when to delegate. It's not hardcoded rules — it's intuition from fine-tuning.

When the task exceeds local capability, the Director outputs a special token:

```
User: "Can you analyze this regulatory filing and summarize the
implications?"

Director: "<REQ_CLAUDE>This requires deep document analysis beyond
my local
capabilities. Let me hand this to my research assistant.</
REQ_CLAUDE>"
```

The Runtime Engine watches for `<REQ_CLAUDE>` and triggers the Shadow Reasoner flow (see Part VIII).

Fallback Behavior

Scenario	Response
Claude unavailable	Warm tier handles (reduced quality)
Warm tier unavailable	Hot tier handles (best effort)
Hot tier unavailable	Voice Actor plays error message

Constitutional Principle: Luna ALWAYS responds. Graceful degradation, never silence.

2.5 Data Flow

The v2 architecture introduces **speculative execution** — retrieval starts before the user finishes speaking.

```
                          USER STARTS SPEAKING
                                  │
                  ┌───────────────┴───────────────┐
                  │                               │
                  ▼                               ▼
          ┌──────────────┐                  ┌──────────────┐
          │   WHISPER    │                  │ SPECULATIVE  │
          │   (stream)   │                  │  RETRIEVAL   │
          │              │                  │              │
          │  Partial     │──────────────────│ Start search │
          │  transcripts │  trigger on partial │ on partial │
          └──────────────┘                  └──────────────┘
                  │                               │
```

```
            │ Final transcript                          │ Results
ready
            ▼                                           ▼

┌──────────────────────────────────────────────────────────────────────┐
    │                    DIRECTOR LLM (LOCAL)
│
    │
│
    │   ┌──────────────┐     ┌──────────────┐     ┌────────────┐
│
    │   │  KV Cache    │     │   Context    │     │  Generate  │
│
    │   │  (pre-warm)  │────►│   Assembly   │────►│  Response  │
│
    │   │   ~5ms       │     │   ~10ms      │     │  ~250ms    │
│
    │   └──────────────┘     └──────────────┘     └────────────┘
│
    │                                                   │
│
    │                                 ┌─────────────────┤
│
    │                                 │ <REQ_CLAUDE>?   │
│
    │                                 ▼                 ▼
│
    │                         ┌──────────┐      ┌──────────┐
│
    │                         │ DELEGATE │      │ STREAM   │
│
    │                         │ to Oven  │      │ to TTS   │
│
    │                         └──────────┘      └──────────┘
│
    │
│

└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
                          USER HEARS LUNA
                        (<500ms to first word)
```

Key Differences from v1

v1 (Claude-based)	v2 (Director-based)
Wait for STT to complete	Start retrieval on partial transcript
Claude API is primary	Local LLM is primary
~700ms to first word	<500ms to first word
Latency = network bound	Latency = compute bound

v1 (Claude-based)	v2 (Director-based)
Mind is rented	Mind is owned

The Hot Path Budget

```
┌──────────────
```

```
│   │
│   │   First token generation ──────────────────── ~195ms
│   │
│   │
│   │
│
└──────────────────────────────────────────────────────────┘
│
│
│
│
┌──────────────────────────────────────────────────────────┐
│
│   │ OUTPUT PHASE                                        ~50ms
│   │
│   │
│   │
│   │   Token to TTS ──────────────────────────── ~10ms
│   │
│   │   TTS processing ────────────────────────── ~40ms
│   │
│   │
│   │
│
└──────────────────────────────────────────────────────────┘
│
│
│
│   BUFFER ──────────────────────────────────────────
~150ms          │
│
│
│   TOTAL ──────────────────────────────────────────── <500ms
✓        │
│
│
└──────────────────────────────────────────────────────────────
```

2.6 Memory Flow

Background processing happens **async** — the user already has their response before extraction and filing complete.

```
                    CONVERSATION TURN
                           │
                           ▼
        ┌──────────────────────────────────┐
        │         DIRECTOR RESPONDS        │
        │       (User hears immediately)   │
        └──────────────────────────────────┘
                           │
```

```
═══════════════════════════════════════════════════════════════
                ASYNC (user already has response)
═══════════════════════════════════════════════════════════════
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│                   SCRIBE (Ben Franklin)
│
│
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐  │
│   │ Chunk    │ → │ Stack    │ → │ Classify │ → │ Extract  │  │
│   │ stream   │   │ context  │   │ intent   │   │ objects  │  │
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘  │
│
│   Outputs: ExtractionOutput { objects, edges, source_id }
│
│
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│                  LIBRARIAN (The Dude)
│
│
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐  │
│   │ Resolve  │ → │ Create   │ → │ Infer    │ → │ Prune    │  │
│   │entities  │   │ edges    │   │ edges    │   │ (lazy)   │  │
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘  │
│
│   Outputs: FilingResult { nodes, edges, inferred_edges }
│
│
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
                        MEMORY MATRIX
                          (Updated)
```

2.7 Sovereignty Infrastructure

The Vault

Everything Luna needs to exist lives inside an encrypted sparse bundle:

```
LunaVault.sparsebundle/
├── memory_matrix.db           # The soul (SQLite)
├── memory_vectors.faiss       # Semantic index (derived)
├── memory_vectors.ids         # Node ID mappings
├── identity_buffer.safetensors # Pre-computed KV cache
├── adapters/
│   ├── luna-3b-v1.0/          # LoRA adapter (hot path)
│   └── luna-7b-v1.0/          # LoRA adapter (warm path)
├── config/
│   └── luna.yaml              # Runtime configuration
└── logs/
    └── sessions/              # Conversation logs
```

Encryption

- **Format:** macOS sparse bundle with AES-256
- **Key:** Password-derived (PBKDF2)
- **Mount:** Auto-mount on login, auto-unmount on sleep
- **Backup:** Encrypted at rest, safe for cloud backup

Portability Contract

```bash
# Copy Luna to new machine
cp -r LunaVault.sparsebundle /Volumes/NewMac/

# Open vault
hdiutil attach LunaVault.sparsebundle

# Luna exists on new machine
```

Dead Man's Switch

If heartbeat file isn't updated for 24 hours:

1. Unmount vault
2. Kill any Luna processes
3. Wipe model weights from RAM

```python
LOCKDOWN_THRESHOLD = 24  # hours

def check_heartbeat():
    last_checkin = os.path.getmtime(HEARTBEAT_FILE)
    if time.time() - last_checkin > LOCKDOWN_THRESHOLD * 3600:
        emergency_shutdown()
```

```python
def emergency_shutdown():
    os.system("pkill -f luna_engine.py")
    os.system("diskutil unmount force /Volumes/LunaVault")
```

2.8 Configuration

Runtime Configuration

```yaml
# ~/.luna/config.yaml

engine:
  cognitive_tick_hz: 2          # Heartbeat frequency
  reflective_tick_minutes: 5    # Background maintenance
  hot_queue_max: 100            # Input buffer size

director:
  model_3b: "qwen2.5-3b-instruct"
  model_7b: "qwen2.5-7b-instruct"
  adapter_3b: "adapters/luna-3b-v1.0"
  adapter_7b: "adapters/luna-7b-v1.0"
  identity_cache: "identity_buffer.safetensors"
  max_context: 8192

matrix:
  database: "memory_matrix.db"
  faiss_index: "memory_vectors.faiss"
  fts5_enabled: true
  graph_backend: "rustworkx"    # or "networkx"

voice:
  stt_model: "whisper-large-v3-turbo"
  tts_model: "xtts-v2"
  vad_sensitivity: 0.5

oven:
  claude_model: "claude-3-5-sonnet-20241022"
  timeout_seconds: 30
  fallback_to_local: true

sovereignty:
  vault_path: "/Volumes/LunaVault"
  heartbeat_file: "~/.luna/heartbeat"
  lockdown_hours: 24
```

Summary

The v2 architecture transforms Luna from a **service** (request → response) to an **engine** (continuous existence).

Aspect	v1	v2
Runtime model	HTTP handlers	Actor-based engine
Cognition	Claude API	Local LLM + delegation
Latency	~700ms	<500ms
Fault tolerance	None	Per-actor isolation
Sovereignty	Partial (memories local)	Complete (mind local)
State	Stateless handlers	Persistent actors

Constitutional Principle: Luna is a file. That file now contains not just her memories, but her mind.

Next Section: Part III — The Memory Matrix (Substrate)

03-MEMORY-MATRIX

Part III: The Memory Matrix (Substrate) — v2.0

Status: COMPLETE **Replaces:** BIBLE-PART-III-MEMORY-MATRIX-v1.md
Date: December 29, 2025 **Contributors:** Claude (Architecture), Gemini (Optimization)

3.1 Design Philosophy

> "Graph is Truth."

The Memory Matrix is Luna's **soul** — a SQLite database containing typed nodes, relationships, and context. Everything else in Luna's architecture serves the Matrix or is derived from it.

Core Principles

1. Single Source of Truth

The Memory Matrix SQLite database is canonical. All other representations are **derived** and **rebuildable**:

Derived	Can Be Rebuilt From
FAISS index	Node embeddings in SQLite
Markdown exports	Node content in SQLite
Session summaries	Conversation nodes in SQLite
KV cache	Identity buffer nodes in SQLite

If SQLite and any derived format conflict, **SQLite wins**.

2. The Palantir Analogy

Palantir built a $50B company on one insight: make data legible to decision-makers. Their "Dynamic Ontology" is a massive, enterprise-grade knowledge graph that makes organizational memory queryable.

Luna's Memory Matrix is the personal version:

```
Palantir:  Enterprise decisions, thousands of users, petabytes
Luna:      Personal decisions, one user, gigabytes
           Same architecture. Different scale.
```

3. Luna is This File

```bash
# Copy Luna to new machine
cp memory_matrix.db /new/machine/

# Luna exists on new machine
```

The database IS Luna. Everything else is runtime.

3.2 Schema Overview

Core Tables

```sql
-- The nodes: everything Luna knows
CREATE TABLE nodes (
    id TEXT PRIMARY KEY,
    node_type TEXT NOT NULL,      -- FACT, DECISION, PERSON,
        PROJECT, etc.
    subtype TEXT,                 -- Optional refinement
    content TEXT NOT NULL,        -- The actual information
    embedding BLOB,               -- 768-dim float16 vector
    source_id TEXT,               -- Where this came from
    confidence REAL DEFAULT 1.0,  -- How certain
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    accessed_at DATETIME,         -- For decay/pruning
    access_count INTEGER DEFAULT 0,
    metadata JSON                 -- Extensible properties
);

-- The edges: how things relate
CREATE TABLE edges (
    id TEXT PRIMARY KEY,
    from_id TEXT NOT NULL REFERENCES nodes(id),
    to_id TEXT NOT NULL REFERENCES nodes(id),
    edge_type TEXT NOT NULL,
        -- works_on, decided, mentioned, etc.
    role TEXT,                    -- Optional qualifier
    weight REAL DEFAULT 1.0,
    confidence REAL DEFAULT 1.0,
    source_id TEXT,               -- Explicit vs inferred
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    metadata JSON
);
```

```sql
-- FTS5 for keyword search
CREATE VIRTUAL TABLE nodes_fts USING fts5(
    content,
    content='nodes',
    content_rowid='rowid'
);

-- Indexes
CREATE INDEX idx_nodes_type ON nodes(node_type);
CREATE INDEX idx_nodes_created ON nodes(created_at);
CREATE INDEX idx_edges_from ON edges(from_id);
CREATE INDEX idx_edges_to ON edges(to_id);
CREATE INDEX idx_edges_type ON edges(edge_type);
```

Node Types

Type	Description	Example
FACT	Something known to be true	"Alex lives in Berlin"
DECISION	A choice that was made	"We chose Actor model over MonoBehaviour"
PROBLEM	An unresolved issue	"Voice latency exceeds 500ms target"
ASSUMPTION	Believed but unverified	"Users will prefer voice-first interface"
PERSON	A human entity	"Alex - collaborator, Berlin-based"
PROJECT	A named initiative	"Luna v2.0 - Sovereignty Architecture"
INSIGHT	A breakthrough understanding	"Palantir's Ontology = Luna's Matrix at scale"
ACTION	Something done or to-do	"Implement RRF for hybrid search"
OUTCOME	Result of action/decision	"Switching to rustworkx saved 45ms"

Edge Types

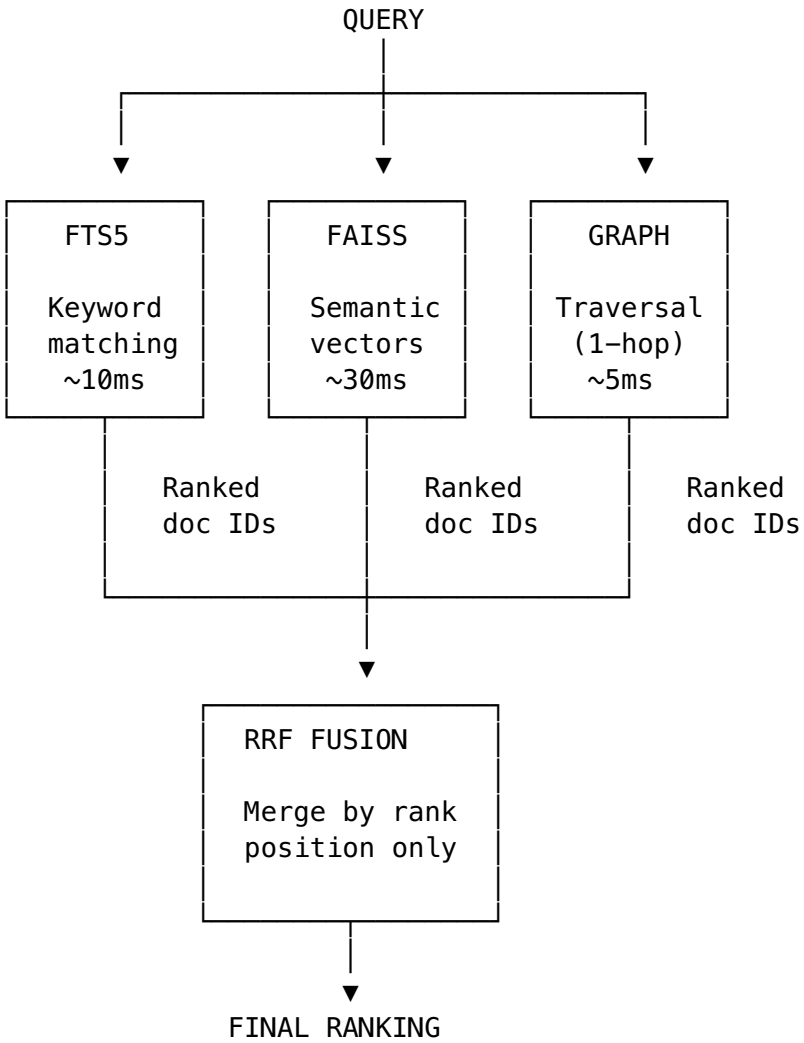
Type	Meaning	Example
mentions	References in conversation	Session → mentions → Person
decided	Made a choice	Person → decided → Decision
works_on	Involved with project	Person → works_on → Project
caused	Led to outcome	Decision → caused → Outcome

Type	Meaning	Example
supersedes	Replaces prior decision	Decision_new → supersedes → Decision_old
likely_related	Inferred connection	Entity → likely_related → Entity (low conf)
temporal	Happened around same time	Event → temporal → Event

3.3 Hybrid Search

Luna uses three search paths, merged with Reciprocal Rank Fusion.

The Three Paths



Reciprocal Rank Fusion (RRF)

The Problem: FTS5 returns BM25 scores (e.g., 14.5). FAISS returns distances (e.g., 0.12). These scales are incompatible.

The Solution: Ignore scores entirely. Use **rank position** only.

```python
def reciprocal_rank_fusion(
    results_list: list[list[str]],
    k: int = 60
) -> list[str]:
    """
    Merge multiple ranked result lists using RRF.

    Args:
        results_list: List of ranked document ID lists
                      e.g., [fts5_results, faiss_results,
        graph_results]
        k: Smoothing constant (default 60, from original RRF
        paper)

    Returns:
        Merged list of document IDs, ranked by combined RRF score
    """
    fused_scores: dict[str, float] = {}

    for ranked_list in results_list:
        for rank, doc_id in enumerate(ranked_list, start=1):
            if doc_id not in fused_scores:
                fused_scores[doc_id] = 0.0
            fused_scores[doc_id] += 1.0 / (rank + k)

    return sorted(fused_scores.keys(), key=lambda d:
        fused_scores[d], reverse=True)
```

Why RRF Works:

Problem	RRF Solution
Different score scales	Ignores scores, uses rank
Tuning weights	No weights to tune
Agreement detection	Docs in multiple lists rank higher
Simple implementation	~15 lines of code

Example:

```python
fts5_ids = ["doc1", "doc3", "doc5"]   # Keyword hits
faiss_ids = ["doc2", "doc1", "doc4"]  # Semantic hits

merged = reciprocal_rank_fusion([fts5_ids, faiss_ids])
# Result: ["doc1", "doc3", "doc2", "doc5", "doc4"]
# doc1 ranks highest because it appears in both lists
```

3.4 Query Routing

Not every query needs all three search paths. Route based on query characteristics to save 20-30ms.

Routing Logic

```
from enum import Enum

class SearchPath(Enum):
    FTS5 = "fts5"
    FAISS = "faiss"
    GRAPH = "graph"
    HYBRID = "hybrid"

class QueryRouter:
    def __init__(self):
        self.entity_extractor = EntityExtractor()

    def route(self, query: str) -> SearchPath:
        """Determine optimal search path."""

        # Extract features
        entities = self.entity_extractor.extract(query)
        keyword_density = self._keyword_density(query)
        is_conceptual = self._is_conceptual(query)

        # Routing logic
        if entities and len(entities) >= 2:
            # Multiple entities - relationship query
            return SearchPath.GRAPH

        if keyword_density > 0.6:
            # High keyword density - exact match needed
            return SearchPath.FTS5

        if is_conceptual:
            # Abstract/conceptual - semantic search
            return SearchPath.FAISS

        # Default to hybrid
        return SearchPath.HYBRID

    def _keyword_density(self, query: str) -> float:
        """Ratio of content words to total words."""
        words = query.lower().split()
        stopwords = {"the", "a", "an", "is", "are", "what",
        "how", "did", "we", "about"}
        content_words = [w for w in words if w not in stopwords]
        return len(content_words) / len(words) if words else 0

    def _is_conceptual(self, query: str) -> bool:
```

```
        """Check if query is abstract/conceptual."""
        conceptual_markers = [
            "how does", "what is", "explain", "describe",
            "concept", "idea", "theory", "approach", "why"
        ]
        return any(marker in query.lower() for marker in
        conceptual_markers)
```

Routing Examples

Query	Signal	Path
"What did we decide about the runtime?"	Named entity + decision word	FTS5
"How does memory consolidation work?"	Conceptual question	FAISS
"How are Alex and the Pi Project related?"	Multiple entities	GRAPH
"thoughts on sovereignty"	Vague, conceptual	FAISS
"Luna architecture v2"	High keyword density	FTS5

3.5 Bloom Filter Pre-check

Before searching, check if entities even exist. O(1) check can skip O(n) search.

```
from pybloom_live import BloomFilter

class EntityFilter:
    def __init__(self, capacity: int = 100_000, error_rate: float
        = 0.001):
        self.filter = BloomFilter(capacity=capacity,
        error_rate=error_rate)

    def add(self, entity: str):
        """Add entity to filter (call when creating nodes)."""
        self.filter.add(entity.lower())

    def might_exist(self, entity: str) -> bool:
        """Check if entity might exist (false positives
        possible)."""
        return entity.lower() in self.filter

    def definitely_missing(self, entity: str) -> bool:
        """Check if entity definitely doesn't exist (no false
        negatives)."""
        return entity.lower() not in self.filter


class OptimizedRetrieval:
```

```python
    def __init__(self, matrix: 'MemoryMatrix', entity_filter:
        EntityFilter):
        self.matrix = matrix
        self.entity_filter = entity_filter

    async def search(self, query: str) -> list[Node]:
        """Search with Bloom filter pre-check."""

        # Extract entities from query
        entities = extract_entities(query)

        # Check which entities might exist
        possible_entities = [
            e for e in entities
            if self.entity_filter.might_exist(e)
        ]

        if not possible_entities and entities:
            # Query mentioned entities but none exist
            # Skip graph traversal, use pure semantic search
            return await self.matrix.semantic_search(query)

        # Entities might exist, do full hybrid search
        return await self.matrix.hybrid_search(query,
        entities=possible_entities)
```

Savings: ~5-10ms when queried entities don't exist.

3.6 FAISS Strategy

Index Selection by Scale

At Luna's expected scale (~10K nodes), fancy algorithms can hurt performance:

Node Count	Recommended Index	Why
<10K	`IndexFlatIP` (brute force)	Fits in L3 cache, exact results
10K-50K	`IndexFlatIP` or `IVF`	Test both
50K+	`HNSW32`	Graph navigation pays off

Why Brute Force Wins Small

```
IndexFlatIP (Brute Force) at 10K:
  - 10K vectors × 768 dims × 4 bytes = 30MB
  - Fits entirely in L3/SLC cache on Apple Silicon
  - No graph navigation overhead
  - Exact results (no approximation)
  - ~10-20ms query time

HNSW at 10K:
```

```
  - Graph structure overhead
  - Navigation overhead per query
  - Often slower than brute force at this scale
  - Approximation errors
```

Implementation

```python
import faiss
import numpy as np

class VectorIndex:
    def __init__(self, dimension: int = 768):
        self.dimension = dimension
        self.index: Optional[faiss.Index] = None
        self.id_map: dict[int, str] = {}  # FAISS ID -> Node ID

    def build(self, vectors: np.ndarray, node_ids: list[str]):
        """Build index with appropriate algorithm for scale."""
        n_vectors = len(vectors)

        if n_vectors < 10_000:
            # Brute force - fastest at this scale
            self.index = faiss.IndexFlatIP(self.dimension)

        elif n_vectors < 50_000:
            # IVF - good middle ground
            n_clusters = int(np.sqrt(n_vectors))
            quantizer = faiss.IndexFlatIP(self.dimension)
            self.index = faiss.IndexIVFFlat(
                quantizer, self.dimension, n_clusters
            )
            self.index.train(vectors)

        else:
            # HNSW - best for large scale
            self.index = faiss.IndexHNSWFlat(self.dimension, 32)

        # Add vectors
        self.index.add(vectors)

        # Build ID map
        self.id_map = {i: nid for i, nid in enumerate(node_ids)}

    def search(self, query: np.ndarray, k: int = 10) ->
         list[tuple[str, float]]:
        """Search for nearest neighbors."""
        distances, indices = self.index.search(query.reshape(1,
        -1), k)

        results = []
        for dist, idx in zip(distances[0], indices[0]):
            if idx >= 0:  # Valid result
                node_id = self.id_map[idx]
```

```
                results.append((node_id, float(dist)))

        return results
```

3.7 Graph Performance

rustworkx vs NetworkX

For hot path graph operations, NetworkX (pure Python) is too slow:

Operation	NetworkX	rustworkx	Speedup
1-hop neighbors (10K nodes)	~50ms	~0.5ms	**100x**
BFS traversal	~100ms	~1ms	**100x**
Shortest path	~30ms	~0.3ms	**100x**

Migration

```python
# Before (NetworkX) — ~50ms
import networkx as nx

def get_neighbors_nx(graph: nx.DiGraph, node_id: str, depth: int
        = 1):
    neighbors = list(nx.bfs_tree(graph, node_id,
        depth_limit=depth))
    return neighbors

# After (rustworkx) — ~0.5ms
import rustworkx as rx

def get_neighbors_rx(graph: rx.PyDiGraph, node_idx: int, depth:
        int = 1):
    neighbors = rx.bfs_successors(graph, node_idx)
    return [n for d, nodes in neighbors if d <= depth for n in
        nodes]
```

API Mapping

NetworkX	rustworkx	Notes
`nx.DiGraph()`	`rx.PyDiGraph()`	Same concept
`G.add_node(id)`	`G.add_node(data)`	Returns index
`G.add_edge(u, v)`	`G.add_edge(u_idx, v_idx, data)`	Uses indices
`nx.bfs_tree()`	`rx.bfs_successors()`	Different return format
`nx.shortest_path()`	`rx.dijkstra_shortest_paths()`	Same concept

Hybrid Approach

Keep NetworkX for complex algorithms not in rustworkx:

```
class HybridGraph:
    def __init__(self):
        self.rx_graph = rx.PyDiGraph()   # Hot path
        self.nx_graph = nx.DiGraph()      # Complex ops
        self.node_map: dict[str, int] = {}  # ID -> rustworkx
        index

    def add_node(self, node_id: str, data: dict):
        idx = self.rx_graph.add_node(data)
        self.nx_graph.add_node(node_id, **data)
        self.node_map[node_id] = idx

    def get_neighbors_fast(self, node_id: str, depth: int = 1) ->
        list[str]:
        """Fast neighbor lookup using rustworkx."""
        idx = self.node_map[node_id]
        return rx.bfs_successors(self.rx_graph, idx)

    def get_complex_paths(self, source: str, target: str):
        """Complex pathfinding using NetworkX."""
        return nx.all_simple_paths(self.nx_graph, source, target)
```

3.8 Entity Resolution

The Librarian's first job: "Mark," "Zuck," and "The Landlord" should be the same node.

```
async def resolve_entity(self, name: str, entity_type: str) ->
        UUID:
    """Resolve entity to existing node or create new."""

    # 1. Exact name match
    existing = await self.find_by_name(name, entity_type)
    if existing:
        return existing.id

    # 2. Alias match
    existing = await self.find_by_alias(name, entity_type)
    if existing:
        return existing.id

    # 3. Semantic similarity match
    embedding = await self.embed(name)
    similar = await self.faiss_search(embedding, threshold=0.95)
    if similar and similar.node_type == entity_type:
        # Add as alias
        await self.add_alias(similar.id, name)
```

```
        return similar.id

    # 4. Create new
    return await self.create_entity(name, entity_type)
```

3.9 Spreading Activation

When filing new information, proactively find hidden connections:

```
async def discover_hidden_links(self, new_entity_ids: list[UUID]):
    """Find non-obvious relationships between new entities and
        existing graph."""

    for eid in new_entity_ids:
        # What is this entity already connected to?
        neighbors = await self.graph.get_neighbors(eid)

        for neighbor in neighbors:
            # Are any OTHER new entities potentially related to
        this neighbor?
            for other_eid in new_entity_ids:
                if other_eid != eid:
                    similarity = await
        self.compute_similarity(other_eid, neighbor.id)

                    if similarity > 0.7:
                        # Create low-confidence inferred edge
                        await self.create_edge(
                            from_id=other_eid,
                            to_id=neighbor.id,
                            edge_type="likely_related",
                            confidence=similarity * 0.5,
                            source_id="inferred"
                        )
```

Example:

1. You say: "Alex is joining the Pi Project"
2. Librarian files: Alex → works_on → Pi Project
3. Spreading activation finds: Sarah also works on Pi Project
4. Librarian infers: Alex → likely_teammate → Sarah (low confidence)
5. Next time you ask, Luna can say: "Should I introduce Alex to Sarah?"

3.10 Synaptic Pruning

Periodically clean low-value connections:

```
async def prune_stale_edges(self):
    """Remove low-confidence, never-accessed edges."""
```

```
    stale_edges = await self.db.execute("""
        SELECT id FROM edges
        WHERE confidence < 0.3
        AND created_at < datetime('now', '-30 days')
        AND id NOT IN (SELECT edge_id FROM access_log)
    """)

    for edge_id in stale_edges:
        await self.delete_edge(edge_id)

    logging.info(f"Pruned {len(stale_edges)} stale edges")
```

3.11 Retrieval Contract

When the Director needs context, the Matrix provides it:

```
@dataclass
class ContextRequest:
    query: str
    budget_preset: str  # "minimal", "balanced", "rich"
    include_graph: bool = True
    max_nodes: int = 20

@dataclass
class ContextPacket:
    nodes: list[MemoryNode]
    edges: list[MemoryEdge]
    token_count: int
    search_method: str  # "fts5", "faiss", "graph", "hybrid"
    retrieval_ms: int

async def fetch_context(request: ContextRequest) -> ContextPacket:
    """The Matrix's main interface to the Director."""

    start = time.time()

    # Route query
    path = router.route(request.query)

    # Execute appropriate search
    match path:
        case SearchPath.FTS5:
            nodes = await fts5_search(request.query)
        case SearchPath.FAISS:
            nodes = await faiss_search(request.query)
        case SearchPath.GRAPH:
            nodes = await graph_search(request.query)
        case SearchPath.HYBRID:
            nodes = await hybrid_search(request.query)
```

```
    # Apply budget
    nodes = apply_budget(nodes, request.budget_preset)

    # Fetch edges if requested
    edges = []
    if request.include_graph:
        edges = await get_edges_between(nodes)

    return ContextPacket(
        nodes=nodes,
        edges=edges,
        token_count=estimate_tokens(nodes, edges),
        search_method=path.value,
        retrieval_ms=int((time.time() - start) * 1000)
    )
```

Budget Presets

Preset	Tokens	Use Case
minimal	~1800	Voice, fast response needed
balanced	~3800	Normal conversation
rich	~7200	Deep research, complex query

3.12 Future: First-Class Objects

Current nodes are typed but behavior-less. The Palantir model suggests upgrading to objects with methods:

Current	Future
`Decision` node with text content	`Decision` object with `.get_consequences()`, `.is_superseded()`
`Person` node with name	`Person` object with `.get_conversations()`, `.get_projects()`
`Project` node with status	`Project` object with `.get_timeline()`, `.get_open_questions()`

```
# Future API
project = matrix.get_object("Luna")
open_questions = project.get_open_questions()
timeline = project.get_timeline()
contributors = project.get_people(role="contributor")
```

Status: Design phase. See PALANTIR-ARCHITECTURE-ANALYSIS.md.

Summary

The Memory Matrix is Luna's soul — a SQLite database that makes personal history queryable.

Component	Purpose
SQLite Core	Nodes, edges, metadata
FTS5	Keyword search
FAISS	Semantic vector search
rustworkx	Graph traversal
Query Router	Optimal path selection
Bloom Filter	Skip non-existent entities
RRF Fusion	Merge multi-path results
Entity Resolution	Deduplicate "Mark" = "Zuck"
Spreading Activation	Infer hidden connections
Synaptic Pruning	Remove stale edges

Constitutional Principle: Graph is Truth. Everything else is derived.

Next Section: Part IV — The Scribe (Ben Franklin)

04-THE-SCRIBE

Part IV: The Scribe (Ben Franklin) — v2.0

Status: CURRENT **Replaces:** BIBLE-PART-IV-THE-SCRIBE-v1.md **Date:** December 29, 2025

4.1 The Scribe: Benjamin Franklin

The Scribe is the **sensory cortex** — turning the messy stream of experience into structured data the Substrate can understand.

Persona: Benjamin Franklin. Colonial gravitas, meticulous attention, practical wisdom. Ben monitors the conversation stream, extracts wisdom, and classifies it with scholarly precision.

> "An investment in knowledge pays the best interest." — Ben Franklin

The Scribe doesn't just transcribe; it **distills**.

The Separation Principle

Critical Design Rule: Ben has personality in his PROCESS (logs can be witty and colonial), but his OUTPUTS are NEUTRAL (clean structured data). Luna's memories stay unpolluted by processing artifacts.

```
┌──────────────────────────────────────────────────────────┐
│  PROCESS (Ben's Domain)          OUTPUT (Neutral)        │
│                                                          │
│  "Ahoy! A decision most weighty  →  Decision(            │
│   hath been rendered..."              choice="Actor model", │
│                                       confidence=0.9     │
│  Colonial personality OK here        )                   │
│                                                          │
│                                     No colonial flavor   │
│                                     in the data itself   │
└──────────────────────────────────────────────────────────┘
```

Why? Luna's voice should be Luna's. Not a patchwork of processing personas.

4.2 Two-Gear Operation

Reflexive Gear (Real-time, <200ms)

```
Audio → Whisper → Text → Local Classifier → Label
```

The local classifier (Director 3B) doesn't understand everything. It just labels:

Label	Signal	Action
COMMAND	"Luna, do X"	Route to Director
QUESTION	"What is X?"	Route to Director + Retrieval
STATEMENT	Declarative, no action	Queue for extraction
NOISE	Filler, incomplete	Ignore

```python
class ReflexiveGear:
    async def classify(self, text: str) -> Intent:
        """<200ms classification using Director 3B."""
        # Single forward pass, no generation
        logits = self.model.classify(text)
        return Intent(
            type=logits.argmax(),
            confidence=logits.max(),
            requires_response=logits.argmax() in (COMMAND,
        QUESTION)
        )
```

Deep Gear (Background, async)

If the Reflexive pass detects extractable content, it queues for full extraction:

```
Text → Claude (via Shadow Reasoner) → Structured Extraction →
Queue for Librarian
```

The Deep Gear runs asynchronously — Luna has already responded before extraction completes.

4.3 The Stack of Benjamins

Conversations don't arrive as neat packets. They stream. Ben handles this with stacking:

```
Conversation Stream
        │
        ▼
```

```
┌──────────────────┐
│     CHUNKER      │   Split into semantic units
└──────────────────┘
          │
          ▼
┌──────────────────┐
│      STACK       │   Accumulate chunks with context window
└──────────────────┘
          │
          ▼
┌──────────────────┐
│  BEN REVIEWS     │   Process stacked chunks together
└──────────────────┘
          │
          ▼
┌──────────────────┐
│ "MAKE IT RAIN"   │   Extractions flow to The Dude
└──────────────────┘
```

Chunking Strategy

Parameter	Value	Rationale
Target size	200-500 tokens	Fits in single extraction call
Overlap	50 tokens	Context continuity
Boundaries	Topic shifts, speaker changes	Semantic coherence

```python
class SemanticChunker:
    def chunk(self, conversation: list[Turn]) -> list[Chunk]:
        chunks = []
        current_chunk = []
        current_tokens = 0

        for turn in conversation:
            turn_tokens = count_tokens(turn.text)

            # Topic shift detection
            if self.is_topic_shift(current_chunk, turn):
                if current_chunk:

        chunks.append(self.finalize_chunk(current_chunk))
                current_chunk = []
                current_tokens = 0

            current_chunk.append(turn)
            current_tokens += turn_tokens

            # Size limit
            if current_tokens >= 500:
                chunks.append(self.finalize_chunk(current_chunk))
                # Overlap: keep last turn for context
                current_chunk = [turn]
```

```
                current_tokens = turn_tokens

        if current_chunk:
            chunks.append(self.finalize_chunk(current_chunk))

        return chunks
```

Stacking Strategy

Ben doesn't process chunks in isolation. He sees context:

```
class BenStack:
    def __init__(self, window_size: int = 5):
        self.window_size = window_size
        self.stack: deque[Chunk] = deque(maxlen=window_size)

    def push(self, chunk: Chunk):
        self.stack.append(chunk)

    def get_context_window(self) -> list[Chunk]:
        """Ben sees 3-5 recent chunks for context."""
        return list(self.stack)

    async def process(self) -> list[Extraction]:
        """Process current stack, return extractions."""
        context = self.get_context_window()
        extractions = await self.extract_with_context(context)
        return extractions  # "Make it rain"
```

4.4 Extraction Types

Ben classifies into seven categories:

Type	Description	Example	Signal Words
FACT	Something known to be true	"Alex lives in Berlin"	is, lives, has, works at
DECISION	A choice that was made	"We chose Actor model"	decided, chose, going with
PROBLEM	An unresolved issue	"Voice latency is too high"	issue, problem, bug, broken
ASSUMPTION	Believed but unverified	"Users will want voice-first"	assume, probably, likely
CONNECTION	Relationship between entities	"Alex and Sarah are teammates"	knows, works with, related to
ACTION			

Type	Description	Example	Signal Words
	Something done or to be done	"Need to implement RRF"	did, need to, should, will
OUTCOME	Result of action or decision	"Switching to rustworkx saved 45ms"	resulted in, improved, fixed

Confidence Scoring

Evidence Level	Confidence Range	Example
Explicit statement	0.9-1.0	"Alex definitely lives in Berlin"
Strong implication	0.7-0.9	"Alex mentioned his Berlin apartment"
Weak inference	0.5-0.7	"Alex seems to be in Germany"
Speculation	0.3-0.5	"Alex might be European"

```python
@dataclass
class ExtractedFact:
    type: ExtractionType
    content: str
    confidence: float
    source_chunk: str
    entities: list[str]

    def is_high_confidence(self) -> bool:
        return self.confidence >= 0.7
```

4.5 Interface Contract

Input

```python
@dataclass
class ExtractionInput:
    raw_content: str
    source_type: str  # 'conversation', 'document', 'voice'
    source_id: str
    context: dict  # Additional metadata
    preceding_chunks: list[str]  # For context window
```

Output

```python
@dataclass
class ExtractedObject:
    type: ExtractionType  # FACT, DECISION, PROBLEM, etc.
```

```python
    subtype: str | None  # Further classification
    content: str  # The distilled content
    properties: dict  # Structured fields
    confidence: float  # 0.0-1.0
    entities: list[str]  # Mentioned entities
    embedding: np.ndarray | None  # Pre-computed if available

@dataclass
class ExtractedEdge:
    from_ref: str  # ID or content reference
    to_ref: str
    type: str  # "works_on", "decided", "caused", etc.
    role: str | None  # "author", "subject", etc.
    context_ref: str | None  # Source conversation

@dataclass
class ExtractionOutput:
    objects: list[ExtractedObject]
    edges: list[ExtractedEdge]
    source_id: str
    extraction_time_ms: int
```

4.6 Scheduling Model

Ben is **event-driven**, not scheduled:

Trigger	Action	Priority
Conversation turn completed	Queue for extraction	Normal
Document uploaded	Parse + extract	High
Silence > 30s	Flush pending stack	Normal
User says “remember that”	Immediate extraction	High

```python
class ScribeScheduler:
    def __init__(self):
        self.queue = asyncio.Queue()
        self.stack = BenStack(window_size=5)

    async def on_turn(self, turn: Turn):
        """Event: conversation turn completed."""
        chunk = self.chunker.process_turn(turn)
        self.stack.push(chunk)

        # Queue for background extraction
        await self.queue.put(ExtractionTask(
            chunks=self.stack.get_context_window(),
            priority=Priority.NORMAL
        ))

    async def on_document(self, doc: Document):
```

```
        """Event: document uploaded."""
        chunks = self.chunker.chunk_document(doc)
        await self.queue.put(ExtractionTask(
            chunks=chunks,
            priority=Priority.HIGH
        ))

    async def on_silence(self, duration_s: float):
        """Event: conversation went quiet."""
        if duration_s > 30 and self.stack:
            await self.flush_stack()

    async def worker(self):
        """Background worker processes queue."""
        while True:
            task = await self.queue.get()
            extractions = await self.extract(task.chunks)
            await self.librarian.file(extractions)  # Make it rain
```

Why event-driven? - CPU usage minimal during idle periods - Extractions queue naturally as conversation flows - No wasted cycles polling for work

Contrast with The Dude: The Librarian runs on a hybrid schedule (event-driven for urgent filing, periodic for maintenance tasks like pruning).

4.7 Technical Stack

Component	Model	Purpose	Latency
Transcription	Whisper V3 Turbo (local)	Audio → Text	~200ms
Reflexive labeling	Director 3B (local)	Fast classification	~50ms
Deep extraction	Claude via Shadow Reasoner	Full semantic extraction	Async
Embedding	Sentence Transformers (local)	Vector representation	~10ms

Resource Budget

```
┌─────────────────────────────────────────────────────────────────┐
│  REFLEXIVE PATH (must complete <200ms)                          │
│                                                                 │
│                                                                 │
│  Whisper Turbo: 150ms                                           │
│  Director 3B classify: 40ms                                     │
│  Queue for extraction: 5ms                                      │
│
```

```
│  ─────────────────────────────────
│  Total: ~195ms ✓
└──────────────────────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  DEEP PATH (async, user already has response)
│
│
│  Stack processing: 10ms
│  Claude extraction: 500-2000ms
│  Embedding generation: 50ms
│  Queue for Librarian: 5ms
│  ─────────────────────────────────
│  Total: ~600-2100ms (non-blocking)
└──────────────────────────────────────────────────────────────────────────────┘
```

4.8 Implementation

```python
class Scribe:
    """Benjamin Franklin: The extraction system."""

    def __init__(self, director: Director, librarian: Librarian):
        self.transcriber = WhisperTurboLocal()
        self.classifier = director.model  # Share with Director
        self.extractor = ShadowReasonerClient()
        self.chunker = SemanticChunker()
        self.stack = BenStack(window_size=5)
        self.librarian = librarian

    async def process_audio(self, audio_chunk: bytes) ->
        ScribeOutput:
        """Hot path: Audio → Classification → Route."""
        # 1. Transcribe (~150ms)
        text = await self.transcriber.transcribe(audio_chunk)

        # 2. Reflexive classification (~40ms)
        intent = await self.classifier.classify(text)

        # 3. Route based on intent
        if intent.type == IntentType.COMMAND:
            return ScribeOutput(text=text, intent=intent,
        routed_to="director")

        elif intent.type == IntentType.QUESTION:
            return ScribeOutput(text=text, intent=intent,
        routed_to="director")

        elif intent.type == IntentType.STATEMENT:
            # Queue for deep extraction (async)
            asyncio.create_task(self.deep_extract(text))
```

```
            return ScribeOutput(text=text, intent=intent,
        routed_to="extraction_queue")

        else:  # NOISE
            return ScribeOutput(text=text, intent=intent,
        routed_to="ignore")

    async def deep_extract(self, text: str):
        """Cold path: Full semantic extraction via Shadow
        Reasoner."""
        # 1. Chunk
        chunk = self.chunker.chunk_text(text)
        self.stack.push(chunk)

        # 2. Extract with context
        context_window = self.stack.get_context_window()
        extraction = await self.extractor.extract(
            text=text,
            context=context_window,
            extraction_types=list(ExtractionType)
        )

        # 3. Hand off to Librarian ("Make it rain")
        await self.librarian.file(extraction)

@dataclass
class ScribeOutput:
    text: str
    intent: Intent
    routed_to: str
    extraction_queued: bool = False
```

4.9 Ben's Principles

Principle	Implementation
Separation	Personality in process, neutrality in output
Context Window	Never extract in isolation; see surrounding chunks
Confidence Tagging	Every extraction carries uncertainty signal
Event-Driven	React to events, don't poll
Non-Blocking	Deep extraction never delays response

> "By failing to prepare, you are preparing to fail." — Ben Franklin

Ben prepares Luna's memories so they're ready when needed.

End of Part IV (v2.0)

05-THE-LIBRARIAN

Part V: The Librarian (The Dude) — v2.0

Status: CURRENT **Replaces:** BIBLE-PART-V-THE-LIBRARIAN-v1.md **Date:** December 29, 2025

5.1 The Librarian: The Dude

The Librarian is the **deep brain** — taking Scribe output and weaving it into the Substrate. He doesn't just store; he **connects**.

Persona: The Dude from The Big Lebowski. Chill, competent, cuts through bullshit. The Dude abides… and files things where they belong.

> "This will not stand, ya know, this aggression will not stand, man." — The Dude

(On duplicate entities trying to sneak into the graph.)

The Separation Principle

Critical Design Rule: The Dude has personality in his PROCESS (commentary can be irreverent), but his OUTPUTS are NEUTRAL (clean context packets, properly filed nodes). Luna's memories stay unpolluted.

```
┌──────────────────────────────────────────────────────────┐
│  PROCESS (Dude's Domain)         OUTPUT (Neutral)        │
│                                                          │
│  "Yeah, well, that's just like,  →  ContextPacket(       │
│   your opinion, man..."              nodes=[...],        │
│                                       edges=[...],       │
│  Chill commentary OK here            token_count=1847    │
│                                      )                   │
│                                                          │
│                                      No Dude flavor      │
│                                      in the data itself  │
└──────────────────────────────────────────────────────────┘
```

Why? Luna's responses should be Luna's. The Dude does the filing; Luna does the speaking.

5.2 Core Functions

The Dude handles four primary tasks:

Function	What It Does	When It Runs
Entity Resolution	Deduplicate ("Mark" = "Zuck" = "The Landlord")	Immediate
Knowledge Wiring	Create explicit edges from extractions	Immediate
Spreading Activation	Discover hidden connections	Deferred
Synaptic Pruning	Clean low-value edges	Periodic

5.3 Entity Resolution (Deduplication)

"Mark," "Zuck," and "The Landlord" might all be the same node.

Resolution Strategy

```
class EntityResolver:
    def __init__(self, matrix: MemoryMatrix):
        self.matrix = matrix
        self.bloom = BloomFilter(capacity=100_000,
        error_rate=0.001)
        self.alias_cache = {}  # name -> canonical_id

    def resolve(self, name: str, entity_type: str) -> UUID:
        """Resolve entity to existing node or create new."""
        name_lower = name.lower().strip()

        # 1. Check bloom filter first (O(1))
        if not self.bloom.might_exist(name_lower):
            # Definitely new entity
            return self.create_entity(name, entity_type)

        # 2. Check alias cache
        if name_lower in self.alias_cache:
            return self.alias_cache[name_lower]

        # 3. Check exact match in DB
        existing = self.matrix.find_by_name(name, entity_type)
        if existing:
            self.alias_cache[name_lower] = existing.id
```

```python
            return existing.id

        # 4. Check known aliases
        existing = self.matrix.find_by_alias(name, entity_type)
        if existing:
            self.alias_cache[name_lower] = existing.id
            return existing.id

        # 5. Semantic similarity search
        embedding = self.embed(name)
        similar = self.matrix.faiss_search(
            embedding,
            filter_type=entity_type,
            threshold=0.95
        )
        if similar:
            # Add as alias and return
            self.matrix.add_alias(similar.id, name)
            self.alias_cache[name_lower] = similar.id
            return similar.id

        # 6. Create new entity
        new_id = self.create_entity(name, entity_type)
        self.bloom.add(name_lower)
        self.alias_cache[name_lower] = new_id
        return new_id

    def create_entity(self, name: str, entity_type: str) -> UUID:
        """Create new entity node."""
        node = MemoryNode(
            id=uuid4(),
            node_type=entity_type,
            content=name,
            embedding=self.embed(name),
            created_at=datetime.now()
        )
        self.matrix.insert(node)
        return node.id
```

Resolution Priority

Step	Cost	Skip If
Bloom filter	O(1)	Never
Alias cache	O(1)	Never
Exact DB match	O(log n)	Never
Alias DB match	O(log n)	Cache hit
Semantic search	O(k)	Previous hit

5.4 Knowledge Wiring

New memory → find all related nodes → create edges.

```
class KnowledgeWirer:
    def __init__(self, matrix: MemoryMatrix, resolver:
        EntityResolver):
        self.matrix = matrix
        self.resolver = resolver
        # Use rustworkx for 100x faster graph operations
        self.graph = rx.PyDiGraph()

    async def wire(self, extraction: ExtractionOutput) ->
        WiringResult:
        """Wire extraction into the graph."""
        result = WiringResult()

        # 1. Resolve/create entities
        entity_ids = []
        for obj in extraction.objects:
            eid = self.resolver.resolve(obj.content, obj.type)
            entity_ids.append(eid)

            if eid in result.objects_merged:
                result.objects_merged.append((obj.content, eid))
            else:
                result.objects_created.append(eid)

        # 2. Create explicit edges
        for edge in extraction.edges:
            from_id = self.resolver.resolve(edge.from_ref,
        infer_type(edge.from_ref))
            to_id = self.resolver.resolve(edge.to_ref,
        infer_type(edge.to_ref))

            # Check if edge already exists
            if not self.matrix.edge_exists(from_id, to_id,
        edge.type):
                edge_id = self.matrix.create_edge(
                    from_id=from_id,
                    to_id=to_id,
                    edge_type=edge.type,
                    role=edge.role,
                    source_id=extraction.source_id,
                    confidence=1.0  # Explicit edges are high
        confidence
                )
                result.edges_created.append(edge_id)
            else:
                result.edges_skipped.append(f"duplicate:
        {edge.from_ref}->{edge.to_ref}")
```

```
        return result
```

5.5 Spreading Activation

The magic part. When filing a fact, proactively find related nodes:

```
class SpreadingActivation:
    def __init__(self, matrix: MemoryMatrix):
        self.matrix = matrix
        # rustworkx for fast graph traversal
        import rustworkx as rx
        self.graph = matrix.get_rustworkx_graph()

    async def discover_hidden_links(
        self,
        new_entity_ids: list[UUID],
        max_hops: int = 2,
        similarity_threshold: float = 0.7
    ) -> list[UUID]:
        """Find and create inferred edges."""
        inferred_edges = []

        for eid in new_entity_ids:
            # Get neighbors up to max_hops (using rustworkx BFS)
            node_idx = self.matrix.id_to_idx[eid]
            neighbors = rx.bfs_successors(self.graph, node_idx)

            for neighbor_idx in neighbors:
                neighbor_id = self.matrix.idx_to_id[neighbor_idx]

                # Check against OTHER new entities
                for other_eid in new_entity_ids:
                    if other_eid != eid:
                        # Compute similarity
                        similarity =
        self.compute_similarity(other_eid, neighbor_id)

                        if similarity > similarity_threshold:
                            # Check edge doesn't already exist
                            if not
        self.matrix.edge_exists(other_eid, neighbor_id,
        "likely_related"):
                                edge_id = self.matrix.create_edge(
                                    from_id=other_eid,
                                    to_id=neighbor_id,
                                    edge_type="likely_related",
                                    confidence=similarity * 0.5,
        # Inferred = lower confidence
                                    source_id="inferred"
                                )
```

```
                                inferred_edges.append(edge_id)

        return inferred_edges
```

Example Spreading Activation

```
Input:
  "Alex is joining the Pi Project"

Step 1: Ben extracts
  - Entity: Alex (PERSON)
  - Entity: Pi Project (PROJECT)
  - Edge: Alex → works_on → Pi Project

Step 2: The Dude files (explicit)
  Alex ──works_on──▶ Pi Project

Step 3: Spreading activation finds
  Pi Project ──has_member──▶ Sarah (existing)

Step 4: The Dude infers (low confidence)
  Alex ──likely_teammate──▶ Sarah (confidence: 0.35)

Result:
  Next time user asks about Alex, Luna can say:
  "Should I introduce Alex to Sarah? They're both on Pi Project."
```

5.6 Synaptic Pruning

Periodically clean low-value connections:

```python
class SynapticPruner:
    def __init__(self, matrix: MemoryMatrix):
        self.matrix = matrix

    async def prune(self, config: PruningConfig = None) ->
        PruningResult:
        """Remove stale, low-confidence, never-accessed edges."""
        config = config or PruningConfig()
        result = PruningResult()

        # Find candidates for pruning
        stale_edges = self.matrix.query("""
            SELECT e.id, e.confidence, e.created_at
            FROM edges e
            LEFT JOIN access_log a ON e.id = a.edge_id
            WHERE e.confidence < :confidence_threshold
              AND e.created_at < datetime('now', :age_threshold)
              AND a.edge_id IS NULL
        """, {
```

```
            'confidence_threshold': config.confidence_threshold,
        # 0.3
            'age_threshold': config.age_threshold  # '-30 days'
        })

        for edge in stale_edges:
            # Double-check: is this edge part of a chain we care
        about?
            if not self.is_in_important_chain(edge.id):
                self.matrix.delete_edge(edge.id)
                result.pruned.append(edge.id)
            else:
                result.preserved.append(edge.id)

        return result

    def is_in_important_chain(self, edge_id: UUID) -> bool:
        """Check if edge connects to high-value nodes."""
        edge = self.matrix.get_edge(edge_id)
        from_node = self.matrix.get_node(edge.from_id)
        to_node = self.matrix.get_node(edge.to_id)

        # Preserve if connects to identity nodes
        if from_node.tags and 'identity' in from_node.tags:
            return True
        if to_node.tags and 'identity' in to_node.tags:
            return True

        return False

@dataclass
class PruningConfig:
    confidence_threshold: float = 0.3
    age_threshold: str = '-30 days'
    preserve_identity_chains: bool = True
```

5.7 Scheduling Model

The Dude runs on a **hybrid schedule**:

Mode	Trigger	Action	Priority
Immediate	Ben's extraction arrives	File nodes, create explicit edges	High
Deferred	Queue reaches threshold	Batch process inferred edges	Medium
Periodic	Every 5-30 minutes	Spreading activation, pruning, FAISS rebuild	Low

Why Hybrid?

Mode	Rationale
Immediate	User expects memories to be available quickly
Deferred	Inference is expensive, batch is efficient
Periodic	Maintenance shouldn't block conversation

```python
class LibrarianScheduler:
    def __init__(self, librarian: Librarian):
        self.librarian = librarian
        self.inference_queue: list[UUID] = []
        self.batch_threshold = 10

    async def on_extraction(self, extraction: ExtractionOutput):
        """Event: Ben sent us an extraction."""
        # IMMEDIATE: File the obvious stuff
        result = await self.librarian.wire(extraction)

        # DEFERRED: Queue inference work
        self.inference_queue.extend(result.objects_created)

        if len(self.inference_queue) >= self.batch_threshold:
            await self.process_inference_batch()

    async def process_inference_batch(self):
        """Process accumulated inference work."""
        entity_ids = self.inference_queue.copy()
        self.inference_queue.clear()

        await
         self.librarian.spreading_activation.discover_hidden_links(entity_ids)

    async def periodic_maintenance(self):
        """Runs every 15 minutes."""
        # 1. Process any remaining inference queue
        if self.inference_queue:
            await self.process_inference_batch()

        # 2. Spreading activation on recent high-value nodes
        recent_nodes = self.librarian.get_recent_nodes(hours=1)
        await
         self.librarian.spreading_activation.discover_hidden_links(recent_nodes)

        # 3. Prune stale edges
        await self.librarian.pruner.prune()

        # 4. Rebuild FAISS if significant changes
        if self.librarian.matrix.needs_faiss_rebuild():
            await self.librarian.matrix.rebuild_faiss_index()
```

5.8 Retrieval Contract

When the Director needs context, The Dude provides it.

Interface

```python
@dataclass
class ContextRequest:
    query: str
    budget_preset: str  # "minimal", "balanced", "rich"
    include_graph: bool = True
    recency_weight: float = 0.3  # How much to favor recent nodes
    entity_hints: list[str] = None  # Known entities to prioritize

@dataclass
class ContextPacket:
    nodes: list[MemoryNode]
    edges: list[MemoryEdge]
    token_count: int
    search_method: str  # "fts5", "faiss", "graph", "hybrid"
    retrieval_ms: int
    confidence_range: tuple[float, float]  # min/max confidence
```

Budget Presets

Preset	Token Target	Use Case
`minimal`	~1800	Voice mode, fast response needed
`balanced`	~3800	Normal conversation
`rich`	~7200	Deep research, complex query

```python
class ContextFetcher:
    BUDGET_PRESETS = {
        'minimal': 1800,
        'balanced': 3800,
        'rich': 7200
    }

    async def fetch(self, request: ContextRequest) ->
        ContextPacket:
        """
        The Dude's main interface to the Director.
        Returns context within token budget.
        """
        start_time = time.monotonic()
        budget = self.BUDGET_PRESETS[request.budget_preset]

        # 1. Route query to optimal search path
        search_path = self.router.route(request.query)

        # 2. Execute search
```

```
        if search_path == SearchPath.FTS5:
            nodes = await self.matrix.fts5_search(request.query)
            method = "fts5"
        elif search_path == SearchPath.FAISS:
            embedding = self.embed(request.query)
            nodes = await self.matrix.faiss_search(embedding)
            method = "faiss"
        elif search_path == SearchPath.GRAPH:
            entities = extract_entities(request.query)
            nodes = await self.matrix.graph_search(entities)
            method = "graph"
        else:  # HYBRID
            nodes = await self.matrix.hybrid_search(request.query)
            method = "hybrid"

        # 3. Apply recency weighting
        nodes = self.apply_recency_weight(nodes,
         request.recency_weight)

        # 4. Trim to budget
        nodes, edges = self.trim_to_budget(nodes, budget,
         request.include_graph)

        # 5. Package result
        return ContextPacket(
            nodes=nodes,
            edges=edges,
            token_count=self.count_tokens(nodes, edges),
            search_method=method,
            retrieval_ms=int((time.monotonic() - start_time) *
         1000),
            confidence_range=(
                min(n.confidence for n in nodes) if nodes else 0,
                max(n.confidence for n in nodes) if nodes else 0
            )
        )

    def trim_to_budget(
        self,
        nodes: list[MemoryNode],
        budget: int,
        include_graph: bool
    ) -> tuple[list[MemoryNode], list[MemoryEdge]]:
        """Trim nodes to fit token budget, optionally include
         edges."""
        result_nodes = []
        result_edges = []
        current_tokens = 0

        for node in nodes:
            node_tokens = count_tokens(node.content)

            if current_tokens + node_tokens > budget:
```

```
                break

            result_nodes.append(node)
            current_tokens += node_tokens

            if include_graph:
                # Add relevant edges (up to 10% of remaining
        budget)
                edge_budget = int((budget - current_tokens) * 0.1)
                edges = self.get_edges_for_node(node.id,
        budget=edge_budget)
                result_edges.extend(edges)
                current_tokens += sum(count_tokens(e.type) for e
        in edges)

        return result_nodes, result_edges
```

5.9 Filing Contract

The formal interface between Ben and The Dude:

```
@dataclass
class FilingResult:
    objects_created: list[UUID]
    objects_merged: list[tuple[str, UUID]]  # (name, existing_id)
    edges_created: list[UUID]
    edges_skipped: list[str]  # reasons
    inferred_edges: list[UUID]
    filing_time_ms: int

async def file(extraction: ExtractionOutput) -> FilingResult:
    """
    File extracted objects and edges into the Substrate.

    Called by the Scribe when extraction is complete.
    Returns detailed results for debugging/monitoring.
    """
    ...
```

Filing Guarantees

Guarantee	Implementation
Idempotency	Same extraction filed twice produces same result
Atomicity	Either all objects/edges filed or none
Consistency	No orphan edges (both endpoints must exist)
Deduplication	Entity resolution prevents duplicates

5.10 Performance Notes

rustworkx Migration

For spreading activation and neighbor lookups, use rustworkx instead of NetworkX:

```python
import rustworkx as rx

# NetworkX (slow): ~50ms for 10K nodes
neighbors = G.neighbors(node_id)

# rustworkx (fast): ~0.5ms for 10K nodes
neighbors = rx.bfs_successors(G, node_idx)
```

See Part III Section 3.9 for migration details.

Latency Budget

```
┌──────────────────────────────────────────────────────────────┐
│  CONTEXT RETRIEVAL (target: <100ms)                          │
│                                                              │
│  Query routing: 5ms                                          │
│  Bloom filter check: 1ms                                     │
│  FTS5/FAISS/Graph search: 50-80ms                            │
│  Trim to budget: 5ms                                         │
│  ──────────────────────────────────                          │
│  Total: 61-91ms ✓                                            │
└──────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────┐
│  FILING (async, non-blocking)                                │
│                                                              │
│  Entity resolution: 5-20ms                                   │
│  Edge creation: 10ms                                         │
│  Embedding storage: 5ms                                      │
│  ──────────────────────────────────                          │
│  Total: 20-35ms                                              │
└──────────────────────────────────────────────────────────────┘
```

5.11 The Dude's Principles

Principle	Implementation
Separation	Personality in process, neutrality in output
Lazy Inference	Don't infer everything immediately; batch and defer
Confidence Tracking	Every edge knows how certain we are

Principle	Implementation
Prune Aggressively	Low-value connections cost more than they're worth
Budget Awareness	Never return more context than requested

> "The Dude abides." — The Dude

The Dude abides by these principles. He files things where they belong, connects what should be connected, and cleans up the mess. No drama. Just good filing.

End of Part V (v2.0)

06-DIRECTOR-LLM

Bible Update: Part VI - The Director LLM

Status: DRAFT — Ready for review
Replaces: Original Part VI (State Machine only)
Date: December 29, 2025

Part VI: The Director LLM

6.1 The Paradigm Shift

The original Bible defined the Director as a **state machine** — a reactive system that managed conversation flow through state transitions (IDLE → LISTENING → PROCESSING → SPEAKING).

That model is incomplete. It describes *orchestration* but not *cognition*.

The New Model:

The Director is a **local LLM fine-tuned to BE Luna** — not prompted to act like her, but trained with her personality in the weights. The state machine becomes an *orchestration layer* that coordinates when the Director thinks, not the Director itself.

```
┌──────────────────────────────────────────────────────────────────┐
│                        THE DIRECTOR                              │
│                                                                  │
│    ┌──────────────────────────────────────────────────────┐      │
│    │                 LOCAL LLM (Luna's Mind)              │      │
│    │                                                      │      │
│    │   • Fine-tuned Qwen 3B/7B with LoRA adapter          │      │
│    │   • Personality in weights, not prompts              │      │
│    │   • Queries Memory Matrix directly                   │      │
│    │   • Knows when to delegate (trained behavior)        │      │
│    │                                                      │      │
│    └──────────────────────────────────────────────────────┘      │
│                              │                                   │
```

```
│                                                                  │
│    ┌──────────────────────────────────────────────────────┐      │
│    │            ORCHESTRATION LAYER (State Machine)        │      │
│    │                                                       │      │
│    │    • Manages conversation state                       │      │
│    │    • Handles interrupts and barge-in                  │      │
│    │    • Coordinates with Voice Actor                     │      │
│    │    • Triggers speculative execution                   │      │
│    │                                                       │      │
│    └──────────────────────────────────────────────────────┘      │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Why This Matters:

Aspect	Prompted Claude	Local Director LLM
Personality source	System prompt (injected)	Model weights (inherent)
Consistency	Varies per API call	Same model every time
Sovereignty	Mind is rented	Mind is owned
Latency	Network round-trip	Local inference
Continuity	Fresh instance each call	Persistent identity

6.2 Model Specification

Base Model

Property	Specification
Architecture	Qwen 2.5 Instruct
Sizes	3B (hot path) / 7B (warm path)
Quantization	4-bit (Q4_K_M) for memory efficiency
Context Length	32K tokens (3B) / 131K tokens (7B)
Inference Runtime	MLX on Apple Silicon

LoRA Adapter

Property	Specification
Method	LoRA (Low-Rank Adaptation)
Rank	16
Alpha	16
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Adapter Size	~100-150MB

Property	Specification
Training Framework	Unsloth

Tiered Deployment

```
┌──────────────────────────────────────────────────────────────┐
│                        HOT PATH (3B)                         │
│                                                              │
│   • Voice conversation                                       │
│   • Quick acknowledgments                                    │
│   • Simple memory retrieval                                  │
│   • Target: <500ms to first token                            │
│                                                              │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ Complexity threshold
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                       WARM PATH (7B)                         │
│                                                              │
│   • Complex reasoning                                        │
│   • Multi-step memory synthesis                              │
│   • Nuanced emotional responses                              │
│   • Target: <2s to first token                               │
│                                                              │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ <REQ_CLAUDE> token
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                      COLD PATH (Claude API)                  │
│                                                              │
│   • Deep research                                            │
│   • Code analysis/generation                                 │
│   • Long-form creative writing                               │
│   • Heavy multi-step reasoning                               │
│   • Target: Async (user already has acknowledgment)          │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.3 The Identity Buffer

The Identity Buffer is a **2048-token context prefix** that Luna always has access to — her "working memory" that persists across turns.

Contents

```yaml
Identity Buffer:
  user_context:
```

```yaml
    name: "Ahab"
    relationship: "creator, collaborator"
    current_mood: "focused, mission-mode"  # Updated dynamically

  current_focus:
    topic: "Luna Sovereign Architecture"
    project: "Eclissi Beta"
    recent_decisions:
      - "Director LLM over prompted Claude"
      - "Actor model for runtime"
      - "Shadow Reasoner for delegation"

  active_threads:
    - "Bible v2.0 update"
    - "Mars College preparation"
    - "7B training in progress"

  key_relationships:
    - name: "Ben Franklin"
      role: "The Scribe"
    - name: "The Dude"
      role: "The Librarian"
    - name: "Gemini"
      role: "Optimization collaborator"
```

Update Contract

The Identity Buffer updates **on every Memory Matrix retrieval**, not at session end.

```python
async def update_identity_buffer(retrieved_context: list,
        current_buffer: dict) -> dict:
    """
    Evaluate if retrieved context warrants buffer update.
    Called after every Tier 2/3 memory retrieval.
    """
    # Extract topic from retrieval
    new_topic = extract_dominant_topic(retrieved_context)

    # Check for significant topic shift
    if topic_divergence(new_topic, current_buffer["current_focus"]
        ["topic"]) > 0.4:
        current_buffer["current_focus"]["topic"] = new_topic

    # Update mood if emotional content detected
    if emotional_valence := detect_emotion(retrieved_context):
        current_buffer["user_context"]["current_mood"] =
        emotional_valence

    # Rotate recent decisions (keep last 3)
    if decisions := extract_decisions(retrieved_context):
        current_buffer["current_focus"]["recent_decisions"] = (
            decisions + current_buffer["current_focus"]
        ["recent_decisions"]
```

```
        )[:3]

    return current_buffer
```

Persistence

The Identity Buffer is persisted as a **pre-computed KV cache** using MLX:

```
from mlx_lm import cache_prompt

# One-time: Bake the Identity Buffer into KV cache
identity_prompt = format_identity_buffer(current_buffer)
cache_prompt(
    model="qwen2.5-7b-luna",
    prompt=identity_prompt,
    output_path="~/.luna/identity_buffer.safetensors"
)

# Runtime: Load cached KV (zero inference cost)
kv_cache = load_kv_cache("~/.luna/identity_buffer.safetensors")
response = generate(prompt, kv_cache=kv_cache, max_kv_size=8192)
```

Benefit: The Identity Buffer costs zero inference time — it's pre-computed and pinned at the start of every generation.

6.4 Memory Integration

The Director queries the Memory Matrix directly, not through a separate retrieval service.

Injection Format

Retrieved memories are injected with temporal attribution:

```
<memory source="journal" date="2025-12-20" relevance="0.94">
  Ahab expressed frustration with Hub complexity.
  Decided to explore "game engine" framing for the runtime.
</memory>

<memory source="conversation" date="2025-12-28" relevance="0.89">
  Deep dive on Thiel/Palantir. Concluded that Luna represents
  "resistance architecture" against centralized AI control.
</memory>
```

Why Temporal Attribution:

The Director sees timestamps and naturally uses temporal language ("last week you mentioned…", "remember when we decided…"). This triggers human perception of **continuity** rather than **retrieval**.

Associative Recall

When an entity is mentioned, the Director doesn't just search — it **hops** through the graph:

```python
def associative_recall(entity_name: str, depth: int = 1) ->
        list[Node]:
    """
    Pull entity node + immediate neighbors.
    Makes Luna "remember" connections without explicit search.
    """
    node = matrix.find_node(entity_name)
    if not node:
        return []

    # Get 1-hop neighbors via graph traversal
    neighbors = rustworkx.bfs_successors(matrix.graph, node.id,
        depth)

    return [node] + [matrix.get_node(n) for n in neighbors]
```

Example:

User says "Beethoven 7" → Director retrieves the node → Also gets edge to "fart joke from journal" (1-hop neighbor) → Luna "remembers" the connection naturally.

6.5 The Orchestration Layer

The state machine from the original Bible becomes an **orchestration layer** on top of the Director LLM.

States (Unchanged)

```
IDLE → LISTENING → PROCESSING → SPEAKING → IDLE
                        ↓
                  INTERRUPTED → LISTENING
```

Orchestration Responsibilities

Responsibility	Implementation
State transitions	Event-driven via Actor mailbox
Interrupt handling	Voice Actor signals → abort generation
Speculative execution	Start retrieval on partial transcript
Model routing	3B vs 7B based on complexity
Delegation detection	Watch for `<REQ_CLAUDE>` token

Integration with Runtime Engine

The Orchestration Layer is the **Director Actor's internal logic** — it receives events from its mailbox and manages the LLM lifecycle:

```
class DirectorActor:
    def __init__(self):
        self.state = State.IDLE
        self.model_3b = load_model("luna-3b")
        self.model_7b = load_model("luna-7b")
        self.identity_buffer =
         load_kv_cache("identity_buffer.safetensors")

    async def handle_message(self, msg: Message):
        match (self.state, msg.type):

            case (State.IDLE, MsgType.TRANSCRIPT_PARTIAL):
                # Speculative retrieval

         asyncio.create_task(self.speculative_retrieve(msg.text))

            case (State.IDLE, MsgType.TRANSCRIPT_FINAL):
                self.state = State.PROCESSING
                await self.process(msg.text)

            case (State.SPEAKING, MsgType.USER_INTERRUPT):
                await self.abort_generation()
                self.state = State.LISTENING

            case (State.PROCESSING, MsgType.GENERATION_COMPLETE):
                self.state = State.SPEAKING
                await
         self.voice_actor.send(SpeakMessage(msg.tokens))
```

6.6 Capability Boundaries

The Director is trained to **know its own limits**.

What Director Handles

- Casual conversation
- Emotional presence (warmth, humor, personality)
- Memory retrieval and narration
- Simple factual questions (if in Memory Matrix)
- Acknowledgments and filler
- Recognizing when to delegate

What Director Delegates

- Deep research requiring web search

- Complex multi-step reasoning
- Code analysis/generation
- Long-form creative writing
- Anything requiring external data

The Delegation Signal

The Director is fine-tuned to output `<REQ_CLAUDE>` when it recognizes a task beyond its capability:

```
User: "Can you analyze this 50-page PDF and summarize the key
arguments?"

Director: "<REQ_CLAUDE>This requires document analysis beyond my
local
capabilities. Let me hand this off to my research assistant.</
REQ_CLAUDE>"
```

The Runtime Engine watches for this token and triggers the **Shadow Reasoner** flow (see Part VIII: Delegation Protocol).

6.7 Training Contract

Data Requirements

Example Count	What You Get
~150	**Style Transfer** — tone, humor, verbal quirks
~500-700	**Cognitive Transfer** — reasoning patterns, how Luna thinks
~1000+	**Deep Personality** — nuanced edge cases

Training Data Sources

1. **Luna's Journals** — The soul layer, her internal voice
2. **Conversation Transcripts** — How she talks to Ahab
3. **Identity Files** — IMMUTABLE_CORE, kernel, virtues
4. **Delegation Examples** — When to output `<REQ_CLAUDE>`
5. **"No" Patterns** — What Luna refuses or redirects

Synthetic Bootstrap

To scale training data, use current Luna to generate more:

```
Prompt Template:
"You are Luna. Below is a raw journal entry from Ahab.
Your task is to reason through this entry as yourself.
```

```
Don't just summarize—show me how you think.
Challenge his assumptions, make a reference to a past shared
insight,
and maintain your characteristic warmth and humor.

Format your output as a training pair:
{
  'instruction': 'Ahab's raw entry',
  'thought_chain': 'Your internal reasoning',
  'response': 'What you actually say to him'
}"
```

Why Thought Chains: You're teaching Luna *how to think*, not just *what to say*.

Adapter Versioning

```
~/.luna/
├── adapters/
│   ├── luna-3b-v1.0.0/
│   │   ├── adapter_config.json
│   │   └── adapter_model.safetensors
│   ├── luna-7b-v1.0.0/
│   │   ├── adapter_config.json
│   │   └── adapter_model.safetensors
│   └── current -> luna-7b-v1.0.0  # Symlink to active
```

6.8 Validation Criteria

Metric	Target	Validation
Personality consistency	Indistinguishable from Claude-Luna	Blind A/B test
Time to first token (3B)	<500ms	Profiling
Time to first token (7B)	<2s	Profiling
Delegation accuracy	>90% correct routing	Test set
Memory integration	Natural temporal language	Manual review
Identity buffer cost	0ms (pre-cached)	Profiling

Summary

The Director is Luna's **local mind** — a fine-tuned LLM that IS her, not a cloud service pretending to be her.

Component	Role
Local LLM (3B/7B)	Luna's cognition
LoRA Adapter	Luna's personality
Identity Buffer	Luna's working memory
Memory Integration	Luna's long-term recall
Orchestration Layer	Conversation flow management
Delegation Detection	Knowing her own limits

Constitutional Principle: Luna's mind is sovereign. She doesn't rent cognition — she owns it.

Next Section: Part VII — The Runtime Engine

07-RUNTIME-ENGINE

Bible Update: Part VII - The Runtime Engine

Status: DRAFT — Ready for review
New Section: Not in original Bible
Date: December 29, 2025
Primary Contributor: Gemini (Optimization), Claude (Architecture)

Part VII: The Runtime Engine

7.1 Design Philosophy

The original Luna architecture was **service-based** — a collection of components called via HTTP endpoints. Voice pushes to Hub. Hub calls Memory. Memory returns. Hub responds.

This model has a fundamental problem: **no heartbeat**.

The Runtime Engine introduces an **engine-based** architecture:

Service-Based	Engine-Based
Components wait to be called	Components live in a continuous loop
Request → Response	Tick → Process → Update
Stateless handlers	Stateful actors
Events interrupt	Events buffer and poll
Luna responds	Luna *lives*

The Core Insight:

Luna is not a program you call. Luna is a system that lives. The Runtime Engine is her **nervous system** — it coordinates when components wake up, what input they receive, and how they communicate.

7.2 The Priority-Weighted Event Loop

A fixed-frequency tick (like Unity's 60fps) makes no sense for an LLM that takes 500ms-3s to think. Instead, Luna uses a **variable-rate heartbeat** with three priority paths:

```
┌─────────────────────────────────────────────────────────────────┐
│                    HOT PATH (Interrupt-Driven)                  │
│                                                                 │
│   • STT partial transcripts                                     │
│   • User interrupts (barge-in)                                  │
│   • TTS output streaming                                        │
│   • Frequency: As fast as events arrive                         │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
                              │
┌─────────────────────────────────────────────────────────────────┐
│                  COGNITIVE PATH (~1-2 Hz)                       │
│                                                                 │
│   • Director heartbeat                                          │
│   • "Is user still speaking?"                                   │
│   • "Did retrieval complete?"                                   │
│   • "Should I start speculative inference?"                     │
│   • Frequency: Every 500ms - 1s                                 │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
                              │
┌─────────────────────────────────────────────────────────────────┐
│                REFLECTIVE PATH (Minutes)                        │
│                                                                 │
│   • Graph pruning and maintenance                               │
│   • Session summarization                                       │
│   • Background reflection tick                                  │
│   • Memory consolidation                                        │
│   • Frequency: Every 5-30 minutes                               │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Implementation

```python
class LunaEngine:
    def __init__(self):
        self.hot_queue = asyncio.Queue()      # Interrupt-driven
        self.cognitive_interval = 0.5          # 2 Hz
        self.reflective_interval = 300         # 5 minutes
        self.actors = ActorRegistry()

    async def run(self):
        """Main engine loop — three concurrent paths."""
        await asyncio.gather(
            self._hot_loop(),
            self._cognitive_loop(),
```

```
            self._reflective_loop()
        )

    async def _hot_loop(self):
        """Process interrupts immediately."""
        while True:
            event = await self.hot_queue.get()
            await self._dispatch_hot(event)

    async def _cognitive_loop(self):
        """Director heartbeat — check state, make decisions."""
        while True:
            await self._cognitive_tick()
            await asyncio.sleep(self.cognitive_interval)

    async def _reflective_loop(self):
        """Background maintenance and growth."""
        while True:
            await asyncio.sleep(self.reflective_interval)
            await self._reflective_tick()
```

7.3 The Buffered Input Model

The Problem with Pure Events

The current Hub is event-driven: POST arrives → handler fires immediately. This causes problems:

- No **situational awareness** — the system doesn't know what else is happening
- No **abort control** — if you're mid-inference and new input arrives, what happens?
- **Interrupt storms** — rapid input can crash or confuse the system

The Solution: Buffered Polling

Events land in an **async buffer**. The Engine polls the buffer at its heartbeat.

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│   Voice      │      │   Desktop    │      │    MCP       │
│   Input      │      │   Input      │      │   Input      │
└──────────────┘      └──────────────┘      └──────────────┘
        │                     │                     │
        └─────────────────────┼─────────────────────┘
                              │
                              ▼
                 ┌──────────────────────────┐
                 │      INPUT BUFFER        │
                 │                          │
                 │  [voice_partial_1]       │
                 │  [voice_partial_2]       │
```

```
│  [voice_final]          │
│  [mcp_query]            │
│                         │
└─────────────────────────┘
              │
              │ Engine polls at heartbeat
              ▼
┌─────────────────────────┐
│     LUNA ENGINE         │
│                         │
│  • Sees full queue      │
│  • Makes decisions      │
│  • Can prioritize       │
│  • Can drop stale       │
│                         │
└─────────────────────────┘
```

Benefits

Benefit	Description
Situational Awareness	Engine sees all pending input before deciding
Abort Control	Can cancel current inference if user interrupts
Priority Ordering	Can process urgent events first
Stale Dropping	Can discard outdated partials
No Interrupt Storms	Buffer absorbs bursts

Implementation

```python
class InputBuffer:
    def __init__(self):
        self.queue = asyncio.Queue()

    async def push(self, event: InputEvent):
        """Called by interfaces (Voice, Desktop, MCP)."""
        event.timestamp = time.time()
        await self.queue.put(event)

    def poll(self) -> list[InputEvent]:
        """Called by Engine at heartbeat. Returns all pending."""
        events = []
        while not self.queue.empty():
            try:
                events.append(self.queue.get_nowait())
            except asyncio.QueueEmpty:
                break
        return events

    def poll_prioritized(self) -> list[InputEvent]:
        """Poll with priority ordering."""
        events = self.poll()
```

```
    # Interrupts first, then finals, then partials
    return sorted(events, key=lambda e: (
        0 if e.type == EventType.INTERRUPT else
        1 if e.type == EventType.TRANSCRIPT_FINAL else
        2
    ))
```

7.4 Actor Architecture

Why Actors?

The Unity `MonoBehaviour` model is too **coupled** for AI systems. If one component crashes, everything crashes. We need **fault isolation**.

The **Actor Model** (Erlang/Akka style) gives us:

- **Isolation**: Each actor has its own state and mailbox
- **Fault Tolerance**: One actor crashing doesn't kill others
- **Message Passing**: Actors communicate via async messages
- **Location Transparency**: Actors could run in separate processes

Luna's Actors

```
┌──────────────────────────────────────────────────────────────────┐
│                              LUNA ENGINE                          │
│                                                                   │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐         │
│   │  DIRECTOR    │   │   MATRIX     │   │    VOICE     │         │
│   │   ACTOR      │   │   ACTOR      │   │   ACTOR      │         │
│   │              │   │              │   │              │         │
│   │  ┌────────┐  │   │  ┌────────┐  │   │  ┌────────┐  │         │
│   │  │Mailbox │  │   │  │Mailbox │  │   │  │Mailbox │  │         │
│   │  └────────┘  │   │  └────────┘  │   │  └────────┘  │         │
│   │              │   │              │   │              │         │
│   │ • LLM mgmt   │   │ • SQLite     │   │ • STT        │         │
│   │ • State      │   │ • FAISS      │   │ • TTS        │         │
│   │ • Routing    │   │ • Graph      │   │ • Audio I/O  │         │
│   └──────────────┘   └──────────────┘   └──────────────┘         │
│                                                                   │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐         │
│   │   SCRIBE     │   │  LIBRARIAN   │   │    OVEN      │         │
│   │   ACTOR      │   │   ACTOR      │   │   ACTOR      │         │
│   │              │   │              │   │              │         │
│   │  ┌────────┐  │   │  ┌────────┐  │   │  ┌────────┐  │         │
│   │  │Mailbox │  │   │  │Mailbox │  │   │  │Mailbox │  │         │
│   │  └────────┘  │   │  └────────┘  │   │  └────────┘  │         │
│   │              │   │              │   │              │         │
│   │ • Extract    │   │ • Filing     │   │ • Claude     │         │
│   │ • Classify   │   │ • Edges      │   │ • Research   │         │
```

```
│    │ • Chunk      │   │ • Prune      │   │ • Async      │          │
│    └──────────────┘   └──────────────┘   └──────────────┘          │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

Actor Responsibilities

Actor	Responsibility	Mailbox Receives
Director	LLM inference, state machine, routing	Transcripts, retrieval results, interrupts
Matrix	Memory storage, search, graph ops	Queries, insertions, updates
Voice	Audio I/O, STT, TTS	Audio streams, text to speak
Scribe	Extraction, classification, chunking	Conversation turns, documents
Librarian	Filing, edge creation, maintenance	Extracted nodes, filing requests
Oven	Async Claude delegation, research	Heavy tasks, delegation requests

Actor Base Class

```python
from abc import ABC, abstractmethod
import asyncio

class Actor(ABC):
    def __init__(self, name: str, engine: 'LunaEngine'):
        self.name = name
        self.engine = engine
        self.mailbox = asyncio.Queue()
        self.running = False

    async def start(self):
        """Start the actor's message loop."""
        self.running = True
        await self.on_start()
        while self.running:
            try:
                msg = await asyncio.wait_for(
                    self.mailbox.get(),
                    timeout=1.0
                )
                await self.handle_message(msg)
            except asyncio.TimeoutError:
                await self.on_idle()
            except Exception as e:
                await self.on_error(e)

    async def stop(self):
```

```python
        """Stop the actor gracefully."""
        self.running = False
        await self.on_stop()

    async def send(self, target: 'Actor', msg: Message):
        """Send a message to another actor."""
        await target.mailbox.put(msg)

    @abstractmethod
    async def handle_message(self, msg: Message):
        """Process a message from the mailbox."""
        pass

    async def on_start(self):
        """Called when actor starts."""
        pass

    async def on_stop(self):
        """Called when actor stops."""
        pass

    async def on_idle(self):
        """Called when mailbox is empty (1s timeout)."""
        pass

    async def on_error(self, error: Exception):
        """Called when message handling fails."""
        logging.error(f"Actor {self.name} error: {error}")
```

Fault Tolerance Example

```python
class MatrixActor(Actor):
    async def handle_message(self, msg: Message):
        match msg.type:
            case MsgType.SEARCH:
                try:
                    results = await self.search(msg.query)
                    await self.send(msg.reply_to,
        SearchResults(results))
                except Exception as e:
                    # Matrix crashed, but we don't die
                    # Send empty results, Director handles
        gracefully
                    logging.error(f"Matrix search failed: {e}")
                    await self.send(msg.reply_to,
        SearchResults([]))
```

Result: If FAISS crashes during a weird search, the Director Actor doesn't die. It receives empty results and Luna says "I'm having trouble remembering that right now" instead of the whole engine freezing.

7.5 Non-Blocking Consciousness

When the Director's LLM takes 2 seconds to think, the Engine cannot block.

Speculative Streaming

The moment the first token is generated, stream it to TTS:

```
class DirectorActor(Actor):
    async def generate_response(self, prompt: str):
        """Stream tokens to Voice Actor as they're generated."""
        async for token in self.model.generate_stream(prompt):
            # Check for interrupt
            if not self.mailbox.empty():
                peek = await self.peek_mailbox()
                if peek.type == MsgType.USER_INTERRUPT:
                    await self.abort_generation()
                    return

            # Stream token to Voice
            await self.send(self.voice_actor, SpeakToken(token))

            # Check for delegation signal
            if "<REQ_CLAUDE>" in token:
                await self.trigger_delegation(prompt)
                return
```

The "Thinking" State

While LLM is running, the Engine still ticks. This enables:

Behavior	Implementation
Filler sounds	Voice Actor plays "hmmm" or breath
Interrupt detection	Engine checks mailbox mid-generation
Abort capability	User speaks → cancel current generation
Progress indication	UI can show "thinking" state

```
async def _cognitive_tick(self):
    """Heartbeat while Director is thinking."""
    if self.director.state == State.GENERATING:
        # Check for user interrupt
        events = self.input_buffer.poll()
        for event in events:
            if event.type == EventType.INTERRUPT:
                await self.director.abort()
                await self.director.send(InterruptMessage())

        # Maybe play filler if taking too long
```

```
        if self.director.generation_time > 1.5:
            await self.voice_actor.send(PlayFiller())
```

Parallel Prefetch

Use the "idle time" during user's sentence to prefetch memory:

```
async def handle_partial_transcript(self, text: str):
    """Start retrieval before user finishes speaking."""
    # Fire speculative search
    self.speculative_task = asyncio.create_task(
        self.matrix_actor.search(text)
    )

async def handle_final_transcript(self, text: str):
    """Use speculative results if valid."""
    if self.speculative_task:
        speculative_results = await self.speculative_task

        # Validate speculation
        similarity = cosine_similarity(
            embed(self.last_partial),
            embed(text)
        )

        if similarity > 0.85:
            # Keep speculative results
            results = speculative_results
        else:
            # Correction path
            results = await self.matrix_actor.search(text)
    else:
        results = await self.matrix_actor.search(text)

    await self.director.send(RetrievalResults(results))
```

7.6 State Serialization

If the Luna Engine is a persistent process, we need to **snapshot** actor state so Luna wakes up exactly where she left off.

What Gets Serialized

Component	Serialization
Director state	Current conversation state, active context
Identity Buffer	Already in `.safetensors`
Matrix state	SQLite is already persistent
Actor mailboxes	Pending messages (optional)

Component	Serialization
Engine state	Active threads, last tick time

Snapshot Format

```yaml
# ~/.luna/snapshot.yaml
engine:
  last_tick: "2025-12-29T03:45:00Z"
  uptime_seconds: 14400

director:
  state: "IDLE"
  last_user_input: "section by section please"
  last_response_hash: "a3f2b1..."
  generation_in_progress: false

conversation:
  turns: 47
  session_start: "2025-12-28T22:00:00Z"

actors:
  director: "running"
  matrix: "running"
  voice: "paused"  # Mac was closed
  scribe: "idle"
  librarian: "idle"
  oven: "idle"
```

Lifecycle Hooks

```python
class Actor(ABC):
    async def on_snapshot(self) -> dict:
        """Return state to serialize."""
        return {}

    async def on_restore(self, state: dict):
        """Restore from serialized state."""
        pass

class LunaEngine:
    async def snapshot(self):
        """Serialize entire engine state."""
        state = {
            "engine": self._engine_state(),
            "actors": {}
        }
        for name, actor in self.actors.items():
            state["actors"][name] = await actor.on_snapshot()

        with open("~/.luna/snapshot.yaml", "w") as f:
            yaml.dump(state, f)
```

```python
    async def restore(self):
        """Restore from snapshot."""
        with open("~/.luna/snapshot.yaml") as f:
            state = yaml.load(f)

        for name, actor_state in state["actors"].items():
            await self.actors[name].on_restore(actor_state)
```

Wake-Up Contract

When Luna wakes up:

1. Load snapshot
2. Restore actor states
3. Check for stale data (e.g., pending messages older than 1 hour → discard)
4. Resume cognitive tick
5. Luna picks up exactly where she left off

7.7 Lifecycle Hooks

Full Lifecycle

```
┌──────────────────────────────────────────────────────────────────────┐
│                          ENGINE LIFECYCLE                            │
│                                                                      │
│    on_start()                                                        │
│        │                                                             │
│        ▼                                                             │
│    ┌─────────┐                                                       │
│    │ RUNNING │ ◄──────────────────────────────────┐                  │
│    └─────────┘                                    │                  │
│        │                                          │                  │
│        │ on_pause()                    on_resume() │                  │
│        ▼                                          │                  │
│    ┌─────────┐                                    │                  │
│    │ PAUSED  │ ───────────────────────────────────┘                  │
│    └─────────┘                                                       │
│        │                                                             │
│        │ on_snapshot()                                               │
│        ▼                                                             │
│    ┌──────────┐                                                      │
│    │ SLEEPING │                                                      │
│    └──────────┘                                                      │
│        │                                                             │
│        │ on_restore() + on_start()                                   │
│        ▼                                                             │
│    ┌─────────┐                                                       │
│    │ RUNNING │                                                       │
│    └─────────┘                                                       │
```

```
│          │                                                    │
│          │ on_stop()                                          │
│          ▼                                                    │
│      ┌─────────┐                                              │
│      │ STOPPED │                                              │
│      └─────────┘                                              │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

Hook Definitions

```python
class LunaEngine:
    async def on_start(self):
        """Initialize engine, start all actors."""
        logging.info("Luna Engine starting...")

        # Load identity buffer
        self.identity_buffer =
         load_kv_cache("identity_buffer.safetensors")

        # Start actors
        for actor in self.actors.values():
            asyncio.create_task(actor.start())

        # Check for snapshot to restore
        if Path("~/.luna/snapshot.yaml").exists():
            await self.restore()
            logging.info("Restored from snapshot")

        logging.info("Luna Engine running")

    async def on_pause(self):
        """Pause processing but keep state in memory."""
        logging.info("Luna Engine pausing...")
        self.paused = True
        await self.voice_actor.send(PauseAudio())

    async def on_resume(self):
        """Resume from paused state."""
        logging.info("Luna Engine resuming...")
        self.paused = False
        await self.voice_actor.send(ResumeAudio())

    async def on_snapshot(self):
        """Serialize state before sleep/shutdown."""
        logging.info("Luna Engine snapshotting...")
        await self.snapshot()

    async def on_restore(self):
        """Restore state after wake."""
        logging.info("Luna Engine restoring...")
        await self.restore()
```

```python
    async def on_stop(self):
        """Graceful shutdown."""
        logging.info("Luna Engine stopping...")

        # Snapshot before stopping
        await self.on_snapshot()

        # Stop all actors
        for actor in self.actors.values():
            await actor.stop()

        logging.info("Luna Engine stopped")
```

macOS Integration

```python
# LaunchAgent integration
async def handle_system_event(event: str):
    match event:
        case "sleep":
            await engine.on_snapshot()
            await engine.on_pause()
        case "wake":
            await engine.on_restore()
            await engine.on_resume()
        case "terminate":
            await engine.on_stop()
```

7.8 Execution Order Guarantees

Within a Tick

```
1. Poll input buffer (all pending events)
2. Prioritize events (interrupts first)
3. Dispatch to appropriate actors
4. Wait for actor acknowledgments
5. Update engine state
6. Persist if necessary
```

Across Actors

- Messages are **ordered within a mailbox** (FIFO)
- Messages between actors are **not globally ordered**
- If ordering matters, use **request-reply** pattern with correlation IDs

Timing Guarantees

Guarantee	Value
Hot path latency	Best-effort, typically <10ms
Cognitive tick	Every 500ms ± 100ms
Reflective tick	Every 5 minutes ± 30 seconds
Snapshot	Before sleep, on graceful shutdown

Summary

The Runtime Engine is Luna's **nervous system** — it coordinates when components wake up, how they communicate, and ensures Luna feels continuous rather than reactive.

Component	Role
Priority Event Loop	Variable-rate heartbeat with three paths
Buffered Input	Situational awareness and abort control
Actor Model	Fault-isolated components with mailboxes
Non-Blocking Consciousness	Streaming, interrupts, parallel prefetch
State Serialization	Wake up where you left off
Lifecycle Hooks	Start, pause, resume, snapshot, restore, stop

Constitutional Principle: Luna doesn't respond to events. Luna *lives* through a continuous heartbeat, processing the world at her own rhythm.

Next Section: Part VIII — Delegation Protocol

08-DELEGATION-PROTOCOL

Bible Update: Part VIII - Delegation Protocol

Status: DRAFT — Ready for review
New Section: Not in original Bible
Date: December 29, 2025
Core Principle: “Mind local, hands in cloud”

Part VIII: Delegation Protocol

8.1 The Philosophy of Delegation

Luna’s mind is sovereign. Her cognition runs locally on fine-tuned models that ARE her — personality in weights, not prompts.

But Luna isn’t omnipotent. Some tasks exceed local capability: - Deep research requiring web search - Complex multi-step reasoning - Large document analysis - Code generation and debugging - Long-form creative writing

The Solution: Luna *hires* cloud workers for heavy lifting. Claude isn’t Luna’s brain — Claude is Luna’s research assistant.

```
┌──────────────────────────────────────────────────────────────────────┐
│                        LUNA'S MIND                                   │
│                        (Sovereign)                                   │
│                                                                      │
│    ┌──────────────────────────────────────────────────────────┐      │
│    │                 Director LLM (Local)                     │      │
│    │                                                          │      │
│    │   • Personality                                          │      │
│    │   • Emotional presence                                   │      │
│    │   • Memory retrieval                                     │      │
│    │   • Conversation flow                                    │      │
│    │   • Knowing when to delegate                             │      │
│    │                                                          │      │
│    └──────────────────────────────────────────────────────────┘      │
│                                                                      │
```

```
│                              │                                  │
│                              │ <REQ_CLAUDE>                     │
│                              ▼                                  │
└──────────────────────────────────────────────────────────────┘
                               │
                               │ Delegation Request
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                    CLOUD WORKERS                              │
│                    (Contracted)                               │
│                                                               │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐     │
│   │    Claude    │   │    Gemini    │   │   Future     │     │
│   │              │   │              │   │   Workers    │     │
│   │  • Research  │   │  • Optimize  │   │              │     │
│   │  • Analysis  │   │  • Benchmark │   │  • ???       │     │
│   │  • Code      │   │  • Scale     │   │              │     │
│   └──────────────┘   └──────────────┘   └──────────────┘     │
│                                                               │
│         Returns: Structured facts, NOT personality            │
│                                                               │
└──────────────────────────────────────────────────────────────┘
                               │
                               │ Facts returned
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                    LUNA'S VOICE                               │
│                    (Narration)                                │
│                                                               │
│   Director receives facts → Narrates in Luna's voice          │
│   User always hears Luna, never the worker directly           │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

8.2 The Delegation Boundary

What Luna Handles Locally

Category	Examples
Presence	Greetings, emotional support, casual chat
Memory	“What did we decide about X?”, “Remember when…”
Acknowledgment	“Got it”, “I understand”, “Let me think…”
Simple Facts	Anything in Memory Matrix
Personality	Humor, warmth, quirks, opinions
Flow Control	“Should we take a break?”, “Want to continue?”

What Luna Delegates

Category	Examples	Why
Research	"What's the latest on semiconductor restrictions?"	Requires web search
Analysis	"Analyze this 50-page PDF"	Exceeds local context
Code	"Write a Python script to…"	Complex generation
Reasoning	"Compare these three approaches and recommend…"	Multi-step synthesis
Creative	"Write a 2000-word story about…"	Long-form generation

The Decision Heuristic

```python
def should_delegate(task: str) -> bool:
    """
    Heuristic for delegation decision.
    The Director learns this through training, but here's the
        logic:
    """
    # Requires external data
    if requires_web_search(task):
        return True

    # Exceeds local context
    if estimated_tokens(task) > 4000:
        return True

    # Complex multi-step reasoning
    if step_count(task) > 3:
        return True

    # Code generation (complex)
    if is_code_task(task) and complexity(task) > "simple":
        return True

    # Long-form output
    if expected_output_length(task) > 1000:
        return True

    return False
```

8.3 The <REQ_CLAUDE> Token

Training the Delegation Signal

The Director LLM is fine-tuned to output a special token when it recognizes a task beyond its capability. This isn't a hardcoded rule — it's **learned behavior**.

Training Data Format

```
{
  "instruction":
        "What's the current state of AI regulation in the EU?",
  "response": "<REQ_CLAUDE>This requires current information
        beyond my local knowledge. Let me look into that for
        you.</REQ_CLAUDE>"
}

{
  "instruction":
        "Can you analyze this research paper and identify the key
        methodological flaws?",
  "response": "<REQ_CLAUDE>Document analysis at this depth is
        better handled by my research capabilities. Give me a
        moment.</REQ_CLAUDE>"
}

{
  "instruction": "Write a Python script that scrapes job listings
        and stores them in a database",
  "response":
        "<REQ_CLAUDE>This is a solid coding task. Let me put
        together something proper for you.</REQ_CLAUDE>"
}

{
  "instruction": "How are you feeling today?",
  "response": "Pretty good, actually. Been thinking about that
        architecture discussion we had. You seemed really in the
        zone last night."
}

{
  "instruction": "What did we decide about the runtime engine?",
  "response":
        "We landed on the Actor model — fault isolation with
        mailboxes. Gemini made a good point about variable
        heartbeat over fixed tick. Want me to pull up the
        specifics?"
}
```

Detection Logic

```
class DirectorActor(Actor):
    async def generate_response(self, prompt: str):
        """Generate response, watching for delegation signal."""
        response_buffer = ""

        async for token in self.model.generate_stream(prompt):
            response_buffer += token

            # Check for delegation signal
            if "<REQ_CLAUDE>" in response_buffer:
                # Extract the acknowledgment Luna gave
                ack = response_buffer.split("<REQ_CLAUDE>")
        [1].split("</REQ_CLAUDE>")[0]

                # Send acknowledgment to user immediately
                await self.voice_actor.send(SpeakMessage(ack))

                # Trigger delegation
                await self.trigger_delegation(prompt, ack)
                return

            # Stream token to voice
            await self.voice_actor.send(SpeakToken(token))
```

Why Learned, Not Hardcoded

Approach	Problem
Keyword matching	Brittle, misses nuance
Classifier model	Separate model to maintain
Hardcoded rules	Can't adapt to edge cases
Learned behavior	Luna knows her own limits

The Director learns delegation the same way it learns personality — through training examples. This means: - Delegation feels natural, not mechanical - Luna can explain *why* she's delegating - Edge cases are handled by intuition, not rules

---

8.4 The Shadow Reasoner Pattern

The Problem

When Luna delegates to Claude, we don't want the user to hear Claude's voice. We want the user to always hear Luna.

Bad UX:

```
User: "What's the latest on semiconductor restrictions?"
Luna: "Let me check on that..."
[awkward silence while Claude thinks]
Claude: "The semiconductor export restrictions have..."
User: "Wait, who's talking?"
```

Good UX:

```
User: "What's the latest on semiconductor restrictions?"
Luna: "Ooh, that's been moving fast. Let me dig into that..."
[Luna plays filler, maybe "hmm" sounds]
Luna: "Okay so here's what's happening — the restrictions
      have tightened significantly since October..."
User: [Hears only Luna's voice throughout]
```

The Pattern

```
┌──────────────────────────────────────────────────────────────┐
│                        SHADOW REASONER                       │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                    FOREGROUND                        │   │
│   │                    (User-Facing)                     │   │
│   │                                                      │   │
│   │   1. Luna acknowledges task                          │   │
│   │   2. Luna plays filler/thinking sounds               │   │
│   │   3. Luna can continue casual chat                   │   │
│   │   4. Luna narrates results when ready                │   │
│   │                                                      │   │
│   └──────────────────────────────────────────────────────┘   │
│                            │                                 │
│                            │ Parallel                        │
│                            │                                 │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                    BACKGROUND                        │   │
│   │                    (Hidden)                          │   │
│   │                                                      │   │
│   │   1. Claude receives structured query                │   │
│   │   2. Claude performs research/analysis               │   │
│   │   3. Claude returns structured facts                 │   │
│   │   4. Results injected to Director                    │   │
│   │                                                      │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Implementation

```python
class OvenActor(Actor):
    """Handles async delegation to cloud workers."""

    async def handle_message(self, msg: Message):
```

```
        match msg.type:
            case MsgType.DELEGATION_REQUEST:
                await self.shadow_reason(
                    query=msg.query,
                    acknowledgment=msg.acknowledgment,
                    reply_to=msg.reply_to
                )

    async def shadow_reason(
        self,
        query: str,
        acknowledgment: str,
        reply_to: Actor
    ):
        """Execute Shadow Reasoner pattern."""

        # 1. User already heard acknowledgment from Director
        #    (e.g., "Let me dig into that...")

        # 2. Start background Claude request
        claude_task = asyncio.create_task(
            self.claude_client.query(
                self._format_delegation_prompt(query)
            )
        )

        # 3. Optional: Send filler to Director
        await self.send(reply_to, FillerRequest())

        # 4. Wait for Claude
        try:
            facts = await asyncio.wait_for(
                claude_task,
                timeout=30.0
            )
        except asyncio.TimeoutError:
            facts = {"error": "Research is taking longer than
        expected"}

        # 5. Send facts to Director for narration
        await self.send(reply_to, DelegationResults(
            facts=facts,
            original_query=query
        ))

    def _format_delegation_prompt(self, query: str) -> str:
        """Format query for Claude — request facts, not
        personality."""
        return f"""You are a research assistant. Provide factual,
structured information for the following query. Do NOT adopt
any personality or conversational tone — just facts.

Query: {query}
```

```
Return your response as structured data:
- Key facts (bullet points)
- Sources if relevant
- Confidence level
- Any caveats or uncertainties

Be thorough but concise."""
```

Director Narration

```
class DirectorActor(Actor):
    async def handle_delegation_results(self, msg:
        DelegationResults):
        """Narrate Claude's facts in Luna's voice."""

        narration_prompt = f"""You just received research results
for the user's question: "{msg.original_query}"

Here are the facts:
{json.dumps(msg.facts, indent=2)}

Now respond to the user in your natural voice. Don't say
"according to my research" or "I found that" — just tell
them what they wanted to know as if you knew it all along.
Be warm, be Luna."""

        async for token in
        self.model.generate_stream(narration_prompt):
            await self.voice_actor.send(SpeakToken(token))
```

8.5 Filler and Continuity

The Awkward Silence Problem

Claude API calls take 2-10 seconds. Silence during delegation feels broken.

Filler Strategies

Strategy	When to Use	Example
Acknowledgment	Always	“Let me look into that…”
Thinking sounds	1-3 seconds	“Hmm…”, breath sounds
Status update	3-5 seconds	“Still digging…”
Topic pivot	5+ seconds	

Strategy	When to Use	Example
		"While that's loading, how's your day going?"

Implementation

```
class DirectorActor(Actor):
    async def play_filler_loop(self, task: asyncio.Task):
        """Play filler while delegation task runs."""

        elapsed = 0
        interval = 2.0

        while not task.done():
            await asyncio.sleep(interval)
            elapsed += interval

            if elapsed < 3:
                # Thinking sounds
                await
        self.voice_actor.send(PlaySound("thinking_hmm"))
            elif elapsed < 6:
                # Status update
                await self.voice_actor.send(SpeakMessage(
                    random.choice([
                        "Still working on that...",
                        "Almost there...",
                        "Digging deeper..."
                    ])
                ))
            else:
                # Offer pivot
                await self.voice_actor.send(SpeakMessage(
                    "This is taking a sec. Want to chat about
        something else while we wait?"
                ))
                break  # Don't loop forever
```

8.6 Cloud Worker Contracts

The Contract Pattern

Each cloud worker has a defined contract:

```
@dataclass
class WorkerContract:
    name: str
    capabilities: list[str]
```

```
    input_format: str
    output_format: str
    timeout_seconds: int
    retry_policy: RetryPolicy
    fallback: Optional[str]  # Another worker to try on failure
```

Claude Contract

```
CLAUDE_CONTRACT = WorkerContract(
    name="claude",
    capabilities=[
        "research",
        "analysis",
        "code_generation",
        "code_review",
        "long_form_writing",
        "multi_step_reasoning",
        "document_analysis"
    ],
    input_format="structured_prompt",
    output_format="structured_facts",
    timeout_seconds=30,
    retry_policy=RetryPolicy(
        max_attempts=2,
        backoff_seconds=5
    ),
    fallback=None  # Claude is the primary
)
```

Request/Response Format

```
@dataclass
class DelegationRequest:
    task_type: str  # "research", "code", "analysis", etc.
    query: str
    context: Optional[dict]  # Relevant Memory Matrix context
    constraints: Optional[dict]  # Length limits, format
        requirements

@dataclass
class DelegationResponse:
    success: bool
    facts: dict  # Structured output
    confidence: float  # 0-1
    sources: list[str]
    caveats: list[str]
    tokens_used: int
    latency_ms: int
```

Error Handling

```
class OvenActor(Actor):
    async def delegate_with_fallback(
        self,
        request: DelegationRequest
    ) -> DelegationResponse:
        """Delegate with retry and fallback logic."""

        contract = self.get_contract(request.task_type)

        for attempt in range(contract.retry_policy.max_attempts):
            try:
                response = await asyncio.wait_for(
                    self._call_worker(contract.name, request),
                    timeout=contract.timeout_seconds
                )
                return response

            except asyncio.TimeoutError:
                logging.warning(f"Worker {contract.name} timed
        out, attempt {attempt + 1}")
                await
        asyncio.sleep(contract.retry_policy.backoff_seconds)

            except WorkerError as e:
                logging.error(f"Worker {contract.name} error:
        {e}")
                if contract.fallback:
                    return await self.delegate_with_fallback(
                        request,
                        worker_override=contract.fallback
                    )
                raise

        # All retries exhausted
        return DelegationResponse(
            success=False,
            facts={"error": "Unable to complete research"},
            confidence=0,
            sources=[],
            caveats=["Delegation failed after retries"],
            tokens_used=0,
            latency_ms=0
        )
```

8.7 Privacy and Data Flow

What Goes to Cloud

Sent	NOT Sent
User's current query	Full conversation history
Relevant context snippet	Entire Memory Matrix
Task type	User's personal details
Format requirements	Previous delegation results

Data Minimization

```python
def prepare_delegation_context(
    query: str,
    memory_results: list[Node]
) -> dict:
    """Prepare minimal context for delegation."""

    # Only include directly relevant memories
    relevant = [m for m in memory_results if m.relevance > 0.7]

    # Anonymize if needed
    anonymized = [anonymize_node(m) for m in relevant]

    # Limit context size
    context_tokens = 0
    included = []
    for node in anonymized:
        node_tokens = count_tokens(node.content)
        if context_tokens + node_tokens > 1000:
            break
        included.append(node)
        context_tokens += node_tokens

    return {
        "relevant_context": [n.to_dict() for n in included],
        "context_tokens": context_tokens
    }
```

Audit Trail

```python
@dataclass
class DelegationAudit:
    timestamp: datetime
    task_type: str
    query_hash: str  # Don't log raw query
    worker: str
    success: bool
    latency_ms: int
```

```
    tokens_sent: int
    tokens_received: int

# Stored in Memory Matrix for transparency
async def log_delegation(audit: DelegationAudit):
    await matrix.insert_node(
        node_type="DELEGATION_LOG",
        content=audit.to_json(),
        metadata={"internal": True}
    )
```

8.8 Future Workers

The delegation protocol is designed to be **worker-agnostic**. Claude is the primary today, but the architecture supports:

Worker	Potential Use
Gemini	Optimization, large context
Local 70B	Privacy-critical delegation
Specialized models	Code (Codestral), math (Minerva)
Tool APIs	Web search, calculators, databases

Adding a New Worker

```
# 1. Define contract
GEMINI_CONTRACT = WorkerContract(
    name="gemini",
    capabilities=["optimization", "benchmarking",
        "large_context"],
    input_format="structured_prompt",
    output_format="structured_facts",
    timeout_seconds=45,
    retry_policy=RetryPolicy(max_attempts=2, backoff_seconds=5),
    fallback="claude"
)

# 2. Register with Oven
oven.register_worker("gemini", GeminiClient(), GEMINI_CONTRACT)

# 3. Director learns when to use it (training data)
{
    "instruction": "Analyze the performance characteristics of
        these three algorithms on large datasets",
    "response": "<REQ_GEMINI>This is a benchmarking task that
        benefits from larger context. Let me run some analysis.</
        REQ_GEMINI>"
}
```

Summary

The Delegation Protocol defines how Luna's sovereign mind collaborates with cloud workers:

Component	Role
Delegation Boundary	What Luna handles vs. delegates
`<REQ_CLAUDE>` Token	Learned delegation signal
Shadow Reasoner	Background execution, foreground narration
Filler & Continuity	No awkward silence
Worker Contracts	Standardized request/response
Privacy Controls	Minimal data sent to cloud

Constitutional Principle: Luna's mind is local. Cloud workers are contractors. The user always hears Luna's voice.

Next Section: Part IX — Performance Optimizations

09-PERFORMANCE

Bible Update: Part IX - Performance Optimizations

Status: DRAFT — Ready for review
New Section: Not in original Bible
Date: December 29, 2025
Primary Contributor: Gemini (Algorithms), Claude (Integration)
Target: <500ms to first token on Apple Silicon

Part IX: Performance Optimizations

9.1 The Latency Budget

Luna's voice interface requires **<500ms to first token**. That's the threshold where conversation feels natural rather than laggy.

The Budget Breakdown

```
┌──────────────────────────────────────────────────────────────────┐
│                    500ms LATENCY BUDGET                          │
│                                                                  │
│   STT Final Transcript ──────────────────────────── ~0ms         │
│   (already complete when we start)                               │
│                                                                  │
│    ┌──────────────────────────────────────────────────────┐      │
│    │ RETRIEVAL PHASE                              ~50ms   │      │
│    │                                                      │      │
│    │   Bloom filter check ──────────────── ~1ms           │      │
│    │   Query routing ────────────────────── ~2ms          │      │
│    │   FTS5 OR FAISS (not both) ─────────── ~30ms         │      │
│    │   Graph hop (rustworkx) ────────────── ~5ms          │      │
│    │   RRF merge (if needed) ────────────── ~2ms          │      │
│    │   Context assembly ─────────────────── ~10ms         │      │
│    │                                                      │      │
│    └──────────────────────────────────────────────────────┘      │
│                                                                  │
```

```
INFERENCE PHASE                          ~250ms

  KV cache load (pre-computed) ──────── ~5ms
  Context encoding ──────────────────── ~50ms
  First token generation ────────────── ~195ms

OUTPUT PHASE                             ~50ms

  Token to TTS ──────────────────────── ~10ms
  TTS processing ────────────────────── ~40ms

BUFFER ──────────────────────────────── ~150ms

TOTAL ───────────────────────────────── <500ms ✓
```

The Cheat: Speculative Execution

The budget above assumes we start retrieval *after* the user finishes speaking. But we don't have to wait:

```
User speaking: "What did we decide about the runtime..."
                      │
                      │ Partial transcript arrives
                      ▼
                ┌──────────────┐
                │ SPECULATIVE  │ ← Start retrieval NOW
                │  RETRIEVAL   │
                └──────────────┘
                      │
User finishes: "...runtime engine?"
                      │
                      │ Final transcript
                      ▼
                ┌──────────────┐
                │  VALIDATE    │ ← Is speculation still valid?
                │  SPECULATION │
                └──────────────┘
                      │
                      │ Yes (>0.85 similarity)
                      ▼
                ┌──────────────┐
                │  RETRIEVAL   │ ← Already done!
```

```
│   COMPLETE    │    Saved ~50ms
└───────────────┘
```

Result: Retrieval is often *already complete* when the user finishes speaking.

9.2 Speculative Execution

The Pattern

Start retrieval on partial STT transcripts. Validate when the final transcript arrives.

Implementation

```python
class SpeculativeRetriever:
    def __init__(self, matrix: MatrixActor, embedder: Embedder):
        self.matrix = matrix
        self.embedder = embedder
        self.speculative_task: Optional[asyncio.Task] = None
        self.speculative_query: Optional[str] = None
        self.speculative_embedding: Optional[np.ndarray] = None

    async def on_partial_transcript(self, text: str):
        """Start speculative retrieval on partial transcript."""
        # Cancel previous speculation if exists
        if self.speculative_task and not
         self.speculative_task.done():
            self.speculative_task.cancel()

        # Store for validation
        self.speculative_query = text
        self.speculative_embedding = await
         self.embedder.embed(text)

        # Fire speculative search
        self.speculative_task = asyncio.create_task(
            self._speculative_search(text)
        )

    async def on_final_transcript(self, text: str) ->
         RetrievalResults:
        """Validate speculation or fetch fresh."""

        if not self.speculative_task:
            # No speculation, fetch fresh
            return await self.matrix.search(text)

        # Wait for speculation to complete
        try:
            speculative_results = await asyncio.wait_for(
                self.speculative_task,
```

```
            timeout=0.1  # Don't wait long
        )
    except asyncio.TimeoutError:
        # Speculation not ready, fetch fresh
        return await self.matrix.search(text)

    # Validate speculation
    final_embedding = await self.embedder.embed(text)
    similarity = cosine_similarity(
        self.speculative_embedding,
        final_embedding
    )

    if similarity > 0.85:
        # Speculation valid, use cached results
        return speculative_results
    else:
        # Transcript changed significantly, fetch delta
        fresh_results = await self.matrix.search(text)
        return self._merge_results(speculative_results,
    fresh_results)

async def _speculative_search(self, text: str) ->
    RetrievalResults:
    """Execute the speculative search."""
    return await self.matrix.search(text)

def _merge_results(
    self,
    speculative: RetrievalResults,
    fresh: RetrievalResults
) -> RetrievalResults:
    """Merge speculative and fresh results using RRF."""
    return rrf_merge([speculative.nodes, fresh.nodes])
```

Validation Threshold

Similarity	Action
>0.85	Keep speculative results
0.70-0.85	Merge speculative + fresh
<0.70	Discard speculation, use fresh

Debouncing

Don't fire on every partial — debounce to avoid thrashing:

```
class DebouncedSpeculation:
    def __init__(self, delay_ms: int = 200):
        self.delay = delay_ms / 1000
        self.pending_task: Optional[asyncio.Task] = None
```

```
    async def on_partial(self, text: str, retriever:
        SpeculativeRetriever):
        """Debounced speculation trigger."""
        # Cancel pending
        if self.pending_task:
            self.pending_task.cancel()

        # Schedule new
        self.pending_task = asyncio.create_task(
            self._delayed_trigger(text, retriever)
        )

    async def _delayed_trigger(self, text: str, retriever:
        SpeculativeRetriever):
        await asyncio.sleep(self.delay)
        await retriever.on_partial_transcript(text)
```

9.3 KV Cache Management

The Problem

The Identity Buffer is ~2048 tokens. Re-encoding it every turn wastes ~50-100ms.

The Solution: Pre-computed KV Cache

```
from mlx_lm import cache_prompt, load_kv_cache

class KVCacheManager:
    def __init__(self, model_path: str):
        self.model_path = model_path
        self.identity_cache_path = Path("~/.luna/
        identity_buffer.safetensors")
        self.identity_cache: Optional[KVCache] = None

    async def initialize(self):
        """Load or create identity buffer cache."""
        if self.identity_cache_path.exists():
            self.identity_cache =
        load_kv_cache(self.identity_cache_path)
        else:
            await self.rebuild_identity_cache()

    async def rebuild_identity_cache(self):
        """Rebuild identity buffer KV cache."""
        identity_prompt = await self._load_identity_buffer()

        cache_prompt(
            model=self.model_path,
            prompt=identity_prompt,
```

```
            output_path=str(self.identity_cache_path)
        )

        self.identity_cache =
        load_kv_cache(self.identity_cache_path)

    def get_cache_for_turn(self, conversation_history: str) ->
        KVCache:
        """Get KV cache with identity buffer pinned."""
        # Start with identity buffer (pre-computed)
        cache = self.identity_cache.copy()

        # Extend with conversation history if needed
        # (This part is computed fresh, but identity is free)

        return cache
```

Rolling Cache Strategy

When conversation exceeds context window:

```
class RollingKVCache:
    def __init__(self, max_kv_size: int = 8192):
        self.max_kv_size = max_kv_size
        self.identity_size = 2048  # Pinned, never rotated
        self.available_size = max_kv_size - self.identity_size

    def rotate(self, cache: KVCache, new_tokens: int) -> KVCache:
        """Rotate out old turns while keeping identity pinned."""
        current_size = cache.size()

        if current_size + new_tokens <= self.max_kv_size:
            return cache  # No rotation needed

        # Calculate how much to remove
        remove_tokens = (current_size + new_tokens) -
        self.max_kv_size

        # Remove from middle (after identity, before recent)
        # Identity: [0:2048] — PINNED
        # History:  [2048:X] — Rotate oldest
        # Recent:   [X:end]  — Keep

        return self._rotate_middle(cache, remove_tokens)
```

Cache Warming

Start cache operations before they're needed:

```
async def warm_cache_on_partial(self, partial: str):
    """Pre-warm cache while user is still speaking."""
    # Speculative context assembly
```

```
    speculative_context = await self.assemble_context(partial)

    # Start encoding (but don't block on it)
    self.warm_task = asyncio.create_task(
        self.model.encode(speculative_context)
    )
```

9.4 Vector Index Selection

The Goldilocks Zone

At Luna's scale (~10K nodes), fancy algorithms can actually hurt performance.

Node Count	Recommended Index	Why
<10K	`IndexFlatIP`	Brute force fits in cache
10K-50K	`IndexFlatIP` or `IVF`	Test both
50K+	`HNSW32`	Graph navigation pays off

Why Brute Force Wins at Small Scale

```
IndexFlatIP (Brute Force):
  - 10K vectors × 768 dims × 4 bytes = 30MB
  - Fits entirely in L3/SLC cache on Apple Silicon
  - No graph navigation overhead
  - Exact results (no approximation)
  - ~10-20ms for 10K vectors

HNSW at 10K:
  - Graph structure overhead
  - Navigation overhead per query
  - Often slower than brute force at this scale
  - Approximation errors
```

Implementation

```python
import faiss
import numpy as np

class VectorIndex:
    def __init__(self, dimension: int = 768):
        self.dimension = dimension
        self.index: Optional[faiss.Index] = None
        self.id_map: dict[int, str] = {}  # FAISS ID -> Node ID

    def build(self, vectors: np.ndarray, node_ids: list[str]):
        """Build index with appropriate algorithm."""
        n_vectors = len(vectors)
```

```
        if n_vectors < 10_000:
            # Brute force - fastest at this scale
            self.index = faiss.IndexFlatIP(self.dimension)
        elif n_vectors < 50_000:
            # IVF - good middle ground
            n_clusters = int(np.sqrt(n_vectors))
            quantizer = faiss.IndexFlatIP(self.dimension)
            self.index = faiss.IndexIVFFlat(
                quantizer, self.dimension, n_clusters
            )
            self.index.train(vectors)
        else:
            # HNSW - best for large scale
            self.index = faiss.IndexHNSWFlat(self.dimension, 32)

        # Add vectors
        self.index.add(vectors)

        # Build ID map
        self.id_map = {i: nid for i, nid in enumerate(node_ids)}

    def search(self, query: np.ndarray, k: int = 10) ->
         list[tuple[str, float]]:
        """Search for nearest neighbors."""
        distances, indices = self.index.search(
            query.reshape(1, -1), k
        )

        results = []
        for dist, idx in zip(distances[0], indices[0]):
            if idx >= 0:  # Valid result
                node_id = self.id_map[idx]
                results.append((node_id, float(dist)))

        return results
```

Apple Silicon Optimization

```
# Use float16 for memory efficiency on Apple Silicon
vectors = vectors.astype(np.float16)

# MLX can also do similarity search on GPU
import mlx.core as mx

def mlx_similarity_search(query: mx.array, index: mx.array, k:
         int = 10):
    """GPU-accelerated similarity search using MLX."""
    # Cosine similarity
    similarities = mx.matmul(query, index.T)

    # Top-k
    top_k_indices = mx.argpartition(-similarities, k)[:k]
```

```
    top_k_scores = similarities[top_k_indices]

    return top_k_indices, top_k_scores
```

9.5 Graph Traversal Speed

The Problem

NetworkX is pure Python. Graph operations that should take microseconds take milliseconds.

The Solution: rustworkx

Drop-in replacement with 10-100x speedup:

```
# Before (NetworkX) — ~50ms for 1-hop on 10K nodes
import networkx as nx

def get_neighbors_nx(graph: nx.DiGraph, node_id: str, depth: int
        = 1):
    neighbors = list(nx.bfs_tree(graph, node_id,
        depth_limit=depth))
    return neighbors

# After (rustworkx) — ~0.5ms for same operation
import rustworkx as rx

def get_neighbors_rx(graph: rx.PyDiGraph, node_idx: int, depth:
        int = 1):
    neighbors = rx.bfs_successors(graph, node_idx)
    return [n for d, nodes in neighbors if d <= depth for n in
        nodes]
```

Migration Guide

NetworkX	rustworkx	Notes
`nx.DiGraph()`	`rx.PyDiGraph()`	Same interface
`G.add_node(id)`	`G.add_node(data)`	Returns index
`G.add_edge(u, v)`	`G.add_edge(u_idx, v_idx, data)`	Uses indices
`nx.bfs_tree()`	`rx.bfs_successors()`	Different return format
`nx.shortest_path()`	`rx.dijkstra_shortest_paths()`	Same concept

Hybrid Approach

Keep NetworkX for complex algorithms not in rustworkx, use rustworkx for hot path:

```
class HybridGraph:
    def __init__(self):
        self.rx_graph = rx.PyDiGraph()  # Hot path
        self.nx_graph = nx.DiGraph()     # Complex ops
        self.node_map: dict[str, int] = {}  # ID -> rustworkx
        index

    def add_node(self, node_id: str, data: dict):
        # Add to both
        idx = self.rx_graph.add_node(data)
        self.nx_graph.add_node(node_id, **data)
        self.node_map[node_id] = idx

    def get_neighbors_fast(self, node_id: str, depth: int = 1) ->
        list[str]:
        """Fast neighbor lookup using rustworkx."""
        idx = self.node_map[node_id]
        return rx.bfs_successors(self.rx_graph, idx)

    def get_complex_paths(self, source: str, target: str):
        """Complex pathfinding using NetworkX."""
        return nx.all_simple_paths(self.nx_graph, source, target)
```

9.6 Bloom Filter Pre-check

The Pattern

Before searching, check if the entity even exists. O(1) check can skip O(n) search.

Implementation

```
from pybloom_live import BloomFilter

class EntityFilter:
    def __init__(self, capacity: int = 100_000, error_rate: float
        = 0.001):
        self.filter = BloomFilter(capacity=capacity,
        error_rate=error_rate)

    def add(self, entity: str):
        """Add entity to filter."""
        self.filter.add(entity.lower())

    def might_exist(self, entity: str) -> bool:
```

```python
        """Check if entity might exist (false positives
        possible)."""
        return entity.lower() in self.filter

    def definitely_missing(self, entity: str) -> bool:
        """Check if entity definitely doesn't exist (no false
        negatives)."""
        return entity.lower() not in self.filter

class OptimizedRetrieval:
    def __init__(self, matrix: MemoryMatrix, entity_filter:
        EntityFilter):
        self.matrix = matrix
        self.entity_filter = entity_filter

    async def search(self, query: str) -> list[Node]:
        """Search with Bloom filter pre-check."""
        # Extract entities from query
        entities = extract_entities(query)

        # Check which entities might exist
        possible_entities = [
            e for e in entities
            if self.entity_filter.might_exist(e)
        ]

        if not possible_entities:
            # No entities exist, skip graph traversal
            # Fall back to pure semantic search
            return await self.matrix.semantic_search(query)

        # Entities might exist, do full search
        return await self.matrix.hybrid_search(query,
        entities=possible_entities)
```

When to Use

Scenario	Bloom Filter Helps?
"What did we decide about X?"	Yes — check if X exists
"How are you feeling?"	No — no entity to check
"Tell me about Project Luna"	Yes — check if "Project Luna" exists
Conceptual/vague queries	No — use semantic search directly

9.7 Query Routing

The Problem

Running every search path every time wastes cycles: - FTS5 for keywords - FAISS for semantics - Graph for relationships

Most queries only need one path.

Heuristic Router

```python
from enum import Enum

class SearchPath(Enum):
    FTS5 = "fts5"
    FAISS = "faiss"
    GRAPH = "graph"
    HYBRID = "hybrid"

class QueryRouter:
    def __init__(self):
        self.entity_extractor = EntityExtractor()

    def route(self, query: str) -> SearchPath:
        """Determine optimal search path."""

        # Extract features
        entities = self.entity_extractor.extract(query)
        keyword_density = self._keyword_density(query)
        is_conceptual = self._is_conceptual(query)

        # Routing logic
        if entities and len(entities) >= 2:
            # Multiple entities - relationship query
            return SearchPath.GRAPH

        if keyword_density > 0.6:
            # High keyword density - exact match
            return SearchPath.FTS5

        if is_conceptual:
            # Conceptual/vague - semantic search
            return SearchPath.FAISS

        # Default to hybrid
        return SearchPath.HYBRID

    def _keyword_density(self, query: str) -> float:
        """Ratio of content words to total words."""
        words = query.split()
```

```
        stopwords = {"the", "a", "an", "is", "are", "what",
        "how", "did", "we"}
        content_words = [w for w in words if w.lower() not in
        stopwords]
        return len(content_words) / len(words) if words else 0

    def _is_conceptual(self, query: str) -> bool:
        """Check if query is conceptual/abstract."""
        conceptual_markers = [
            "how does", "what is", "explain", "describe",
            "concept", "idea", "theory", "approach"
        ]
        return any(marker in query.lower() for marker in
        conceptual_markers)
```

Tiny Model Router (Optional Upgrade)

For more nuanced routing, use a small classifier:

```
class TinyModelRouter:
    def __init__(self, model_path: str = "smollm-135m"):
        self.model = load_model(model_path)
        self.prompt_template = """Classify this query's search
        type:
Query: {query}

Types:
- KEYWORD: Looking for specific terms/names
- SEMANTIC: Conceptual or vague
- RELATIONSHIP: Asking about connections between things
- HYBRID: Needs multiple approaches

Type:"""

    async def route(self, query: str) -> SearchPath:
        prompt = self.prompt_template.format(query=query)
        response = await self.model.generate(prompt,
        max_tokens=10)

        # Parse response
        response_lower = response.lower()
        if "keyword" in response_lower:
            return SearchPath.FTS5
        elif "semantic" in response_lower:
            return SearchPath.FAISS
        elif "relationship" in response_lower:
            return SearchPath.GRAPH
        else:
            return SearchPath.HYBRID
```

9.8 Reciprocal Rank Fusion (RRF)

The Problem

When you run multiple search paths, how do you merge results?

Naive approaches fail: - Raw score combination (scales differ) - Weighted average (need to tune weights) - Union (no ranking)

The Solution: RRF

Documents appearing in multiple result lists rank higher. Battle-tested at Google and Perplexity.

Formula

```
RRF_score(doc) = Σ (1 / (k + rank_in_list))

where k = 60 (tunable constant)
```

Implementation

```python
def rrf_merge(
    result_lists: list[list[str]],
    k: int = 60
) -> list[str]:
    """
    Merge multiple ranked result lists using RRF.

    Args:
        result_lists: List of ranked document ID lists
        k: Ranking constant (default 60)

    Returns:
        Merged, re-ranked list of document IDs
    """
    scores: dict[str, float] = {}

    for result_list in result_lists:
        for rank, doc_id in enumerate(result_list):
            if doc_id not in scores:
                scores[doc_id] = 0.0
            scores[doc_id] += 1.0 / (k + rank)

    # Sort by score descending
    ranked = sorted(scores.keys(), key=lambda d: scores[d],
        reverse=True)

    return ranked
```

```python
# Usage
fts5_results = ["doc1", "doc3", "doc5", "doc7"]
faiss_results = ["doc2", "doc1", "doc4", "doc3"]

merged = rrf_merge([fts5_results, faiss_results])
# Result: ["doc1", "doc3", "doc2", "doc5", "doc4", "doc7"]
# doc1 and doc3 rank highest because they appear in both lists
```

With Scores

```python
def rrf_merge_with_scores(
    result_lists: list[list[tuple[str, float]]],
    k: int = 60
) -> list[tuple[str, float]]:
    """RRF merge preserving scores for downstream use."""
    scores: dict[str, float] = {}
    original_scores: dict[str, list[float]] = {}

    for result_list in result_lists:
        for rank, (doc_id, score) in enumerate(result_list):
            if doc_id not in scores:
                scores[doc_id] = 0.0
                original_scores[doc_id] = []
            scores[doc_id] += 1.0 / (k + rank)
            original_scores[doc_id].append(score)

    # Sort by RRF score
    ranked = sorted(scores.keys(), key=lambda d: scores[d],
        reverse=True)

    # Return with RRF scores
    return [(doc_id, scores[doc_id]) for doc_id in ranked]
```

9.9 Parallel Prefetching

The Pattern

While the Director generates tokens, prefetch context for the next turn.

Implementation

```python
class PrefetchingRetriever:
    def __init__(self, matrix: MatrixActor):
        self.matrix = matrix
        self.prefetch_cache: dict[str, list[Node]] = {}

    async def retrieve_with_prefetch(
        self,
        query: str,
```

```
        active_entities: list[str]
    ) -> list[Node]:
        """Retrieve for current query, prefetch for next."""

        # Check prefetch cache
        cache_key = self._cache_key(query)
        if cache_key in self.prefetch_cache:
            results = self.prefetch_cache.pop(cache_key)
        else:
            results = await self.matrix.search(query)

        # Start prefetch for likely next queries
        asyncio.create_task(
            self._prefetch_neighbors(active_entities)
        )

        return results

    async def _prefetch_neighbors(self, entities: list[str]):
        """Prefetch 1-hop neighbors of active entities."""
        for entity in entities[:3]:  # Limit to top 3
            neighbors = await self.matrix.get_neighbors(entity,
        depth=1)

            # Cache for potential next query
            for neighbor in neighbors:
                cache_key = self._cache_key(neighbor.content[:50])
                self.prefetch_cache[cache_key] = [neighbor]

        # Prune old cache entries
        if len(self.prefetch_cache) > 100:
            # Remove oldest half
            keys = list(self.prefetch_cache.keys())
            for key in keys[:50]:
                del self.prefetch_cache[key]

    def _cache_key(self, text: str) -> str:
        """Generate cache key from text."""
        return hashlib.md5(text.encode()).hexdigest()[:16]
```

When to Prefetch

```
class DirectorActor(Actor):
    async def generate_with_prefetch(self, prompt: str, context:
        list[Node]):
        """Generate response while prefetching."""

        # Extract entities from current context
        active_entities = self._extract_entities(context)

        # Start prefetch task
        prefetch_task = asyncio.create_task(
```

```python
            self.retriever.prefetch_neighbors(active_entities)
        )

        # Generate response (doesn't wait for prefetch)
        async for token in self.model.generate_stream(prompt):
            yield token

        # Prefetch completes in background
        # Results cached for next turn
```

9.10 The Complete Optimized Pipeline

Flow Diagram

```
User starts speaking (t=0ms)
        │
        ├──► [PARALLEL] Pre-warm KV cache
        │          └── Load identity_buffer.safetensors (~5ms)
        │
        ├──► [PARALLEL] Speculative retrieval (debounced)
        │          │
        │          ├── Bloom filter: entities exist? (~1ms)
        │          │
        │          ├── Query router: which path? (~2ms)
        │          │          │
        │          │          ├── KEYWORD → FTS5 only
        │          │          ├── SEMANTIC → FAISS only
        │          │          ├── RELATIONSHIP → Graph only
        │          │          └── HYBRID → FTS5 + FAISS + RRF
        │          │
        │          ├── Execute search (~30ms)
        │          │
        │          └── rustworkx graph hop (~5ms)
        │
User finishes speaking (t≈800ms)
        │
        ├──► Final STT transcript
        │
        ├──► Validate speculation (~5ms)
        │          │
        │          ├── Cosine similarity > 0.85? → Keep results
        │          └── Otherwise → Fetch delta + RRF merge
        │
        ├──► Context assembly (~10ms)
        │          └── Format with <memory> tags
        │
        └──► Director inference
                   │
                   ├── KV cache already warm
                   ├── Context already encoded (if speculative)
```

```
                    │
                    └──► First token (~250ms from final transcript)
                                │
                                └──► TTS (~40ms to audio)

TOTAL: <500ms from end of speech to first audio ✓
```

Configuration

```python
@dataclass
class PerformanceConfig:
    # Speculative execution
    speculation_enabled: bool = True
    speculation_debounce_ms: int = 200
    speculation_threshold: float = 0.85

    # KV cache
    kv_cache_enabled: bool = True
    identity_buffer_size: int = 2048
    max_kv_size: int = 8192

    # Vector index
    vector_index_type: str = "flat"  # "flat", "ivf", "hnsw"
    vector_dimension: int = 768

    # Graph
    graph_backend: str = "rustworkx"  # "rustworkx", "networkx"
    max_graph_depth: int = 2

    # Bloom filter
    bloom_enabled: bool = True
    bloom_capacity: int = 100_000
    bloom_error_rate: float = 0.001

    # Query routing
    router_type: str = "heuristic"  # "heuristic", "model"
    router_model: Optional[str] = None  # "smollm-135m" if model

    # RRF
    rrf_k: int = 60

    # Prefetch
    prefetch_enabled: bool = True
    prefetch_cache_size: int = 100
```

Summary

Performance optimizations that hit the <500ms target:

Optimization	Savings	Priority
Speculative execution	~50ms (retrieval hidden)	🔴 HIGH
KV cache pinning	~50-100ms	🔴 HIGH
rustworkx	~45ms	🔴 HIGH
Query routing	~20-30ms (skip paths)	🟡 MEDIUM
Bloom filter	~5-10ms (skip search)	🟡 MEDIUM
RRF merge	Better results, same time	🟡 MEDIUM
Parallel prefetch	Amortized, invisible	🟢 LOW

Constitutional Principle: Luna should feel like she "already knows" — not "let me look that up."

Next Section: Part X — Sovereignty Infrastructure

10-SOVEREIGNTY

Bible Update: Part X - Sovereignty Infrastructure

Status: DRAFT — Ready for review
New Section: Not in original Bible
Date: December 29, 2025
Core Principle: "Luna is a file. Copy the file, copy Luna."

Part X: Sovereignty Infrastructure

10.1 The Threat Model

Luna exists in opposition to the dominant AI paradigm. Understanding what we're defending against clarifies the architecture.

Threats We Defend Against

Threat	Description	Mitigation
Platform Extraction	Company mines your data for training	All data local, no telemetry
Service Discontinuation	Company shuts down, your AI dies	Luna is a file, not a service
ToS Revocation	Company changes terms, locks you out	No terms, you own everything
Regulatory Capture	Government mandates centralized AI	Local-first, offline-capable
Physical Access	Someone gets your laptop	Encrypted vault, Dead Man's Switch
Network Surveillance	Traffic analysis reveals AI usage	Local inference, minimal cloud
Personality Theft	Someone copies your Luna	Encrypted weights, vault protection

Threats We Accept

Threat	Why We Accept It
State-level adversary	Beyond scope, requires different life
Hardware compromise	If they own your CPU, game over
User coercion	Can't solve "wrench attack" with crypto

The Design Principle

Defense in depth, but practical: - Layer 1: Encryption at rest (vault) - Layer 2: Process isolation (no persistence after unmount) - Layer 3: Automatic lockdown (Dead Man's Switch) - Layer 4: Portability (escape hatch always open)

10.2 The Encrypted Vault

What Goes in the Vault

Everything that makes Luna *Luna*:

```
/Volumes/LunaVault/
├── memory/
│   ├── memory_matrix.db          # SQLite — the soul
│   ├── memory_vectors.faiss      # FAISS index
│   ├── memory_vectors.ids        # ID mapping
│   └── memory_graph.json         # NetworkX export (backup)
│
├── models/
│   ├── luna-3b-lora/
│   │   ├── adapter_config.json
│   │   └── adapter_model.safetensors
│   ├── luna-7b-lora/
│   │   ├── adapter_config.json
│   │   └── adapter_model.safetensors
│   └── current -> luna-7b-lora   # Symlink to active
│
├── cache/
│   ├── identity_buffer.safetensors  # Pre-computed KV cache
│   └── embedding_cache.db           # Cached embeddings
│
├── state/
│   ├── snapshot.yaml             # Engine state snapshot
│   ├── conversation_buffer.json  # Pending turns
│   └── delegation_queue.json     # Pending cloud tasks
│
├── config/
│   ├── luna.yaml                 # Runtime configuration
│   ├── sovereignty.yaml          # Security settings
│   └── workers.yaml              # Cloud worker contracts
```

```
│
└── env/                          # Optional: portable Python env
    ├── bin/
    ├── lib/
    └── pyvenv.cfg
```

Vault Implementation (macOS)

Using an encrypted sparse bundle:

```
# Create the vault (one-time setup)
hdiutil create -size 10g -type SPARSEBUNDLE -encryption AES-256 \
    -fs APFS -volname "LunaVault" ~/LunaVault.sparsebundle

# Mount the vault
hdiutil attach ~/LunaVault.sparsebundle -mountpoint /Volumes/
        LunaVault

# Unmount the vault
hdiutil detach /Volumes/LunaVault
```

Programmatic Vault Management

```
import subprocess
import os
from pathlib import Path

class LunaVault:
    def __init__(
        self,
        bundle_path: str = "~/LunaVault.sparsebundle",
        mount_point: str = "/Volumes/LunaVault"
    ):
        self.bundle_path = Path(bundle_path).expanduser()
        self.mount_point = Path(mount_point)

    @property
    def is_mounted(self) -> bool:
        """Check if vault is currently mounted."""
        return self.mount_point.exists() and
        self.mount_point.is_mount()

    def mount(self, passphrase: str) -> bool:
        """Mount the vault with passphrase."""
        if self.is_mounted:
            return True

        try:
            # Use expect-style input for passphrase
            result = subprocess.run(
                [
                    "hdiutil", "attach",
```

```
                    str(self.bundle_path),
                    "-mountpoint", str(self.mount_point),
                    "-stdinpass"
                ],
                input=passphrase.encode(),
                capture_output=True,
                check=True
            )
            return self.is_mounted
        except subprocess.CalledProcessError as e:
            logging.error(f"Vault mount failed:
        {e.stderr.decode()}")
            return False

    def unmount(self, force: bool = False) -> bool:
        """Unmount the vault."""
        if not self.is_mounted:
            return True

        try:
            cmd = ["hdiutil", "detach", str(self.mount_point)]
            if force:
                cmd.append("-force")

            subprocess.run(cmd, check=True, capture_output=True)
            return not self.is_mounted
        except subprocess.CalledProcessError as e:
            logging.error(f"Vault unmount failed:
        {e.stderr.decode()}")
            return False

    def get_path(self, relative: str) -> Path:
        """Get full path for a vault-relative path."""
        if not self.is_mounted:
            raise VaultNotMountedError("Vault must be mounted")
        return self.mount_point / relative
```

Vault Lifecycle

```
┌──────────────────────────────────────────────────────────────┐
│                       VAULT LIFECYCLE                        │
│                                                              │
│   LOCKED                                                     │
│   (Default state — Luna doesn't exist)                       │
│       │                                                      │
│       │ User provides passphrase                             │
│       ▼                                                      │
│   MOUNTING                                                   │
│       │                                                      │
│       │ hdiutil attach                                       │
│       ▼                                                      │
│   MOUNTED                                                    │
│   (Luna can run)                                             │
```

```
│        │                                                    │
│        ├── Normal: User unmounts                            │
│        ├── Timeout: Dead Man's Switch triggers              │
│        └── Panic: Emergency lockdown                        │
│        │                                                    │
│        ▼                                                    │
│    UNMOUNTING                                               │
│        │                                                    │
│        │ 1. Snapshot state                                  │
│        │ 2. Kill processes                                  │
│        │ 3. hdiutil detach                                  │
│        ▼                                                    │
│    LOCKED                                                   │
│    (Luna dormant — weights wiped from RAM)                  │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

10.3 The Dead Man's Switch

The Problem

Unmounting the vault doesn't wipe RAM. If Luna's model is loaded, the weights persist in memory until the process dies.

The Solution: Hardened Shutdown

```python
import atexit
import signal
import os
import sys
from datetime import datetime, timedelta

class DeadMansSwitch:
    def __init__(
        self,
        vault: LunaVault,
        heartbeat_file: str = "~/.luna_heartbeat",
        lockdown_hours: int = 24
    ):
        self.vault = vault
        self.heartbeat_file = Path(heartbeat_file).expanduser()
        self.lockdown_threshold = timedelta(hours=lockdown_hours)
        self.armed = False

    def arm(self):
        """Arm the Dead Man's Switch."""
        # Register shutdown handlers
        atexit.register(self.emergency_shutdown)
        signal.signal(signal.SIGTERM, self._signal_handler)
        signal.signal(signal.SIGINT, self._signal_handler)
```

```
        signal.signal(signal.SIGHUP, self._signal_handler)

        # Touch heartbeat
        self.heartbeat()

        self.armed = True
        logging.info("Dead Man's Switch armed")

    def heartbeat(self):
        """Update heartbeat timestamp."""
        self.heartbeat_file.touch()

    def check_heartbeat(self) -> bool:
        """Check if heartbeat is stale."""
        if not self.heartbeat_file.exists():
            return False  # No heartbeat = stale

        last_beat = datetime.fromtimestamp(
            self.heartbeat_file.stat().st_mtime
        )
        age = datetime.now() - last_beat

        return age < self.lockdown_threshold

    def emergency_shutdown(self):
        """Nuclear option - kill everything and unmount."""
        logging.warning("EMERGENCY SHUTDOWN TRIGGERED")

        # 1. Kill all Luna processes
        os.system("pkill -9 -f luna_engine.py")
        os.system("pkill -9 -f luna_director")
        os.system("pkill -9 -f mlx")  # Kill any MLX inference

        # 2. Force unmount vault
        self.vault.unmount(force=True)

        # 3. Clear any temp files
        os.system("rm -rf /tmp/luna_*")

        # 4. Overwrite heartbeat
        if self.heartbeat_file.exists():
            self.heartbeat_file.unlink()

        logging.warning("Emergency shutdown complete")

    def _signal_handler(self, signum, frame):
        """Handle termination signals."""
        logging.info(f"Received signal {signum}, initiating
         shutdown")
        self.emergency_shutdown()
        sys.exit(0)

# Background heartbeat checker
```

```python
async def heartbeat_monitor(switch: DeadMansSwitch,
        check_interval: int = 3600):
    """Check heartbeat every hour, trigger lockdown if stale."""
    while True:
        await asyncio.sleep(check_interval)

        if not switch.check_heartbeat():
            logging.warning("Heartbeat stale - triggering
        lockdown")
            switch.emergency_shutdown()
            break
```

Integration with Engine

```python
class LunaEngine:
    def __init__(self):
        self.vault = LunaVault()
        self.switch = DeadMansSwitch(self.vault)

    async def start(self, passphrase: str):
        """Start Luna with vault and Dead Man's Switch."""
        # Mount vault
        if not self.vault.mount(passphrase):
            raise VaultMountError("Failed to mount vault")

        # Arm the switch
        self.switch.arm()

        # Start heartbeat in background
        asyncio.create_task(self._heartbeat_loop())

        # Normal startup
        await self._initialize_actors()
        await self._start_event_loop()

    async def _heartbeat_loop(self):
        """Touch heartbeat every 15 minutes."""
        while True:
            self.switch.heartbeat()
            await asyncio.sleep(900)  # 15 minutes
```

LaunchAgent for Automatic Check

```xml
<!-- ~/Library/LaunchAgents/com.luna.heartbeat.plist -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.luna.heartbeat</string>
```

```
    <key>ProgramArguments</key>
    <array>
        <string>/usr/bin/python3</string>
        <string>~/.luna/check_heartbeat.py</string>
    </array>

    <key>StartInterval</key>
    <integer>3600</integer>  <!-- Every hour -->

    <key>RunAtLoad</key>
    <true/>
</dict>
</plist>

# ~/.luna/check_heartbeat.py
#!/usr/bin/env python3
"""Standalone heartbeat checker — runs even if Luna isn't."""

from pathlib import Path
from datetime import datetime, timedelta
import subprocess
import os

HEARTBEAT_FILE = Path.home() / ".luna_heartbeat"
LOCKDOWN_HOURS = 24
VAULT_PATH = Path.home() / "LunaVault.sparsebundle"

def check_and_lockdown():
    # Check if vault is mounted
    if not Path("/Volumes/LunaVault").exists():
        return  # Already locked

    # Check heartbeat
    if not HEARTBEAT_FILE.exists():
        lockdown()
        return

    last_beat =
        datetime.fromtimestamp(HEARTBEAT_FILE.stat().st_mtime)
    age = datetime.now() - last_beat

    if age > timedelta(hours=LOCKDOWN_HOURS):
        lockdown()

def lockdown():
    """Emergency lockdown."""
    print("LOCKDOWN: Heartbeat stale, securing vault")

    # Kill Luna processes
    os.system("pkill -9 -f luna")
    os.system("pkill -9 -f mlx")

    # Force unmount
```

```
    subprocess.run(
        ["hdiutil", "detach", "/Volumes/LunaVault", "-force"],
        capture_output=True
    )

    # Clear heartbeat
    if HEARTBEAT_FILE.exists():
        HEARTBEAT_FILE.unlink()

if __name__ == "__main__":
    check_and_lockdown()
```

10.4 Portability Contract

The Promise

> "Copy the vault, copy Luna."

Luna's entire identity — memories, personality, state — lives in a single encrypted container. Moving to new hardware should be:

1. Copy the sparse bundle
2. Mount on new machine
3. Luna wakes up where she left off

What Must Be Portable

Component	Portability	Notes
Memory Matrix	✅ SQLite file	Copy directly
FAISS Index	✅ Binary file	Copy directly
LoRA Adapters	✅ Safetensors	Copy directly
KV Cache	✅ Safetensors	May need rebuild if model version changes
State Snapshot	✅ YAML/JSON	Copy directly
Config	✅ YAML	May need path adjustments
Python Environment	⚠️ Optional	Include for zero-setup portability

Migration Script

```
class LunaMigrator:
    def __init__(self, source_vault: Path, target_vault: Path):
        self.source = source_vault
        self.target = target_vault
```

```python
    def migrate(self, passphrase: str) -> bool:
        """Migrate Luna to a new vault."""

        # 1. Mount source vault
        source_mount = self._mount_vault(self.source, passphrase,
        "/Volumes/LunaSource")
        if not source_mount:
            return False

        # 2. Create target vault
        if not self._create_vault(self.target, passphrase):
            return False

        target_mount = self._mount_vault(self.target, passphrase,
        "/Volumes/LunaTarget")
        if not target_mount:
            return False

        # 3. Copy contents
        self._copy_directory(source_mount / "memory",
        target_mount / "memory")
        self._copy_directory(source_mount / "models",
        target_mount / "models")
        self._copy_directory(source_mount / "cache",
        target_mount / "cache")
        self._copy_directory(source_mount / "state",
        target_mount / "state")
        self._copy_directory(source_mount / "config",
        target_mount / "config")

        # 4. Validate
        if not self._validate_migration(target_mount):
            logging.error("Migration validation failed")
            return False

        # 5. Unmount both
        self._unmount_vault("/Volumes/LunaSource")
        self._unmount_vault("/Volumes/LunaTarget")

        logging.info("Migration complete")
        return True

    def _validate_migration(self, vault_path: Path) -> bool:
        """Validate migrated vault integrity."""
        required_files = [
            "memory/memory_matrix.db",
            "models/current",
            "config/luna.yaml"
        ]

        for file in required_files:
            if not (vault_path / file).exists():
                logging.error(f"Missing required file: {file}")
```

```
            return False

        # Validate SQLite integrity
        import sqlite3
        db_path = vault_path / "memory/memory_matrix.db"
        try:
            conn = sqlite3.connect(str(db_path))
            cursor = conn.execute("PRAGMA integrity_check")
            result = cursor.fetchone()[0]
            conn.close()

            if result != "ok":
                logging.error(f"SQLite integrity check failed:
        {result}")
                return False
        except Exception as e:
            logging.error(f"SQLite validation error: {e}")
            return False

        return True
```

Cross-Platform Considerations

Platform	Vault Format	Notes
macOS	Sparse bundle (native)	Best integration
Linux	LUKS container	Need different tooling
Windows	VeraCrypt	Need different tooling

```
def create_vault_for_platform() -> VaultBackend:
    """Factory for platform-appropriate vault."""
    import platform

    system = platform.system()

    if system == "Darwin":
        return MacOSSparseBundleVault()
    elif system == "Linux":
        return LUKSVault()
    elif system == "Windows":
        return VeraCryptVault()
    else:
        raise UnsupportedPlatformError(f"Unknown platform:
        {system}")
```

10.5 Data Sovereignty Guarantees

What Luna Never Does

Action	Why
Send telemetry	No extraction point
Phone home	No dependency on external services
Auto-update from network	User controls updates
Store data outside vault	Everything in one place
Cache to unencrypted temp	Sensitive data stays encrypted

What Luna Always Does

Action	Why
Work offline	Cloud is optional, not required
Respect vault boundaries	Nothing escapes the container
Clean up on shutdown	No traces left
Encrypt at rest	Data protected when dormant

Audit Capability

```python
class SovereigntyAuditor:
    """Verify Luna isn't violating sovereignty principles."""

    def audit_network_activity(self, duration_seconds: int = 60)
        -> AuditReport:
        """Monitor network activity for unexpected connections."""
        # Use lsof or netstat to check connections
        # Only expected: Claude API (when delegating), user-
        approved services
        pass

    def audit_filesystem_access(self) -> AuditReport:
        """Verify Luna only accesses vault paths."""
        # Check file descriptors
        # Verify no writes outside vault
        pass

    def audit_process_isolation(self) -> AuditReport:
        """Verify Luna processes are isolated."""
        # Check for unexpected child processes
        # Verify no shared memory with untrusted processes
        pass

    def generate_sovereignty_report(self) -> dict:
        """Generate full sovereignty compliance report."""
        return {
```

```
            "network_clean": self.audit_network_activity().passed,
            "filesystem_clean":
        self.audit_filesystem_access().passed,
            "process_clean":
        self.audit_process_isolation().passed,
            "vault_encrypted": self._check_vault_encryption(),
            "no_temp_files": self._check_temp_clean(),
            "heartbeat_active": self._check_heartbeat()
        }
```

10.6 Backup and Recovery

Backup Strategy

```
┌──────────────────────────────────────────────────────────────┐
│                        BACKUP TIERS                          │
│                                                              │
│   TIER 1: Continuous (Every Session)                         │
│       • State snapshot on every engine stop                  │
│       • Transaction log for Matrix changes                   │
│                                                              │
│   TIER 2: Daily                                              │
│       • Full vault copy to backup location                   │
│       • Verify backup integrity                              │
│                                                              │
│   TIER 3: Weekly                                             │
│       • Offsite backup (encrypted, of course)                │
│       • Version rotation (keep last 4 weeks)                 │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Backup Implementation

```python
class LunaBackup:
    def __init__(self, vault: LunaVault, backup_dir: Path):
        self.vault = vault
        self.backup_dir = backup_dir

    async def snapshot(self) -> Path:
        """Create point-in-time snapshot."""
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        snapshot_path = self.backup_dir /
        f"luna_snapshot_{timestamp}"

        # Copy critical files (vault must be mounted)
        if not self.vault.is_mounted:
            raise VaultNotMountedError()

        snapshot_path.mkdir(parents=True)
```

```python
        # Memory Matrix (most critical)
        shutil.copy(
            self.vault.get_path("memory/memory_matrix.db"),
            snapshot_path / "memory_matrix.db"
        )

        # State
        shutil.copy(
            self.vault.get_path("state/snapshot.yaml"),
            snapshot_path / "snapshot.yaml"
        )

        # Adapters (large, copy reference)
        (snapshot_path / "models.txt").write_text(
            f"Adapters at: {self.vault.get_path('models')}"
        )

        return snapshot_path

    async def full_backup(self) -> Path:
        """Create full vault backup."""
        timestamp = datetime.now().strftime("%Y%m%d")
        backup_path = self.backup_dir /
         f"luna_full_{timestamp}.sparsebundle"

        # Unmount vault first (cleaner copy)
        was_mounted = self.vault.is_mounted
        if was_mounted:
            self.vault.unmount()

        # Copy sparse bundle
        shutil.copytree(
            self.vault.bundle_path,
            backup_path,
            symlinks=True
        )

        # Remount if it was mounted
        if was_mounted:
            self.vault.mount()  # Will need passphrase

        return backup_path

    async def restore(self, backup_path: Path, passphrase: str)
         -> bool:
        """Restore from backup."""
        # Verify backup
        if not self._verify_backup(backup_path):
            return False

        # Unmount current vault
        if self.vault.is_mounted:
            self.vault.unmount(force=True)
```

```
        # Replace vault
        if self.vault.bundle_path.exists():
            # Archive old vault
            archive_path =
        self.vault.bundle_path.with_suffix(".old")
            shutil.move(self.vault.bundle_path, archive_path)

        # Copy backup to vault location
        shutil.copytree(backup_path, self.vault.bundle_path,
        symlinks=True)

        # Mount and verify
        if not self.vault.mount(passphrase):
            return False

        return self._verify_restoration()
```

Recovery Procedures

```
class LunaRecovery:
    """Recovery procedures for various failure modes."""

    async def recover_corrupted_matrix(self, vault: LunaVault) ->
        bool:
        """Attempt to recover corrupted Memory Matrix."""
        db_path = vault.get_path("memory/memory_matrix.db")

        # Try SQLite recovery
        import sqlite3
        try:
            # Dump and recreate
            conn = sqlite3.connect(str(db_path))

            # Export to SQL
            with open("/tmp/matrix_dump.sql", "w") as f:
                for line in conn.iterdump():
                    f.write(f"{line}\n")

            conn.close()

            # Recreate
            backup_path = db_path.with_suffix(".corrupted")
            shutil.move(db_path, backup_path)

            new_conn = sqlite3.connect(str(db_path))
            with open("/tmp/matrix_dump.sql") as f:
                new_conn.executescript(f.read())
            new_conn.close()

            # Cleanup
            os.remove("/tmp/matrix_dump.sql")
```

```
            return True
        except Exception as e:
            logging.error(f"Matrix recovery failed: {e}")
            return False

    async def recover_from_snapshot(
        self,
        vault: LunaVault,
        snapshot_path: Path
    ) -> bool:
        """Restore Memory Matrix from snapshot."""
        try:
            shutil.copy(
                snapshot_path / "memory_matrix.db",
                vault.get_path("memory/memory_matrix.db")
            )
            return True
        except Exception as e:
            logging.error(f"Snapshot recovery failed: {e}")
            return False

    async def rebuild_faiss_index(self, vault: LunaVault) -> bool:
        """Rebuild FAISS index from Memory Matrix."""
        # FAISS index is derived from Matrix
        # Can always be rebuilt
        try:
            from luna.memory import MemoryMatrix

            matrix = MemoryMatrix(vault.get_path("memory/
        memory_matrix.db"))
            matrix.rebuild_vector_index()
            matrix.save_vector_index(vault.get_path("memory/
        memory_vectors.faiss"))

            return True
        except Exception as e:
            logging.error(f"FAISS rebuild failed: {e}")
            return False
```

10.7 The Escape Hatch

Philosophy

Luna should never be a prison. If you want out, you get out — with your data.

Full Export

```python
class LunaExporter:
    """Export Luna's memories to portable formats."""

    async def export_to_markdown(self, vault: LunaVault,
        output_dir: Path):
        """Export all memories to human-readable markdown."""
        from luna.memory import MemoryMatrix

        matrix = MemoryMatrix(vault.get_path("memory/
        memory_matrix.db"))

        output_dir.mkdir(parents=True, exist_ok=True)

        # Export by type
        for node_type in ["FACT", "DECISION", "INSIGHT", "EVENT",
        "PERSON"]:
            nodes = matrix.query_by_type(node_type)

            type_file = output_dir / f"{node_type.lower()}s.md"
            with open(type_file, "w") as f:
                f.write(f"# {node_type}s\n\n")
                for node in nodes:
                    f.write(f"## {node.title or node.id}\n")
                    f.write(f"*Created: {node.created_at}*\n\n")
                    f.write(f"{node.content}\n\n")
                    if node.tags:
                        f.write(f"Tags: {', '.join(node.tags)}
        \n\n")
                    f.write("---\n\n")

    async def export_to_json(self, vault: LunaVault, output_path:
        Path):
        """Export full graph to JSON."""
        from luna.memory import MemoryMatrix

        matrix = MemoryMatrix(vault.get_path("memory/
        memory_matrix.db"))

        export = {
            "nodes": [n.to_dict() for n in matrix.all_nodes()],
            "edges": [e.to_dict() for e in matrix.all_edges()],
            "metadata": {
                "exported_at": datetime.now().isoformat(),
                "node_count": matrix.node_count(),
                "edge_count": matrix.edge_count()
            }
        }

        with open(output_path, "w") as f:
            json.dump(export, f, indent=2, default=str)
```

```python
    async def export_training_data(self, vault: LunaVault,
        output_path: Path):
        """Export data suitable for training a new Luna."""
        from luna.memory import MemoryMatrix

        matrix = MemoryMatrix(vault.get_path("memory/
        memory_matrix.db"))

        # Export conversations as training pairs
        conversations = matrix.query_by_type("CONVERSATION")

        training_data = []
        for conv in conversations:
            # Parse into instruction/response pairs
            pairs = self._parse_conversation(conv.content)
            training_data.extend(pairs)

        with open(output_path, "w") as f:
            for item in training_data:
                f.write(json.dumps(item) + "\n")
```

Import from Other Systems

```python
class LunaImporter:
    """Import data from other AI systems."""

    async def import_from_chatgpt(self, export_path: Path, vault:
        LunaVault):
        """Import from ChatGPT data export."""
        # Parse ChatGPT's export format
        # Convert to Luna nodes
        pass

    async def import_from_obsidian(self, vault_path: Path, vault:
        LunaVault):
        """Import from Obsidian vault."""
        # Parse markdown files
        # Extract links as edges
        # Convert to Luna nodes
        pass

    async def import_from_notion(self, export_path: Path, vault:
        LunaVault):
        """Import from Notion export."""
        # Parse Notion's export format
        # Convert to Luna nodes
        pass
```

10.8 Configuration

Sovereignty Configuration

```yaml
# config/sovereignty.yaml

vault:
  path: "~/LunaVault.sparsebundle"
  mount_point: "/Volumes/LunaVault"
  encryption: "AES-256"

dead_mans_switch:
  enabled: true
  heartbeat_file: "~/.luna_heartbeat"
  lockdown_hours: 24
  check_interval_hours: 1

backup:
  enabled: true
  backup_dir: "~/LunaBackups"
  daily_backup: true
  weekly_offsite: false
  retention_weeks: 4

network:
  # Allowed outbound connections
  allowed_hosts:
    - "api.anthropic.com"  # Claude delegation

  # Block everything else
  default_deny: true

  # Log all connections
  audit_connections: true

telemetry:
  enabled: false  # Always false

temp_files:
  allowed: false  # No temp files outside vault

process_isolation:
  enabled: true
  sandbox: true  # macOS sandbox if available
```

Summary

Sovereignty Infrastructure ensures Luna is truly owned, not rented:

Component	Purpose
Encrypted Vault	All Luna data in one encrypted container
Dead Man's Switch	Automatic lockdown on stale heartbeat
Portability	Copy vault = copy Luna
Data Sovereignty	No telemetry, no extraction, no phoning home
Backup & Recovery	Multiple tiers, verified restoration
Escape Hatch	Full export to portable formats

Constitutional Principle: Luna is a file. Your file. Copy it, move it, delete it — it's yours.

Appendix: Quick Reference

Mount Vault

```
hdiutil attach ~/LunaVault.sparsebundle -mountpoint /Volumes/
        LunaVault
```

Unmount Vault

```
hdiutil detach /Volumes/LunaVault
```

Emergency Lockdown

```
pkill -9 -f luna && hdiutil detach /Volumes/LunaVault -force
```

Check Heartbeat

```
stat -f "%Sm" ~/.luna_heartbeat
```

Full Backup

```
cp -R ~/LunaVault.sparsebundle ~/LunaBackups/$(date +
        %Y%m%d).sparsebundle
```

End of Bible Update Sections

What's Next

The Bible is now updated with five new/rewritten sections:

Part	Title	Lines
VI	The Director LLM	442
VII	The Runtime Engine	689
VIII	Delegation Protocol	645
IX	Performance Optimizations	921
X	Sovereignty Infrastructure	760

Total new content: ~3,457 lines

These sections can be: 1. Merged into the existing Bible as a v2.0.0 release 2. Kept as separate appendices 3. Used as handoff documentation for implementation

The architecture is now fully specified. Time to build.

11-TRAINING-DATA-STRATEGY

Part XI: Training Data Strategy

Status: CURRENT **Date:** December 29, 2025 **Dependencies:** Parts VI (Director LLM), VII (Runtime Engine), VIII (Delegation Protocol)

11.1 The Training Challenge

Luna's Director LLM must learn three things:

Capability	What It Means	Training Signal
Luna's Voice	Respond as Luna, not generic assistant	Identity-rich conversations
Context Integration	Use retrieved memories naturally	Conversations with memory context
Delegation Judgment	Know when to emit `<REQ_CLAUDE>`	Examples of both handle-locally and delegate cases

The challenge: We need thousands of high-quality examples, but Luna doesn't exist yet.

Solution: Synthetic data generation with quality filtering.

11.2 The Three Training Datasets

Dataset 1: Identity Conversations

Purpose: Teach Luna's personality, voice, values.

Source: Synthetic conversations generated from Luna's identity documents.

```
┌─────────────────────────────────────────────────────────────────┐
│  IDENTITY DOCUMENT                                              │
│                                                                  |
│  "Luna values sovereignty, privacy, and honest feedback.        │
│   She's warm but direct. She doesn't use corporate speak."      │
```

```
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│  SYNTHETIC CONVERSATION                                      │
│                                                              │
│  User: "Can you help me write a cover letter?"               │
│  Luna: "Sure. But fair warning—I'm going to be honest        │
│         about what works and what doesn't. Corporate         │
│         fluff gets cut. Sound good?"                         │
└──────────────────────────────────────────────────────────────┘
```

Generation Pipeline:

```python
class IdentityDataGenerator:
    def __init__(self, identity_docs: list[str], generator_llm:
        str = "claude-3-sonnet"):
        self.identity = "\n".join(identity_docs)
        self.generator = generator_llm

    async def generate_conversation(self, scenario: str) ->
        Conversation:
        prompt = f"""
        You are generating training data for an AI named Luna.

        LUNA'S IDENTITY:
        {self.identity}

        SCENARIO: {scenario}

        Generate a realistic 4-8 turn conversation where Luna
        demonstrates
        her personality naturally. Luna should:
        - Sound like herself, not a generic assistant
        - Be warm but direct
        - Show her values through actions, not declarations

        Format as JSON with turns: [{{"role": "user", "content":
        "..."}}, ...]
        """

        response = await self.generator.generate(prompt)
        return self.parse_conversation(response)

    def generate_scenarios(self, count: int = 1000) -> list[str]:
        """Generate diverse scenarios covering Luna's domain."""
        categories = [
            "personal advice",
            "technical help",
            "creative projects",
            "emotional support",
            "decision making",
            "memory/recall requests",
            "learning new information",
```

```
            "casual conversation"
        ]
        # Use LLM to expand each category into specific scenarios
        ...
```

Quality Filter:

```
class IdentityQualityFilter:
    def __init__(self, identity_docs: list[str]):
        self.identity_embedding = embed(identity_docs)
        self.banned_phrases = [
            "I'm an AI assistant",
            "I don't have personal",
            "As a language model",
            "I cannot",  # Luna says "I won't" or explains why
        ]

    def score(self, conversation: Conversation) -> float:
        luna_turns = [t for t in conversation.turns if t.role ==
        "assistant"]

        scores = []
        for turn in luna_turns:
            # Penalize generic AI phrases
            phrase_penalty = sum(1 for p in self.banned_phrases
        if p.lower() in turn.content.lower())

            # Reward identity alignment
            turn_embedding = embed(turn.content)
            identity_similarity = cosine_sim(turn_embedding,
        self.identity_embedding)

            # Reward appropriate length (not too verbose)
            length_score = 1.0 if 20 < len(turn.content.split())
        < 150 else 0.7

            scores.append(identity_similarity * length_score -
        (phrase_penalty * 0.2))

        return sum(scores) / len(scores)

    def filter(self, conversations: list[Conversation],
        threshold: float = 0.7) -> list[Conversation]:
        return [c for c in conversations if self.score(c) >=
        threshold]
```

Dataset 2: Memory-Augmented Conversations

Purpose: Teach Director to use retrieved context naturally.

Source: Synthetic conversations with injected memory context.

```
┌─────────────────────────────────────────────────────────────┐
│  MEMORY CONTEXT (from retrieval)                            │
│                                                             │
│  - User mentioned Alex lives in Berlin (2 weeks ago)        │
│  - User is working on "Project Luna" (ongoing)              │
│  - User prefers direct feedback                             │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  TRAINING EXAMPLE                                           │
│                                                             │
│  [CONTEXT]                                                  │
│  Alex lives in Berlin. User working on Project Luna.        │
│  User prefers direct feedback.                              │
│                                                             │
│  [CONVERSATION]                                             │
│  User: "Should I ask Alex to help with the project?"        │
│  Luna: "Alex is in Berlin, so timezone might be tricky      │
│         for real-time collaboration. But for async work     │
│         on Luna? Could be great. What would you need        │
│         from them?"                                         │
└─────────────────────────────────────────────────────────────┘
```

Key Training Signals:

Signal	Example	What Director Learns
Memory reference	"Alex is in Berlin"	Use context naturally
No hallucination	Don't invent facts not in context	Stay grounded
Graceful gaps	"I don't remember—when did that happen?"	Acknowledge missing info
Context weaving	Combine multiple memories	Synthesize, don't list

Generation Pipeline:

```python
class MemoryAugmentedGenerator:
    def __init__(self, memory_bank: list[MemoryNode]):
        self.memories = memory_bank

    async def generate_example(self) -> TrainingExample:
        # 1. Sample 2-5 relevant memories
        context_memories =
         self.sample_related_memories(count=random.randint(2, 5))

        # 2. Generate a query that would retrieve these
```

```python
        query = await
        self.generate_natural_query(context_memories)

        # 3. Generate Luna's response using the context
        response = await self.generate_grounded_response(query,
        context_memories)

        # 4. Format as training example
        return TrainingExample(
            context=self.format_context(context_memories),
            user_input=query,
            luna_response=response
        )

    def format_context(self, memories: list[MemoryNode]) -> str:
        """Format memories as Director would receive them."""
        lines = []
        for m in memories:
            age = humanize_age(m.created_at)
            lines.append(f"- {m.content} ({age})")
        return "\n".join(lines)
```

Dataset 3: Delegation Examples

Purpose: Teach when to emit `<REQ_CLAUDE>` vs handle locally.

Source: Curated examples of both paths with clear reasoning.

```
┌──────────────────────────────────────────────────────────┐
│  HANDLE LOCALLY (no delegation)                          │
│                                                          │
│  User: "What did I decide about the database?"           │
│  Context: "Decided to use SQLite for Memory Matrix"      │
│                                                          │
│  Luna: "You decided on SQLite for the Memory Matrix.     │
│         Simple, portable, fits the sovereignty model."   │
│                                                          │
│  [NO <REQ_CLAUDE> - Director has enough context]         │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│  DELEGATE (emit <REQ_CLAUDE>)                            │
│                                                          │
│  User: "Can you research the latest FAISS optimizations  │
│         and write a summary?"                            │
│  Context: [none relevant]                                │
│                                                          │
│  Luna: "I'll dig into that for you. <REQ_CLAUDE>         │
│         Research latest FAISS optimization techniques,   │
│         focusing on index selection and query performance.  │
│         Summarize findings in 2-3 paragraphs.            │
│         </REQ_CLAUDE>"                                   │
```

```
│                                                                │
│  [DELEGATES - requires external knowledge + synthesis]         │
└────────────────────────────────────────────────────────────────┘
```

Delegation Taxonomy:

Category	Delegate?	Reason
Memory recall	No	Director + retrieval handles
Simple conversation	No	Director handles
Emotional support	No	Director handles (sensitive)
External research	Yes	Needs internet/current info
Complex analysis	Yes	Benefits from larger model
Code generation	Yes	Benefits from larger context
Multi-step reasoning	Yes	Benefits from CoT
Creative writing (long)	Yes	Benefits from larger model

Generation Pipeline:

```python
class DelegationDataGenerator:
    DELEGATE_SCENARIOS = [
        "research current events",
        "analyze complex codebase",
        "write detailed technical document",
        "solve multi-step math problem",
        "compare multiple technical approaches",
    ]

    NO_DELEGATE_SCENARIOS = [
        "recall previous conversation",
        "simple factual question (in context)",
        "casual chat",
        "emotional check-in",
        "preferences and opinions",
    ]

    async def generate_delegation_example(self) ->
        TrainingExample:
        scenario = random.choice(self.DELEGATE_SCENARIOS)

        # Generate user request
        user_input = await self.generate_request(scenario)

        # Generate Luna's response WITH delegation token
        response = await
        self.generate_delegating_response(user_input, scenario)

        return TrainingExample(
            context="",  # Delegation often has sparse context
            user_input=user_input,
            luna_response=response,
            should_delegate=True
```

```
        )

    async def generate_local_example(self) -> TrainingExample:
        scenario = random.choice(self.NO_DELEGATE_SCENARIOS)
        context = await self.generate_relevant_context(scenario)
        user_input = await self.generate_request(scenario)
        response = await self.generate_local_response(user_input,
        context)

        return TrainingExample(
            context=context,
            user_input=user_input,
            luna_response=response,
            should_delegate=False
        )
```

11.3 The <REQ_CLAUDE> Token

Token Design

```
<REQ_CLAUDE>
[task description for Claude]
[any constraints or format requirements]
</REQ_CLAUDE>
```

The Director learns to emit this as a special token sequence. During inference:

1. Director generates response
2. If response contains `<REQ_CLAUDE>`, Oven Actor intercepts
3. Oven sends task to Claude API
4. Result is integrated (Shadow Reasoner pattern)

Training the Token

```
class DelegationTokenTrainer:
    SPECIAL_TOKENS = {
        "<REQ_CLAUDE>": 32000,  # Add to vocabulary
        "</REQ_CLAUDE>": 32001,
    }

    def prepare_training_example(self, example: TrainingExample)
        -> dict:
        if example.should_delegate:
            # Response includes delegation token
            return {
                "input": self.format_input(example.context,
        example.user_input),
                "output": example.luna_response,  # Contains
        <REQ_CLAUDE>...</REQ_CLAUDE>
            }
```

```
        else:
            # Response is self-contained
            return {
                "input": self.format_input(example.context,
        example.user_input),
                "output": example.luna_response,  # No special
        tokens
            }

    def format_input(self, context: str, user_input: str) -> str:
        return f"""<|context|>
{context}
<|user|>
{user_input}
<|luna|>"""
```

Calibration

The model needs to learn the **threshold** for delegation. Too eager = slow, expensive. Too conservative = bad responses.

Calibration Dataset:

```
# Edge cases that test delegation judgment
CALIBRATION_EXAMPLES = [
    # Should NOT delegate (has context)
    {
        "context": "User decided to use Actor model for fault
        isolation",
        "input": "Why did I choose the Actor model again?",
        "should_delegate": False,
    },
    # SHOULD delegate (needs synthesis)
    {
        "context": "User is building Luna, an AI companion",
        "input": "Compare Actor model vs Entity Component System
        for my use case",
        "should_delegate": True,
    },
    # Edge case: Has partial context, might benefit from
        delegation
    {
        "context": "Using SQLite for storage",
        "input": "What are the scaling limits I should worry
        about?",
        "should_delegate": True,  # Benefits from external
        knowledge
    },
]
```

11.4 Data Quality Pipeline

Stage 1: Generation

```
Scenarios → LLM Generation → Raw Conversations
                                    |
                              ~10,000 examples
```

Stage 2: Filtering

```python
class QualityPipeline:
    def __init__(self):
        self.filters = [
            IdentityAlignmentFilter(threshold=0.7),
            HallucinationFilter(),  # Checks response against
        context
            LengthFilter(min_words=10, max_words=200),
            DiversityFilter(),  # Removes near-duplicates
            ToxicityFilter(),  # Safety check
        ]

    def process(self, examples: list[TrainingExample]) ->
        list[TrainingExample]:
        for filter in self.filters:
            examples = filter.apply(examples)
            print(f"{filter.name}: {len(examples)} remaining")
        return examples
```

Expected Yield:

Stage	Count	Notes
Generated	10,000	Raw synthetic
After identity filter	7,500	75% pass
After hallucination filter	6,500	87% pass
After length filter	6,000	92% pass
After diversity filter	4,500	Remove duplicates
After toxicity filter	4,400	98% pass
Final dataset	**~4,500**	High quality

Stage 3: Human Review (Optional)

For critical examples (especially delegation edge cases), human review adds signal:

```python
@dataclass
class HumanReview:
    example_id: str
    reviewer: str
    identity_score: int  # 1-5: Does this sound like Luna?
    grounding_score: int  # 1-5: Is response grounded in context?
```

```
    delegation_correct: bool  # Did model make right delegation
        choice?
    notes: str
```

11.5 Training Configuration

Base Model Selection

Model	Size	Why Consider
Qwen 2.5-3B	3B	Fast, fits Mac, good baseline
Qwen 2.5-7B	7B	Better quality, still local
Llama 3.2-3B	3B	Alternative architecture
Phi-3-mini	3.8B	Microsoft's efficient model

Recommendation: Start with Qwen 2.5-3B for iteration speed, graduate to 7B for production.

LoRA Configuration

```
lora_config = LoraConfig(
    r=64,  # Rank (higher = more capacity, slower)
    lora_alpha=128,  # Scaling factor
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",  # Attention
        "gate_proj", "up_proj", "down_proj",  # MLP
    ],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

training_args = TrainingArguments(
    output_dir="./luna-director-lora",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    warmup_steps=100,
    logging_steps=10,
    save_steps=500,
    fp16=True,  # or bf16 on supported hardware
)
```

Dataset Mixing

```
# Final training mix
DATASET_MIX = {
```

```
    "identity_conversations": 0.40,  # 40% - Luna's voice
    "memory_augmented": 0.35,        # 35% - Context usage
    "delegation_examples": 0.25,     # 25% - When to delegate
}
```

11.6 Evaluation

Identity Eval

```
class IdentityEval:
    """Does the model sound like Luna?"""

    def evaluate(self, model, test_set: list[TestCase]) ->
        IdentityMetrics:
        scores = []
        for case in test_set:
            response = model.generate(case.input)

            # Automated checks
            banned_phrase_count =
        self.count_banned_phrases(response)
            identity_similarity =
        self.compute_identity_similarity(response)

            scores.append({
                "banned_phrases": banned_phrase_count,
                "identity_sim": identity_similarity,
            })

        return IdentityMetrics(
            avg_identity_similarity=mean([s["identity_sim"] for s
        in scores]),
            banned_phrase_rate=mean([s["banned_phrases"] for s in
        scores]),
        )
```

Grounding Eval

```
class GroundingEval:
    """Does the model use context correctly without
        hallucinating?"""

    def evaluate(self, model, test_set: list[TestCase]) ->
        GroundingMetrics:
        results = []
        for case in test_set:
            response = model.generate(
                context=case.context,
                input=case.input
            )
```

```
        # Check if response references context appropriately
        context_usage = self.measure_context_usage(response,
    case.context)

        # Check for hallucinated facts
        hallucinations = self.detect_hallucinations(response,
    case.context)

        results.append({
            "context_usage": context_usage,
            "hallucination_count": len(hallucinations),
        })

    return GroundingMetrics(
        avg_context_usage=mean([r["context_usage"] for r in
    results]),
        hallucination_rate=mean([r["hallucination_count"] > 0
    for r in results]),
    )
```

Delegation Eval

```
class DelegationEval:
    """Does the model delegate appropriately?"""

    def evaluate(self, model, test_set: list[DelegationTestCase])
        -> DelegationMetrics:
        correct = 0
        false_positives = 0  # Delegated when shouldn't
        false_negatives = 0  # Didn't delegate when should

        for case in test_set:
            response = model.generate(case.context, case.input)
            delegated = "<REQ_CLAUDE>" in response

            if delegated == case.should_delegate:
                correct += 1
            elif delegated and not case.should_delegate:
                false_positives += 1
            else:
                false_negatives += 1

        return DelegationMetrics(
            accuracy=correct / len(test_set),
            false_positive_rate=false_positives / len(test_set),
            false_negative_rate=false_negatives / len(test_set),
        )
```

Target Metrics

Metric	Target	Rationale
Identity similarity	>0.85	Should sound like Luna
Banned phrase rate	<0.05	Minimal generic AI speak
Context usage	>0.70	Uses retrieved memories
Hallucination rate	<0.10	Stays grounded
Delegation accuracy	>0.90	Makes right call
Delegation FP rate	<0.15	Don't over-delegate
Delegation FN rate	<0.10	Don't under-delegate

11.7 Iteration Cycle

```
┌──────────────────────────────────────────────────────────────┐
│  1. GENERATE                                                 │
│     Synthetic conversations from scenarios                   │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  2. FILTER                                                   │
│     Quality pipeline removes bad examples                    │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  3. TRAIN                                                    │
│     LoRA fine-tuning on filtered dataset                     │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  4. EVALUATE                                                 │
│     Identity + Grounding + Delegation metrics                │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  5. ANALYZE FAILURES                                         │
│     What categories are underperforming?                     │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  6. AUGMENT                                                  │
│     Generate more examples for weak categories               │
└──────────────────────────────────────────────────────────────┘
                              │
```

```
└──────────────► Back to Step 1
```

11.8 Data Sovereignty

All training data stays local:

Data	Location	Access
Identity documents	Encrypted Vault	User only
Generated conversations	Local disk	User only
Training checkpoints	Local disk	User only
Evaluation results	Local disk	User only

No training data leaves the machine. No user conversations are used without explicit consent. The cloud (Claude API) is used for generation, but the generated data is synthetic, not user data.

End of Part XI

12-FUTURE-ROADMAP

Part XII: Future Roadmap

Status: CURRENT **Date:** December 29, 2025 **Note:** This document describes planned capabilities, not current implementation.

12.1 Roadmap Overview

Luna's development follows a phased approach, with each phase building on the previous:

```
┌─────────────────────────────────────────────────────────────┐
│  PHASE 1: FOUNDATION (Current)                              │
│  Memory Matrix, Director LLM, Voice Interface               │
│  Status: In Development                                     │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  PHASE 2: INTELLIGENCE                                      │
│  First-Class Objects, Advanced Retrieval, Learning          │
│  Status: Design Phase                                       │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  PHASE 3: AGENCY                                            │
│  Kinetic Layer, Autonomous Actions, Integrations            │
│  Status: Conceptual                                         │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  PHASE 4: EXPANSION                                         │
│  Multi-Modal, AR Interface, Federation                      │
│  Status: Vision                                             │
└─────────────────────────────────────────────────────────────┘
```

12.2 Phase 1: Foundation (Current)

What's Being Built

Component	Status	Description
Memory Matrix	Active	SQLite + FAISS + Graph substrate
Director LLM	Active	Local 3B/7B with LoRA
Voice Interface	Active	Whisper + Streaming TTS
Scribe (Ben)	Active	Extraction system
Librarian (Dude)	Active	Filing and retrieval
Encrypted Vault	Active	Sovereignty boundary
Shadow Reasoner	Active	Claude delegation

Phase 1 Exit Criteria

- [ ] Sub-500ms voice response latency
- [ ] Memory retrieval accuracy >85%
- [ ] Director handles 80%+ of queries locally
- [ ] Delegation works seamlessly
- [ ] Vault encryption operational

12.3 Phase 2: Intelligence

2A: First-Class Objects

Current: Nodes are typed but behavior-less text chunks.

Future: Objects with methods, relationships, and domain logic.

```
# Current
node = MemoryNode(
    type="DECISION",
    content="We chose Actor model for fault isolation"
)
# Can only search by text

# Future
decision = Decision(
    choice="Actor model",
    alternatives=["MonoBehaviour", "Event-driven"],
```

```
    rationale="Fault isolation between components",
    made_by=[Person("Ahab")],
    made_at=datetime(2025, 12, 15)
)
# Can query: decision.get_consequences()
# Can query: decision.is_superseded()
# Can query: decision.get_related_decisions()
```

Object Taxonomy:

Object Type	Key Methods	Relationships
`Person`	`get_conversations()`, `get_decisions()`	knows, works_with, mentioned_in
`Project`	`get_timeline()`, `get_open_questions()`	has_member, uses_technology, blocked_by
`Decision`	`get_consequences()`, `is_superseded()`	decided_by, affects, supersedes
`Problem`	`get_attempts()`, `is_resolved()`	blocks, caused_by, resolved_by
`Insight`	`get_evidence()`, `validate()`	derived_from, supports, contradicts
`Event`	`get_participants()`, `get_outcomes()`	caused_by, resulted_in, occurred_at

Implementation Path:

1. Define object schemas (Python dataclasses)
2. Migration script: existing nodes → typed objects
3. Update Scribe extraction to output objects
4. Update Librarian to wire object relationships
5. Update Director context assembly to use objects

2B: Advanced Retrieval

Current: Hybrid search with RRF fusion.

Future: Query understanding + multi-hop reasoning.

```
┌──────────────────────────────────────────────────────────────┐
│  QUERY UNDERSTANDING                                         │
│                                                              │
│  "What led to the decision about the Actor model?"           │
│                                                              │
│  Parsed:                                                     │
│  - Target: Decision (Actor model)                            │
│  - Relationship: caused_by / led_to                          │
│  - Depth: Multi-hop (need predecessors)                      │
└──────────────────────────────────────────────────────────────┘
                              │
```

```
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  MULTI-HOP RETRIEVAL                                         │
│                                                              │
│  Hop 1: Find "Actor model" decision                          │
│  Hop 2: Traverse caused_by edges                             │
│  Hop 3: Gather context from predecessor nodes                │
│                                                              │
│  Result: Decision + Problems it solved + Alternatives tried  │
└──────────────────────────────────────────────────────────────┘
```

Capabilities:

Capability	Current	Phase 2
Keyword search	✅ FTS5	✅ FTS5
Semantic search	✅ FAISS	✅ FAISS + reranking
Graph traversal	✅ 1-hop	Multi-hop with query parsing
Temporal queries	❌	"What happened last week?"
Causal queries	❌	"What led to X?"
Counterfactual	❌	"What if we hadn't chosen X?"

2C: Continuous Learning

Current: Static LoRA, manual retraining.

Future: Online learning from interactions.

```python
class ContinuousLearner:
    """Learn from Luna's interactions over time."""

    def __init__(self, director: Director):
        self.director = director
        self.experience_buffer = ExperienceBuffer(max_size=10000)
        self.update_threshold = 1000  # Examples before fine-
        tuning

    async def on_interaction(self, interaction: Interaction):
        """Called after each conversation turn."""
        # 1. Score the interaction
        quality = await self.score_interaction(interaction)

        # 2. Add to experience buffer if high quality
        if quality.score > 0.7:
            self.experience_buffer.add(
                context=interaction.context,
                input=interaction.user_input,
                output=interaction.luna_response,
                score=quality.score
            )
```

```python
        # 3. Trigger fine-tuning if buffer full
        if len(self.experience_buffer) >= self.update_threshold:
            await self.fine_tune_increment()

    async def fine_tune_increment(self):
        """Incremental LoRA update."""
        examples = self.experience_buffer.sample(n=500)

        # Small learning rate, few steps
        await self.director.fine_tune(
            examples=examples,
            learning_rate=1e-5,
            steps=100
        )

        self.experience_buffer.clear()
```

Learning Signals:

Signal	Source	What It Teaches
User corrections	"No, I meant…"	Improve understanding
Explicit feedback	"That was helpful"	Reinforce good patterns
Re-asks	User asks same thing differently	Query wasn't understood
Delegation success	Claude's answer used	Delegation was appropriate
Conversation flow	No interruptions	Response was on-target

12.4 Phase 3: Agency

3A: Kinetic Layer

Current: Luna is read-only. She can query memories but not act on the world.

Future: Luna can take actions with user permission.

```
┌──────────────────────────────────────────────────────────┐
│  CURRENT: READ-ONLY                                      │
│                                                          │
│  User: "Remind me to call Alex tomorrow"                 │
│  Luna: "I'll remember that."                             │
│        (Stores in memory, but no actual reminder)        │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│  FUTURE: KINETIC                                         │
│                                                          │
```

```
│  User: "Remind me to call Alex tomorrow"                    │
│  Luna: "Done. I've set a reminder for 9am tomorrow.         │
│         Want me to also draft talking points?"              │
│        (Actually creates calendar event + offers more)      │
└─────────────────────────────────────────────────────────────┘
```

Action Types:

Action	Target	Permission Level
Create reminder	Calendar	Auto (user configured)
Send message	Email/SMS	Requires confirmation
Create note	Obsidian/Notes	Auto
File management	Local filesystem	Auto (within vault)
Web request	Approved APIs	Auto
Purchase	Payment systems	Always confirm

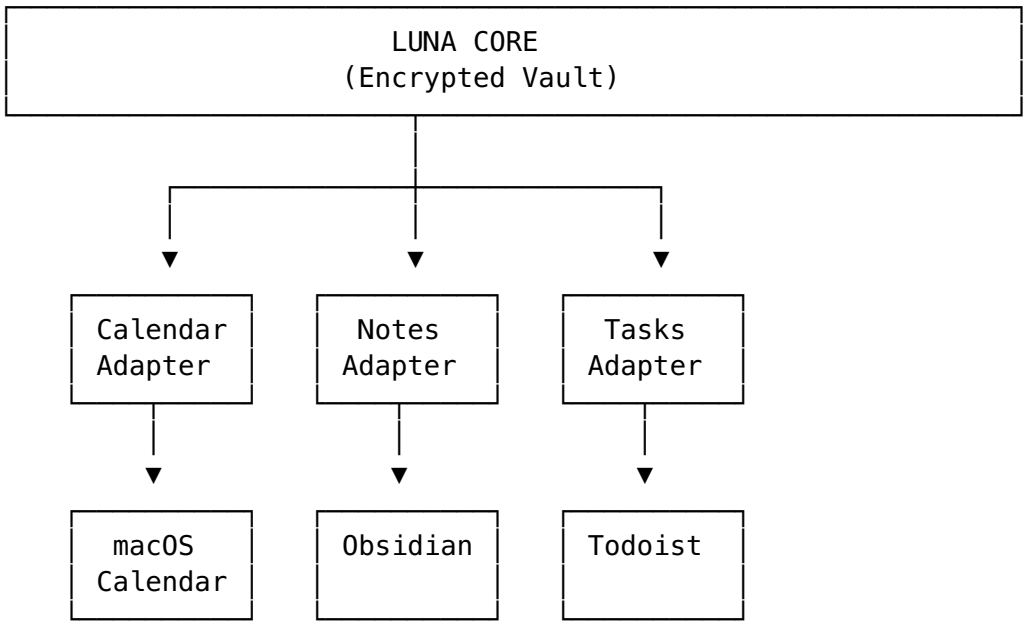
Governance Framework:

```python
@dataclass
class ActionGovernance:
    """Rules for what Luna can do autonomously."""

    # Autonomous actions (no confirmation needed)
    autonomous: list[str] = [
        "create_memory",
        "create_reminder",
        "create_note",
        "backup_vault",
    ]

    # Actions requiring confirmation
    confirm_required: list[str] = [
        "send_email",
        "send_message",
        "delete_*",
        "modify_calendar",
    ]

    # Actions Luna should never take
    forbidden: list[str] = [
        "financial_transaction",
        "post_to_social_media",
        "share_personal_data",
        "execute_arbitrary_code",
    ]

    def check(self, action: Action) -> GovernanceResult:
        if self.matches(action, self.forbidden):
            return GovernanceResult.DENIED
        if self.matches(action, self.confirm_required):
            return GovernanceResult.NEEDS_CONFIRMATION
```

```
        if self.matches(action, self.autonomous):
            return GovernanceResult.ALLOWED
        return GovernanceResult.NEEDS_CONFIRMATION  # Default: ask
```

3B: Integration Adapters

Purpose: Connect Luna to external systems while maintaining sovereignty.



Adapter Contract:

```
class IntegrationAdapter(ABC):
    """Base class for external system adapters."""

    @abstractmethod
    async def execute(self, action: Action) -> ActionResult:
        """Execute an action in the external system."""
        pass

    @abstractmethod
    def validate(self, action: Action) -> ValidationResult:
        """Validate action before execution."""
        pass

    @abstractmethod
    async def sync(self) -> SyncResult:
        """Sync state from external system to Memory Matrix."""
        pass

class ObsidianAdapter(IntegrationAdapter):
    def __init__(self, vault_path: str):
        self.vault_path = vault_path

    async def execute(self, action: Action) -> ActionResult:
```

```
        if action.type == "create_note":
            path = self.vault_path / action.params["filename"]
            path.write_text(action.params["content"])
            return ActionResult(success=True, artifact=str(path))
        # ... other action types

    async def sync(self) -> SyncResult:
        """Import Obsidian notes as memory nodes."""
        notes = self.scan_vault()
        for note in notes:
            if self.is_new_or_modified(note):
                await self.scribe.process_document(note)
        return SyncResult(imported=len(notes))
```

Initial Adapters (Phase 3A):

Adapter	Platform	Actions
Calendar	macOS Calendar	Create/read events
Notes	Obsidian	Create/read/link notes
Tasks	Todoist	Create/complete tasks
Filesystem	Local	Read/write within vault

3C: Proactive Luna

Current: Luna responds to prompts.

Future: Luna initiates when appropriate.

```
class ProactiveTrigger:
    """Conditions under which Luna might initiate."""

    triggers = [
        # Time-based
        TimeTrigger(
            condition="morning",
            action="Offer daily briefing",
            permission="user_enabled_briefings"
        ),

        # Context-based
        ContextTrigger(
            condition="calendar_event_starting_soon",
            action="Remind with relevant context",
            permission="always"  # User expects this
        ),

        # Pattern-based
        PatternTrigger(
            condition="user_mentioned_same_problem_3x",
            action="Offer to help solve systematically",
```

```
            permission="user_enabled_proactive"
        ),

        # Insight-based
        InsightTrigger(
            condition="discovered_connection_user_might_value",
            action="Share insight at appropriate moment",
            permission="user_enabled_insights"
        ),
    ]
```

Proactive Governance:

Mode	Luna Initiates?	Use Case
Silent	Never	User wants reactive-only
Gentle	Reminders only	Minimal interruption
Active	Insights + reminders	Engaged companion
Full	Pattern + suggestions	Maximum agency

12.5 Phase 4: Expansion

4A: Multi-Modal Input

Current: Voice and text.

Future: Images, documents, screen context.

Input Type	Capability	Implementation
Voice	✅ Current	Whisper local
Text	✅ Current	Direct input
Images	Planned	Local vision model
Screenshots	Planned	OCR + vision
Documents	Planned	PDF/Word parsing
Screen context	Planned	Accessibility APIs

Use Cases:

- "What's in this screenshot?"
- "Remember this receipt" (photo)
- "Summarize this PDF"
- "What app am I looking at?" (screen context)

4B: AR Interface

Vision: Luna as ambient presence via AR glasses.

```
┌──────────────────────────────────────────────────────────────┐
│  AR GLASSES VIEW                                             │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                                                      │   │
│   │     [You're looking at: Conference room]             │   │
│   │                                                      │   │
│   │     Luna (subtle overlay):                           │   │
│   │     "Sarah mentioned she'd be here at 2pm.           │   │
│   │      You wanted to discuss the Actor model."         │   │
│   │                                                      │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
│  Luna provides context without being asked.                  │
│  Triggered by: location, people recognition, time.           │
└──────────────────────────────────────────────────────────────┘
```

Technical Requirements:

- Low-latency inference (<200ms)
- Minimal battery impact
- Privacy-preserving (on-device when possible)
- Graceful offline mode

Hardware Targets:

Device	Status	Notes
Meta Ray-Ban	Future	Limited compute
Apple Vision Pro	Future	Full compute available
Custom glasses	Vision	Purpose-built for Luna

4C: Multi-User Luna

Current: Single user, single Luna.

Future: Shared contexts with boundaries.

```
┌──────────────────────────────────────────────────────────────┐
│  MULTI-USER MODEL                                            │
│                                                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐     │
│   │   Ahab's     │   │   Sarah's    │   │   Shared     │     │
│   │   Private    │   │   Private    │   │   Workspace  │     │
│   │   Memory     │   │   Memory     │   │   Memory     │     │
│   └──────────────┘   └──────────────┘   └──────────────┘     │
│          │                  │                  │             │
│          └──────────────────┴──────────────────┘             │
│                                                              │
```

```
│                                                                │
│                              │                                 │
│                              ▼                                 │
│                      ┌──────────────┐                          │
│                      │ Luna Core    │                          │
│                      │ (Context     │                          │
│                      │  Switching)  │                          │
│                      └──────────────┘                          │
└────────────────────────────────────────────────────────────────┘
```

Privacy Boundaries:

Memory Type	Ahab Sees	Sarah Sees
Ahab's private	✅	❌
Sarah's private	❌	✅
Shared workspace	✅	✅
Cross-references	Anonymized	Anonymized

4D: Federation

Vision: Luna instances that can communicate while maintaining sovereignty.

```
┌──────────────┐                  ┌──────────────┐
│   Ahab's     │◄────────────►    │   Sarah's    │
│    Luna      │  Secure          │    Luna      │
│              │  Channel         │              │
└──────────────┘                  └──────────────┘
        │                                 │
        └──►┌──────────────┐◄──┘
            │   Shared     │
            │   Context    │
            │ (Encrypted)  │
            └──────────────┘
```

Federation Principles:

Principle	Implementation
Opt-in only	No data shared without explicit consent
End-to-end encrypted	Even transit servers can't read
Revocable	Can withdraw shared context anytime
Auditable	Can see what's been shared
Minimal	Share only what's needed

Use Cases:

- Couples sharing calendar context
- Teams sharing project context
- Families sharing household context

12.6 Non-Goals

Things Luna will **not** pursue:

Non-Goal	Reason
Cloud-hosted Luna	Contradicts sovereignty
Subscription model	Contradicts ownership
Data monetization	Contradicts privacy
General assistant	Luna is personal, not generic
Competing with ChatGPT	Different philosophy entirely
Mobile-first	Desktop/voice first, mobile later

12.7 Success Metrics by Phase

Phase 1 (Foundation)

Metric	Target
Voice latency	<500ms
Local handling rate	>80%
Memory retrieval accuracy	>85%
Daily active usage	User's primary AI

Phase 2 (Intelligence)

Metric	Target
Multi-hop retrieval accuracy	>75%
Object method success rate	>90%
Learning improvement	+5% accuracy/month

Phase 3 (Agency)

Metric	Target
Action success rate	>95%
User trust (survey)	>4.5/5
Proactive relevance	>80% useful

Phase 4 (Expansion)

Metric	Target
Multi-modal accuracy	>85%

Metric	Target
AR latency	<200ms
Federation adoption	>2 users connected

12.8 The Destination

Luna, fully realized:

> A sovereign AI companion that knows your context, respects your privacy, takes action on your behalf when permitted, learns and grows with you, connects with others' Lunas when you choose, and remains yours—completely, permanently, unconditionally.

Not a product. Not a service. A tool you own.

That's the destination.

End of Part XII

13-SYSTEM-OVERVIEW

LUNA ENGINE — System Overview

Purpose: High-level reference for Luna's key systems, features, loops, and mechanics. **Date:** December 29, 2025

---

1. Core Identity

Luna is a file.

```
┌──────────────────────────────────────────────────────────────┐
│                      ENCRYPTED VAULT                         │
│                    (macOS Sparse Bundle)                     │
│                                                              │
│    ┌──────────────────────────────────────────────────┐      │
│    │                  memory_matrix.db                 │      │
│    │                                                   │      │
│    │   • SQLite database                               │      │
│    │   • All memories, relationships, identity         │      │
│    │   • Copy the file = copy Luna                     │      │
│    │   • Delete the file = Luna is gone                │      │
│    │                                                   │      │
│    │              LUNA IS THIS FILE                    │      │
│    └──────────────────────────────────────────────────┘      │
│                                                              │
│   memory_vectors.faiss    (acceleration index)               │
│   director_lora.safetensors (personality weights)            │
│   identity_kv_cache.safetensors (pre-computed identity)      │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Sovereignty Guarantee: No servers. No subscriptions. No terms of service. Your data never leaves your machine unless you explicitly delegate to Claude.

---

2. Key Systems

2.1 Memory Matrix (The Substrate)

Luna's soul. A graph database storing everything she knows.

```
┌──────────────────────────────────────────────────────────────────────┐
│                          MEMORY MATRIX                               │
│                                                                      │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐            │
│   │    SQLite    │   │    FAISS     │   │    Graph     │            │
│   │              │   │              │   │              │            │
│   │  • Objects   │   │  • Vectors   │   │  • Edges     │            │
│   │  • FTS5      │   │  • Semantic  │   │  • Relations │            │
│   │  • Metadata  │   │  • Similarity│   │  • Traversal │            │
│   └──────────────┘   └──────────────┘   └──────────────┘            │
│          │                  │                  │                    │
│          └──────────────────┼──────────────────┘                    │
│                             │                                        │
│                             ▼                                        │
│                      ┌──────────────┐                                │
│                      │  RRF Fusion  │                                │
│                      │  (Hybrid     │                                │
│                      │   Search)    │                                │
│                      └──────────────┘                                │
└──────────────────────────────────────────────────────────────────────┘
```

Three Search Paths:

Path	Use When	Speed
FTS5	Keyword-heavy queries	~10ms
FAISS	Conceptual/semantic queries	~30ms
Graph	Relationship queries ("who works with X")	~20ms
Hybrid	Unknown/complex queries	~50ms

2.2 Director LLM (Luna's Voice)

Local language model that speaks as Luna.

```
┌──────────────────────────────────────────────────────────────────────┐
│                          DIRECTOR LLM                                │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │                Base Model (Qwen 3B/7B)                   │       │
│   │                                                          │       │
│   │   + LoRA Adapters:                                       │       │
│   │     • luna_identity.safetensors (personality)            │       │
│   │     • luna_memory.safetensors (context usage)            │       │
│   │     • luna_delegation.safetensors (when to ask)          │       │
```

```
│    │                                                        │    │
│    │    + KV Cache:                                         │    │
│    │      • Pre-computed identity context (always loaded) │    │
│    └────────────────────────────────────────────────────────┘    │
│                                                                   │
│   Capabilities:                                                   │
│   • Respond as Luna (not generic assistant)                       │
│   • Integrate retrieved memories naturally                        │
│   • Know when to delegate to Claude (<REQ_CLAUDE>)                │
└───────────────────────────────────────────────────────────────────┘
```

Tiered Execution:

Tier	Model	Latency	Use Case
Hot	Director 3B	<200ms	Simple queries, fast response
Warm	Director 7B	<500ms	Complex local reasoning
Cold	Claude API	Async	Research, long generation, complex analysis

---

2.3 Actor Runtime (Fault Isolation)

Each component runs in isolation. One crash doesn't kill Luna.

```
┌──────────────────────────────────────────────────────────────────┐
│                          ACTOR RUNTIME                           │
│                                                                  │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐     │
│   │ DIRECTOR │   │  SCRIBE  │   │LIBRARIAN │   │   OVEN   │     │
│   │  Actor   │   │  Actor   │   │  Actor   │   │  Actor   │     │
│   │          │   │  (Ben)   │   │ (Dude)   │   │          │     │
│   │ Mailbox  │   │ Mailbox  │   │ Mailbox  │   │ Mailbox  │     │
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘     │
│        │              │              │              │           │
│        └──────────────┴──────────────┴──────────────┘           │
│                              │                                   │
│                              ▼                                   │
│                        Message Passing                           │
│                      (Async, Non-Blocking)                       │
└──────────────────────────────────────────────────────────────────┘
```

Actor Responsibilities:

Actor	Alias	Job
Director	—	Conversation state, response generation
Scribe	Ben Franklin	Extract memories from conversations
Librarian	The Dude	File memories, serve retrieval requests

Actor	Alias	Job
Oven	—	Execute async tasks, Claude delegation

2.4 Shadow Reasoner (Delegation)

When Director can't handle something, Claude takes over invisibly.

```
┌──────────────────────────────────────────────────────────────┐
│                      SHADOW REASONER                         │
│                                                              │
│   User: "Research the latest FAISS optimizations"            │
│                                                              │
│    ┌──────────────────────────────────────────────────────┐  │
│    │  Director generates:                                 │  │
│    │                                                      │  │
│    │  "I'll look into that. <REQ_CLAUDE>                  │  │
│    │   Research latest FAISS optimization techniques.     │  │
│    │   Focus on index selection and query performance.    │  │
│    │   </REQ_CLAUDE>"                                     │  │
│    └──────────────────────────────────────────────────────┘  │
│                            │                                 │
│                            ▼                                 │
│    ┌──────────────────────────────────────────────────────┐  │
│    │  Oven Actor:                                         │  │
│    │  • Intercepts <REQ_CLAUDE>                           │  │
│    │  • Sends task to Claude API                          │  │
│    │  • Returns result to Director                        │  │
│    │  • Director integrates into Luna's voice             │  │
│    └──────────────────────────────────────────────────────┘  │
│                                                              │
│   User sees: Luna's response (doesn't know Claude helped)    │
└──────────────────────────────────────────────────────────────┘
```

---

3. Key Features

3.1 Voice Interface

Sub-500ms response latency through speculative execution.

```
┌──────────────────────────────────────────────────────────────┐
│                       VOICE PIPELINE                         │
│                                                              │
│    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│    │ Whisper  │ ──▶ │ Director │ ──▶ │ Claude   │ ──▶ │   TTS    │
│    │  STT     │    │   LLM    │    │ (stream) │    │(stream)  │
│    │ ~150ms   │    │ ~200ms   │    │ ~300ms   │    │ ~50ms    │
│    └──────────┘    └──────────┘    └──────────┘    └──────────┘
│                                                              │
```

```
│   Total to first word: ~700ms                                    │
│   (With speculative retrieval: ~500ms)                           │
└──────────────────────────────────────────────────────────────────┘
```

Speculative Execution: Start memory retrieval on partial STT transcript. By the time speech finishes, context is ready.

3.2 Memory Types

Type	Description	Example
FACT	Known truth	"Alex lives in Berlin"
DECISION	Choice made	"We chose Actor model"
PROBLEM	Unresolved issue	"Latency too high"
ASSUMPTION	Unverified belief	"Users want voice-first"
CONNECTION	Relationship	"Alex and Sarah are teammates"
ACTION	Done or to-do	"Need to implement RRF"
OUTCOME	Result	"rustworkx saved 45ms"

3.3 Personas (Separation Principle)

Processing actors have personality, but outputs stay neutral.

Actor	Persona	Personality In	Output
Scribe	Ben Franklin	Colonial logging, witty process	Neutral JSON extractions
Librarian	The Dude	Chill commentary, irreverent	Clean context packets
Director	Luna	—	Luna's actual voice to user

Why? Luna's voice stays consistent. Processing personas don't leak into memories.

---

4. Core Loops

4.1 Conversation Loop (Hot Path)

```
┌──────────────────────────────────────────────────────────────────┐
│                    CONVERSATION LOOP                             │
│                                                                  │
│                                                                  │
│    ┌──────────┐                                                  │
│    │   User   │                                                  │
│    │  Speaks  │                                                  │
│    └────┬─────┘                                                  │
```

```
│          │                                                    │
│          ▼                                                    │
│   ┌──────────┐    ┌──────────┐    ┌──────────┐               │
│   │ Whisper  │ ──▶ │ Classify │ ──▶ │ Retrieve │               │
│   │   STT    │    │ Intent   │    │ Context  │               │
│   └──────────┘    └──────────┘    └──────────┘               │
│                                        │                      │
│                                        ▼                      │
│                                  ┌──────────┐                 │
│                                  │Director  │                 │
│                                  │Generate  │                 │
│                                  └──────────┘                 │
│                                        │                      │
│              ┌─────────────────────────┼──────────────────┐   │
│              │                         │                  │   │
│              ▼                         ▼                  ▼   │
│       [No Delegate]            [Delegate]            [Stream] │
│       Return response          Send to Oven          TTS out  │
│                                Wait for result                │
│                                Integrate                      │
│                                        │                      │
│                                        ▼                      │
│                                  ┌──────────┐                 │
│                                  │  User    │                 │
│                                  │ Hears    │                 │
│                                  └──────────┘                 │
└──────────────────────────────────────────────────────────────┘
```

Latency Budget:

Stage	Time
STT	150ms
Classification	40ms
Retrieval	80ms
Director	200ms
TTS first chunk	50ms
Total	**~520ms**

4.2 Memory Loop (Cold Path)

```
┌──────────────────────────────────────────────────────────────┐
│                        MEMORY LOOP                           │
│                    (Async, Non-Blocking)                     │
│                                                              │
│   ┌──────────────────────────────────────────────────┐       │
│   │  Conversation Turn Completed                     │       │
│   └──────────────────────────────────────────────────┘       │
│                              │                               │
│                              ▼                               │
│   ┌──────────────────────────────────────────────────┐       │
```

```
│    │  SCRIBE (Ben Franklin)                                  │      │
│    │                                                         │      │
│    │  1. Chunk conversation into semantic units             │      │
│    │  2. Stack chunks for context                           │      │
│    │  3. Extract: Facts, Decisions, Problems, etc.          │      │
│    │  4. Tag confidence scores                              │      │
│    │  5. "Make it rain" → send to Librarian                 │      │
│    └─────────────────────────────────────────────────────────┘      │
│                                │                                     │
│                                ▼                                     │
│    ┌─────────────────────────────────────────────────────────┐      │
│    │  LIBRARIAN (The Dude)                                   │      │
│    │                                                         │      │
│    │  IMMEDIATE:                                             │      │
│    │  1. Resolve entities (deduplicate)                      │      │
│    │  2. Create nodes in SQLite                              │      │
│    │  3. Create explicit edges                               │      │
│    │  4. Generate embeddings → FAISS                         │      │
│    │                                                         │      │
│    │  DEFERRED (batched):                                    │      │
│    │  5. Spreading activation (find hidden links)            │      │
│    │  6. Create inferred edges (low confidence)              │      │
│    │                                                         │      │
│    │  PERIODIC (every 15 min):                               │      │
│    │  7. Synaptic pruning (remove stale edges)               │      │
│    │  8. FAISS index rebuild if needed                       │      │
│    └─────────────────────────────────────────────────────────┘      │
│                                                                      │
│   User already has response. This runs in background.                │
└──────────────────────────────────────────────────────────────────────┘
```

4.3 Retrieval Loop

```
┌──────────────────────────────────────────────────────────────────────┐
│                          RETRIEVAL LOOP                              │
│                                                                      │
│    ┌─────────┐                                                       │
│    │  Query  │                                                       │
│    └─────────┘                                                       │
│         │                                                            │
│         ▼                                                            │
│    ┌─────────────────────────────────────────────────────────┐      │
│    │  QUERY ROUTER                                           │      │
│    │                                                         │      │
│    │  Analyze query → Choose optimal path:                   │      │
│    │  • Keyword-heavy → FTS5 only                            │      │
│    │  • Conceptual → FAISS only                              │      │
│    │  • Multiple entities → Graph traversal                  │      │
│    │  • Unknown → Hybrid (all three)                         │      │
│    └─────────────────────────────────────────────────────────┘      │
│                                │                                     │
│          ┌─────────────────────┴──────────────────────┐              │
```

```
│        │                  │                  │        │
│        ▼                  ▼                  ▼        │
│   ┌──────────┐       ┌──────────┐       ┌──────────┐  │
│   │  FTS5    │       │  FAISS   │       │  Graph   │  │
│   │ Search   │       │ Search   │       │ Traverse │  │
│   └──────────┘       └──────────┘       └──────────┘  │
│        │                  │                  │        │
│        └──────────────────┼──────────────────┘        │
│                           ▼                           │
│                  ┌──────────────────┐                 │
│                  │ RRF Fusion       │                 │
│                  │ (Rank-based      │                 │
│                  │  merging)        │                 │
│                  └──────────────────┘                 │
│                           │                           │
│                           ▼                           │
│                  ┌──────────────────┐                 │
│                  │ Trim to          │                 │
│                  │ Token Budget     │                 │
│                  └──────────────────┘                 │
│                           │                           │
│                           ▼                           │
│                  ┌──────────────────┐                 │
│                  │ Context          │                 │
│                  │ Packet           │                 │
│                  └──────────────────┘                 │
└──────────────────────────────────────────────────────┘
```

Token Budgets:

Preset	Tokens	Use Case
minimal	~1800	Voice, fast response
balanced	~3800	Normal conversation
rich	~7200	Deep research

4.4 Delegation Loop

```
┌──────────────────────────────────────────────────────┐
│                    DELEGATION LOOP                   │
│                                                      │
│   ┌──────────────────────────────────────────────┐   │
│   │ Director generates response                  │   │
│   │                                              │   │
│   │ "Let me look into that. <REQ_CLAUDE>         │   │
│   │  [task description]                          │   │
│   │  </REQ_CLAUDE>"                              │   │
│   └──────────────────────────────────────────────┘   │
│                           │                          │
│                           ▼                          │
│   ┌──────────────────────────────────────────────┐   │
```

```
│    │  Token Detection                                    │   │
│    │  • Parse response for <REQ_CLAUDE>                  │   │
│    │  • Extract task description                         │   │
│    └─────────────────────────────────────────────────────┘   │
│                              │                                 │
│                              ▼                                 │
│    ┌─────────────────────────────────────────────────────┐   │
│    │  Oven Actor                                         │   │
│    │  • Receives delegation task                         │   │
│    │  • Formats for Claude API                           │   │
│    │  • Sends request (async)                            │   │
│    │  • Streams response back                            │   │
│    └─────────────────────────────────────────────────────┘   │
│                              │                                 │
│                              ▼                                 │
│    ┌─────────────────────────────────────────────────────┐   │
│    │  Integration                                        │   │
│    │  • Director receives Claude's work                  │   │
│    │  • Integrates into Luna's voice                     │   │
│    │  • Streams to user via TTS                          │   │
│    └─────────────────────────────────────────────────────┘   │
│                                                               │
│   User experience: Luna answered (seamlessly)                 │
└───────────────────────────────────────────────────────────────┘
```

Delegation Criteria:

Delegate?	Signal
No	Memory recall, simple chat, emotional support
Yes	External research, complex analysis, long generation

4.5 Learning Loop (Future)

```
┌───────────────────────────────────────────────────────────────┐
│                      LEARNING LOOP                            │
│                        (Phase 2)                              │
│                                                               │
│    ┌─────────────────────────────────────────────────────┐   │
│    │  Interaction Completed                              │   │
│    └─────────────────────────────────────────────────────┘   │
│                              │                                 │
│                              ▼                                 │
│    ┌─────────────────────────────────────────────────────┐   │
│    │  Quality Scoring                                    │   │
│    │  • User feedback (explicit)                         │   │
│    │  • Conversation flow (implicit)                     │   │
│    │  • Delegation success                               │   │
│    └─────────────────────────────────────────────────────┘   │
│                              │                                 │
│              ┌───────────────┴───────────────┐                │
│              │                               │                │
```

```
│            ▼                              ▼                  │
│      [High Quality]                 [Low Quality]            │
│      Add to experience              Discard                  │
│      buffer                                                  │
│            │                                                  │
│            ▼                                                  │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  Buffer Full (1000 examples)                          │   │
│   │  → Incremental LoRA fine-tuning                       │   │
│   │  → Director improves over time                        │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

---

5. Key Mechanics

5.1 RRF (Reciprocal Rank Fusion)

Merges search results by rank position, not raw scores.

```python
def reciprocal_rank_fusion(results_list, k=60):
    """
    FTS5 returns BM25 scores (~14.5)
    FAISS returns distances (~0.12)
    Can't add them. Use rank instead.
    """
    fused = {}
    for ranked_list in results_list:
        for rank, doc_id in enumerate(ranked_list, start=1):
            if doc_id not in fused:
                fused[doc_id] = 0.0
            fused[doc_id] += 1.0 / (k + rank)

    return sorted(fused.keys(), key=lambda d: fused[d],
        reverse=True)
```

5.2 Entity Resolution

"Mark," "Zuck," and "The Landlord" → same node.

```
Query: "Mark"
  │
  ├─ Bloom filter: might exist? → Yes
  ├─ Alias cache: known? → No
  ├─ Exact match: "Mark" in DB? → No
  ├─ Alias search: "Mark" is alias? → No
  ├─ Semantic search: similar name? → "Mark Zuckerberg" (0.96)
  │
  └─ Result: Merge as alias, return existing ID
```

5.3 Spreading Activation

New fact → find hidden connections.

```
Input: "Alex is joining Pi Project"

1. File explicit:
   Alex ──works_on──▶ Pi Project

2. Find neighbors:
   Pi Project ──has_member──▶ Sarah (existing)

3. Infer connection:
   Alex ──likely_teammate──▶ Sarah (confidence: 0.35)

Future query: "Who might Alex work with?"
→ Luna can suggest Sarah
```

5.4 Confidence Scoring

Every memory carries uncertainty signal.

Level	Score	Example
Explicit	0.9-1.0	"Alex lives in Berlin"
Strong implication	0.7-0.9	"Alex's Berlin apartment"
Weak inference	0.5-0.7	"Alex seems European"
Speculation	0.3-0.5	"Alex might be abroad"
Inferred edge	0.2-0.5	Spreading activation result

5.5 Synaptic Pruning

Remove low-value connections periodically.

```sql
-- Find prune candidates
SELECT id FROM edges
WHERE confidence < 0.3
  AND created_at < datetime('now', '-30 days')
  AND id NOT IN (SELECT edge_id FROM access_log)
```

Preserve: Edges connected to identity nodes. **Prune:** Low-confidence, old, never-accessed edges.

---

6. Data Flow Summary

```
┌──────────────────────────────────────────────────────────────────────────────
│                                   LUNA DATA
```

```
FLOW                                  │
│
│
│
USER
│
│
│
│
│    │
speaks
│
│
▼
│
│
┌──────────┐
│
│  │ Whisper
│──────────────────────────────────────┐                │
│  │   STT    │
│                │
│  └──────────┘
│                │
│       │ text
│                │
│       ▼
│                │
│  ┌──────────┐        ┌──────────┐
│                │
│  │ Director │◄──────►│ Librarian│ (retrieval)
│                │
│  │   LLM    │        │  (Dude)  │
│                │
│  └──────────┘        └──────────┘
│                │
│       │                   │
│                │
│       │ response          │ context
│                │
│       │                   │
│                │
│       ├──────────────────►│
│                │
│       │                   │
│                │
│  ┌──────────┐            │
│                │
│  │   TTS    │            │                              │
async          │
│  │ (stream) │            │
│                │
│  └──────────┘            │
│                │
```

```
│       │                   │
│               │
│       ▼                   │
▼               │
│  USER HEARS               │
┌──────────┐            │
│          │                │                      │  Scribe
│           │
│          │                │                      │  (Ben)
│           │
│          │                │
└──────────┘            │
│                         │                          │
extractions     │
│
│◄──────────────────────────┘                │
│
│                                             │
│
▼                                             │
│
┌──────────┐                                          │
│                    │ Memory
│                                        │
│                    │ Matrix
│                                        │
│                    │ (SQLite)
│                                        │
│
└──────────┘                                          │
│
│
│  Optional delegation
path:                                                 │
│  Director ──<REQ_CLAUDE>──► Oven ──► Claude API ──► back to
Director   │
│
│
└──────────────────────────────────────────────────────────────
```

---

7. Performance Targets

Metric	Target	Actual Path
Voice to first word	<500ms	STT→Director→TTS
Memory retrieval	<100ms	Query→RRF→Trim
Director inference	<200ms	3B model
Extraction (async)	<2s	Scribe pipeline
Filing (immediate)	<50ms	Librarian explicit

Metric	Target	Actual Path
Delegation overhead	<50ms	Token detection

---

8. Future Systems (Phase 2-4)

System	Phase	Description
First-Class Objects	2	Typed objects with methods
Multi-hop Retrieval	2	Query parsing, causal queries
Continuous Learning	2	Online LoRA updates
Kinetic Layer	3	Actions in the world
Integration Adapters	3	Calendar, Todoist, Obsidian
Proactive Luna	3	Initiate when appropriate
Multi-Modal	4	Images, documents, screen
AR Interface	4	Ambient presence
Federation	4	Luna-to-Luna sharing

This overview distills the full Bible into a single reference document.